

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



Đồ án 2

PHÁT TRIỂN DIỄN ĐÀN HỖ TRỢ LẬP TRÌNH
TÍCH HỢP HỆ THỐNG CHẤM BÀI TỰ ĐỘNG
VÀ CHATBOT HỖ TRỢ

Giảng viên hướng dẫn:
TS. Đỗ Thị Thanh Tuyền

Nhóm sinh viên thực hiện

Nguyễn Quý Phong

23521169

LỜI CẢM ƠN

Kính thưa cô Đỗ Thị Thanh Tuyền,

Trước hết, nhóm em xin gửi lời cảm ơn chân thành và sâu sắc đến cô - người đã trực tiếp hướng dẫn, tận tình chỉ bảo và đồng hành cùng nhóm em trong suốt quá trình thực hiện Đồ án 1 cũng như tiếp tục hỗ trợ nhóm trong Đồ án 2 với đề tài “Phát triển diễn đàn hỗ trợ lập trình tích hợp hệ thống chấm bài tự động và chatbot hỗ trợ”.

Trong quá trình thực hiện đồ án, cô đã luôn định hướng cho nhóm về mặt chuyên môn, góp ý cho nhóm trong việc xác định bài toán, phân tích yêu cầu, thiết kế hệ thống và triển khai sản phẩm theo hướng thực tế, khoa học và hiệu quả hơn. Những góp ý, nhận xét và sự chỉ dẫn tận tình của cô không chỉ giúp nhóm hoàn thiện đồ án, mà còn giúp em nâng cao tư duy phát triển phần mềm, khả năng làm việc nhóm và cách tiếp cận một sản phẩm công nghệ có tính ứng dụng.

Bên cạnh đó, nhóm em cũng xin chân thành cảm ơn quý thầy cô trong khoa và nhà trường đã truyền đạt những kiến thức nền tảng cũng như chuyên ngành quý báu trong suốt quá trình học tập. Những kiến thức đó là cơ sở quan trọng để nhóm có thể vận dụng vào quá trình nghiên cứu, xây dựng và phát triển hệ thống trong đồ án này.

Mặc dù nhóm đã cố gắng trong quá trình thực hiện, nhưng do kiến thức, kinh nghiệm và thời gian còn hạn chế, đồ án chắc chắn không tránh khỏi những thiếu sót. Nhóm em rất mong nhận được những nhận xét, góp ý từ cô và quý thầy cô để có thể tiếp tục hoàn thiện sản phẩm, đồng thời rút kinh nghiệm cho quá trình học tập, nghiên cứu và phát triển sau này.

Nhóm em xin chân thành cảm ơn!

Nhóm sinh viên thực hiện

Nguyễn Quý Phong

MỤC LỤC

TÓM TẮT ĐỒ ÁN.....	1
Chương 1. GIỚI THIỆU ĐỀ TÀI	3
1.1. Tên đề tài.....	3
1.2. Lý do chọn đề tài.....	3
1.3. Mục tiêu của đề tài	4
1.4. Phạm vi của đề tài	4
1.5. Đối tượng sử dụng.....	5
1.6. Phương pháp thực hiện	5
1.7. Kết quả mong đợi.....	6
Chương 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	7
2.1. Cơ sở lý thuyết	7
2.1.1. Mô hình Client - Server	7
2.1.2. Kiến trúc nhiều thành phần trong ứng dụng web.....	8
2.1.3. RESTful API	9
2.1.4. Sandbox.....	9
2.2. Công nghệ sử dụng.....	10
2.2.1. Svelte/SvelteKit	10
2.2.2. Node.js và ExpressJS	11
2.2.3. Python	12
2.2.4. Prisma ORM	13
2.2.5. PostgreSQL	14
2.2.6. Docker	15

2.2.7. Socket.IO.....	16
2.2.8. BullMQ	17
2.2.9. Redis.....	18
2.2.10. k6.....	19
Chương 3. PHÂN TÍCH THIẾT KẾ HỆ THỐNG.....	20
3.1. Khảo sát và phân tích yêu cầu.....	20
3.2. Xác định yêu cầu của hệ thống	22
3.2.1. Yêu cầu chức năng	22
3.2.2. Yêu cầu phi chức năng	23
3.3. Sơ đồ use case	24
3.3.1. Sơ đồ Use case quản lý người dùng.....	25
3.3.2. Sơ đồ Use case quản lý bài viết và bài viết đã lưu.....	26
3.3.3. Sơ đồ Use case quản lý danh sách người theo dõi và đang theo dõi	26
3.3.4. Sơ đồ Use case quản lý thử thách lập trình.....	26
3.3.5. Sơ đồ Use case quản lý chuyên mục và chủ đề	27
3.3.6. Sơ đồ Use case quản lý báo cáo và khiếu nại	27
3.4. Đặc tả use case	27
3.4.1. Đặc tả Use case thêm bài viết	28
3.4.2. Đặc tả Use case tham gia thử thách lập trình.....	29
3.4.3. Đặc tả Use case thêm thử thách lập trình.....	31
3.4.4. Đặc tả Use case hỏi đáp với AI	33
3.4.5. Đặc tả Use case xử lý báo cáo khiếu nại.....	34
3.4.6. Đặc tả Use case tham gia cuộc trò chuyện.....	36

3.5. Thiết kế kiến trúc hệ thống.....	38
3.5.1. Kiến trúc tổng thể.....	38
3.5.2. Mô hình phân lớp trong Backend.....	40
3.6. Thiết kế luồng xử lý chính	40
3.6.1. Luồng đăng nhập và xác thực người dùng.....	40
3.6.2. Luồng đăng bài và kiểm duyệt nội dung.....	41
3.6.3. Luồng hỏi đáp với AI	41
3.6.4. Luồng nộp bài và đưa vào hàng đợi chấm bài	42
3.6.5. Luồng xử lý chấm bài tự động	42
3.7. Thiết kế cơ sở dữ liệu.....	44
3.7.1. Nhóm bảng người dùng và cá nhân hóa.....	44
3.7.2. Nhóm bảng diễn đàn và tương tác nội dung	45
3.7.3. Nhóm bảng trò chuyện thời gian thực.....	45
3.7.4. Nhóm bảng thử thách lập trình và chấm bài tự động.....	46
3.7.5. Nhóm bảng AI và gợi ý học tập	47
3.7.6. Nhóm bảng báo cáo và kiểm duyệt.....	47
3.7.7. Mô tả chi tiết các bảng	48
Chương 4. XÂY DỰNG VÀ TRIỂN KHAI ỨNG DỤNG	65
4.1. Xây dựng giao diện người dùng.....	65
4.1.1. Màn hình đăng ký	65
4.1.2. Màn hình đăng nhập.....	66
4.1.3. Màn hình trang hồ sơ cá nhân	67
4.1.4. Màn hình trang thảo luận	68

4.1.5. Màn hình chi tiết bài đăng.....	69
4.1.6. Màn hình trang thử thách lập trình.....	70
4.1.7. Màn hình chi tiết thử thách lập trình.....	71
4.1.8. Màn hình trang nhấn tin	72
4.1.9. Màn hình hỏi đáp với chatbot	73
4.1.10. Màn hình trang học tập	74
4.1.11. Màn hình trang quản lý người dùng.....	75
4.1.12. Màn hình trang quản lý chuyên mục và chủ đề	76
4.1.13. Màn hình trang quản lý thử thách lập trình.....	76
4.1.14. Màn hình thêm thử thách lập trình.....	77
4.1.15. Màn hình quản lý báo cáo khiếu nại	78
4.1.16. Màn hình xử lý báo cáo khiếu nại.....	79
4.2. Xử lý nghiệp vụ hệ thống.....	79
4.2.1. Xử lý đăng nhập và xác thực người dùng	80
4.2.2. Xử lý bài viết và kiểm duyệt nội dung.....	80
4.2.3. Xử lý hỏi đáp với chatbot AI.....	82
4.2.4. Xử lý nộp bài và chấm bài tự động.....	83
4.2.5. Xử lý báo cáo khiếu nại	85
4.2.6. Hệ thống cá nhân hóa người dùng	86
Chương 5. TỔNG KẾT	90
5.1. Kết quả đạt được	90
5.1.1. Kết quả về chức năng diễn đàn và quản lý người dùng.....	90
5.1.2. Kết quả về kiểm duyệt nội dung tự động bằng AI	91

5.1.3. Kết quả về chatbot AI hỗ trợ người dùng.....	91
5.1.4. Kết quả về hệ thống chấm bài tự động.....	92
5.1.5. Kết quả về cá nhân hóa người dùng.....	93
5.1.6. Kết quả trò chuyện về thời gian thực	93
5.1.7. Kết quả về giao diện người dùng	94
5.1.8. Đánh giá kiểm thử theo tiêu chí chất lượng	94
5.2. Ưu điểm của hệ thống	98
5.3. Hạn chế.....	99
5.4. Hướng phát triển	100
5.5. Nguồn kỹ thuật và khảo sát.....	101
TÀI LIỆU THAM KHẢO.....	102

DANH MỤC CÁC HÌNH ẢNH

Hình 2.1. Mô hình Client - Server	7
Hình 2.2. Kiến trúc nhiều thành phần trong ứng dụng web.....	8
Hình 2.3. RESTful API	9
Hình 2.4. Logo Svelte	10
Hình 2.5. Logo Node.js và ExpressJS.....	11
Hình 2.6. Logo Python.....	12
Hình 2.7. Logo Prisma	13
Hình 2.8. Logo PostgreSQL.....	14
Hình 2.9. Logo Docker.....	15
Hình 2.10. Logo Socket.io	16
Hình 2.11. Logo BullMQ	17
Hình 2.12. Logo Redis	18
Hình 2.13. Logo k6	19
Hình 3.1. Sơ đồ use case tổng quát	25
Hình 3.2. Sơ đồ use case quản lý người dùng.....	25
Hình 3.3. Sơ đồ use case quản lý bài viết và bài viết đã lưu.....	26
Hình 3.4. Sơ đồ use case quản lý danh sách người theo dõi và đang theo dõi	26
Hình 3.5. Sơ đồ use case quản lý thử thách lập trình.....	26
Hình 3.6. Sơ đồ use case quản lý chuyên mục và chủ đề	27
Hình 3.7. Sơ đồ use case quản lý báo cáo và khiếu nại	27
Hình 3.8. Sơ đồ kiến trúc tổng thể	38
Hình 3.9. Sơ đồ cơ sở dữ liệu nhóm người dùng và cá nhân hóa	44

Hình 3.10. Sơ đồ cơ sở dữ liệu nhóm bảng diễn đàn và tương tác nội dung.....	45
Hình 3.11. Sơ đồ cơ sở dữ liệu nhóm bảng trò chuyện thời gian thực	45
Hình 3.12. Sơ đồ cơ sở dữ liệu nhóm bảng thử thách lập trình	46
Hình 3.13. Sơ đồ cơ sở dữ liệu nhóm bảng AI và gợi ý học tập	47
Hình 3.14. Sơ đồ cơ sở dữ liệu nhóm bảng báo cáo và kiểm duyệt.....	47
Hình 4.1. Màn hình đăng ký	65
Hình 4.2. Màn hình đăng nhập.....	66
Hình 4.3. Màn hình trang hồ sơ cá nhân	67
Hình 4.4. Màn hình trang thảo luận	68
Hình 4.5. Màn hình chi tiết bài đăng.....	69
Hình 4.6. Màn hình trang thử thách lập trình.....	70
Hình 4.7. Màn hình chi tiết thử thách lập trình.....	71
Hình 4.8. Màn hình trang nhấn tin	72
Hình 4.9. Màn hình hỏi đáp với chatbot	73
Hình 4.10. Màn hình trang học tập	74
Hình 4.11. Màn hình trang quản lý người dùng.....	75
Hình 4.12. Màn hình trang quản lý chuyên mục và chủ đề	76
Hình 4.13. Màn hình trang quản lý thử thách lập trình.....	76
Hình 4.14. Màn hình thêm thử thách lập trình.....	77
Hình 4.15. Màn hình quản lý báo cáo khiếu nại	78
Hình 4.16. Màn hình xử lý báo cáo khiếu nại.....	79
Hình 5.1. Kết quả kiểm thử tải 50 người dùng đồng thời bằng k6	95
Hình 5.2. Kết quả truy vấn thời gian chấm bài trong cơ sở dữ liệu.....	96

Hình 5.3. Kết quả kiểm thử kiểm duyệt nội dung bằng AI97

DANH MỤC CÁC BẢNG

Bảng 3.1. Bảng so sánh các diễn đàn có trên thị trường.....	21
Bảng 3.2. Bảng đặc tả use case thêm bài viết	28
Bảng 3.3. Bảng đặc tả use case tham gia thử thách lập trình.....	29
Bảng 3.4. Bảng đặc tả use case thêm thử thách lập trình.....	31
Bảng 3.5. Bảng đặc tả use case hỏi đáp với AI	33
Bảng 3.6. Bảng đặc tả use case xử lý báo cáo khiếu nại.....	34
Bảng 3.7. Bảng đặc tả use case tham gia cuộc trò chuyện.....	36
Bảng 3.8. Bảng các thành phần chính của hệ thống	39
Bảng 3.9. Bảng mô tả các lớp xử lý ở Backend.....	40
Bảng 3.10. Bảng User	48
Bảng 3.11. Bảng Profile	50
Bảng 3.12. Bảng Category	51
Bảng 3.13. Bảng Topic	52
Bảng 3.14. Bảng Post	53
Bảng 3.15. Bảng Comment	54
Bảng 3.16. Bảng Reaction.....	56
Bảng 3.17. Bảng Conversation	57
Bảng 3.18. Bảng Message.....	58
Bảng 3.19. Bảng Challenge.....	59
Bảng 3.20. Bảng Language	60
Bảng 3.21. Bảng Testcase	61
Bảng 3.22. Bảng Submission	62

Bảng 3.23. Bảng Report.....	64
Bảng 4.1. Nhóm API chức năng xử lý đăng nhập và xác thực người dùng.....	80
Bảng 4.2. Nhóm API backend chức năng xử lý bài viết và kiểm duyệt nội dung	81
Bảng 4.3. Nhóm API kiểm duyệt của AI Server	81
Bảng 4.4. Nhóm API backend chức năng hỏi đáp với chatbot AI	82
Bảng 4.5. Nhóm API chức năng hỏi đáp với chatbot AI của AI Server	82
Bảng 4.6. Nhóm API backend chức năng xử lý nộp bài và chấm bài tự động	83
Bảng 4.7. Nhóm API phân tích lỗi bài nộp của AI Server	84
Bảng 4.8. Quy tắc cập nhật năng lực topic sau khi chấm bài	84
Bảng 4.9. Giá trị khởi tạo năng lực topic	85
Bảng 4.10. Nhóm API backend xử lý báo cáo khiếu nại	85
Bảng 4.11. Nhóm API chức năng gợi ý, cá nhân hóa người dùng.....	86
Bảng 4.12. Trọng số xây dựng hồ sơ chủ đề người dùng	87
Bảng 4.13. Công thức tính điểm gợi ý bài viết	88
Bảng 4.14. Công thức tính điểm gợi ý người dùng.....	89
Bảng 5.1. Kết quả kiểm thử tải hệ thống bằng k6.....	95
Bảng 5.2. Kết quả đo thời gian chấm bài tự động.....	96
Bảng 5.3. Kết quả kiểm thử kiểm duyệt bằng AI.....	97

TÓM TẮT ĐỒ ÁN

Đồ án “Phát triển diễn đàn hỗ trợ lập trình tích hợp hệ thống chấm bài tự động và chatbot hỗ trợ” được thực hiện với mục tiêu xây dựng một nền tảng trực tuyến hỗ trợ người học lập trình và lập trình viên trong quá trình trao đổi kiến thức, luyện tập kỹ năng lập trình và nhận hỗ trợ khi gặp vấn đề kỹ thuật.

Hệ thống không chỉ đóng vai trò là một diễn đàn trao đổi kiến thức lập trình, mà còn tích hợp các chức năng mở rộng nhằm nâng cao trải nghiệm học tập và thực hành. Người dùng có thể đăng ký tài khoản, đăng nhập, quản lý thông tin cá nhân, tạo bài viết, bình luận, trả lời thảo luận và tìm kiếm nội dung theo chủ đề hoặc từ khóa. Bên cạnh đó, hệ thống còn hỗ trợ tính năng nhắn tin trực tuyến giữa các người dùng, giúp việc trao đổi kiến thức trở nên thuận tiện và nhanh chóng hơn.

Một điểm nổi bật của hệ thống là việc tích hợp chatbot hỗ trợ lập trình, cho phép người dùng đặt câu hỏi và nhận phản hồi liên quan đến các vấn đề trong quá trình học tập, thực hành hoặc phát triển phần mềm. Ngoài ra, đồ án còn phát triển hệ thống chấm bài tự động thông qua các thử thách lập trình, cho phép người dùng nộp lời giải, hệ thống xử lý và đánh giá kết quả dựa trên các bộ kiểm thử được xây dựng sẵn. Tính năng bảng xếp hạng cũng được tích hợp nhằm tạo môi trường cạnh tranh lành mạnh, khuyến khích người dùng rèn luyện tư duy giải thuật và kỹ năng lập trình.

Về mặt tổng thể, hệ thống được xây dựng theo mô hình Client - Server, gồm các thành phần chính như Frontend, Backend, Judge Worker và AI Server. Frontend được phát triển bằng Svelte, đảm nhiệm việc xây dựng giao diện, xử lý tương tác và hiển thị dữ liệu cho người dùng. Backend được xây dựng bằng NodeJS, chịu trách nhiệm xử lý nghiệp vụ, xác thực, quản lý dữ liệu và cung cấp API cho Frontend. Bên cạnh đó, Judge Worker được sử dụng để xử lý việc chấm bài tự động, còn AI Server hỗ trợ các chức năng liên quan đến chatbot trong hệ thống.

Thông qua đồ án này, nhóm hướng đến việc xây dựng một sản phẩm có tính ứng dụng thực tế, kết hợp giữa diễn đàn học tập, môi trường luyện tập lập trình và công cụ hỗ

trợ thông minh. Hệ thống giúp người dùng vừa có thể trao đổi kiến thức, vừa có thể luyện tập thông qua các bài toán lập trình, đồng thời nhận được sự hỗ trợ nhanh chóng từ chatbot trong quá trình học tập và phát triển kỹ năng.

Trong nội dung báo cáo, nhóm em xin trình bày:

- **Chương 1. Giới thiệu đề tài:** Chương này trình bày tổng quan về đề tài, bao gồm bối cảnh hình thành, lý do chọn đề tài, mục tiêu xây dựng hệ thống, phạm vi thực hiện, đối tượng sử dụng, phương pháp thực hiện và kết quả mong đợi.
- **Chương 2. Cơ sở lý thuyết và công nghệ sử dụng:** Chương này trình bày các kiến thức nền tảng và công nghệ được sử dụng trong quá trình phát triển hệ thống.
- **Chương 3. Phân tích thiết kế hệ thống:** Chương này tập trung phân tích yêu cầu và thiết kế tổng thể hệ thống, bao gồm định tác nhân sử dụng hệ thống, yêu cầu chức năng, yêu cầu phi chức năng, sơ đồ use case, đặc tả use case, kiến trúc hệ thống, thiết kế cơ sở dữ liệu và luồng hệ thống.
- **Chương 4. Xây dựng và triển khai ứng dụng:** Chương này trình bày về quá trình thiết kế và xây dựng ứng dụng, mô tả các chi tiết màn hình giao diện và chức năng tương ứng, đồng thời trình bày cách Backend xử lý nghiệp vụ bên trong.
- **Chương 5. Tổng kết và hướng phát triển:** Chương này tổng kết những kết quả đạt được sau quá trình thực hiện đề án, những điểm mạnh của hệ thống, các hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

Chương 1. GIỚI THIỆU ĐỀ TÀI

Chương 1 trình bày tổng quan về đề tài, làm rõ lý do chọn đề tài, mục tiêu, phạm vi thực hiện, đối tượng sử dụng, phương pháp thực hiện và kết quả mong đợi. Qua đó, chương giúp xác định định hướng phát triển của hệ thống và tạo cơ sở cho việc phân tích, thiết kế và triển khai ở các chương tiếp theo.

1.1. Tên đề tài

Phát triển diễn đàn hỗ trợ lập trình tích hợp hệ thống chấm bài tự động và Chatbot hỗ trợ.

1.2. Lý do chọn đề tài

Trong thời đại công nghệ hiện nay, nhu cầu chia sẻ thông tin, kiến thức giữa mọi người ngày càng cao. Đặc biệt trong lĩnh vực Công nghệ Thông tin, việc trao đổi, thảo luận và học hỏi các kiến thức lập trình luôn là mối quan tâm của nhiều học sinh, sinh viên và các lập trình viên [1]. Vì vậy, nhu cầu về một diễn đàn chia sẻ về những kỹ năng lập trình hay thảo luận về những vấn đề gặp phải khi lập trình là một nhu cầu thiết yếu.

Và trong đồ án 1, nhóm đã xây dựng được một website diễn đàn hỗ trợ lập trình, cho phép người dùng đăng bài, trao đổi kiến thức và bước đầu tích hợp chatbot AI hỗ trợ trả lời câu hỏi. Bên cạnh đó, hệ thống cũng có chức năng tổ chức các thử thách lập trình cơ bản dựa trên cơ chế chấm bài bằng test case.

Tuy nhiên, hệ thống thi lập trình hiện tại mới chỉ dừng ở việc chấm các bài toán thuật toán truyền thống, chưa đáp ứng được nhu cầu đánh giá đa dạng kỹ năng lập trình trong thực tế như truy vấn cơ sở dữ liệu SQL, xử lý logic backend với Node.js hay kiểm thử API [2][3]. Đồng thời, chatbot AI còn hạn chế về khả năng phân tích sâu, cá nhân hóa người dùng và chưa hỗ trợ kiểm duyệt nội dung một cách tự động.

Xuất phát từ những hạn chế trên, nhóm quyết định phát triển đồ án 2 với định hướng nâng cấp toàn diện hệ thống, tập trung vào việc xây dựng hệ thống thi lập trình đa kỹ

năng, hỗ trợ chấm nhiều loại bài khác nhau (thuật toán, SQL, backend), đồng thời cải tiến chatbot AI và kiểm duyệt nội dung, nhằm nâng cao tính thực tiễn và giá trị ứng dụng của sản phẩm.

1.3. Mục tiêu của đề tài

Mục tiêu của Đồ án 2 là hoàn thiện và phát triển hệ thống diễn đàn hỗ trợ lập trình đã xây dựng ở giai đoạn trước, theo hướng nâng cao hiệu quả hỗ trợ người dùng và khả năng ứng dụng trong thực tế.

Cụ thể:

- Hoàn thiện mô hình hoạt động của diễn đàn hỗ trợ lập trình, đảm bảo hệ thống vận hành ổn định và phù hợp với nhu cầu học tập, trao đổi kiến thức.
- Nâng cao vai trò hỗ trợ của chatbot AI trong việc cung cấp thông tin, định hướng và hỗ trợ người dùng khi gặp vấn đề trong quá trình học tập.
- Mở rộng các hoạt động trên diễn đàn nhằm hỗ trợ người dùng rèn luyện và đánh giá kỹ năng lập trình ở mức độ phù hợp với môi trường học tập, tập trung vào Judge System đa kỹ năng.
- Cải thiện trải nghiệm người dùng và khả năng quản lý hệ thống, làm nền tảng cho các hướng phát triển tiếp theo.

1.4. Phạm vi của đề tài

Nền tảng hỗ trợ: Website dựa trên web (desktop và mobile responsive).

Các chức năng chính:

- Kiểm duyệt nội dung tự động bằng AI (lọc bài viết, bình luận vi phạm như spam, nội dung độc hại).
- Cá nhân hóa người dùng ở mức cơ bản.
- Hệ thống chấm thi lập trình (Judge System) hỗ trợ đánh giá bài toán thuật toán truyền thống, truy vấn SQL, và xử lý logic backend Node.js, với cơ chế

sandbox sử dụng Docker để cô lập môi trường chạy code, giảm rủi ro thực thi mã từ xa (Remote Code Execution - RCE) và tấn công chèn mã SQL (SQL Injection) gây ảnh hưởng hệ thống chính [4][5]. Đối với chấm SQL: So sánh kết quả trả về và rollback dữ liệu sau mỗi lần chấm.

- Cải tiến chatbot AI để hỗ trợ hỏi đáp, phân tích lỗi.
- Tái cấu trúc lại hoàn toàn Frontend.
- Các tính năng người dùng: Nộp bài tự động, xem kết quả chấm chi tiết, theo dõi bảng xếp hạng và tiến độ cá nhân, giữ trò chuyện realtime đã thực hiện ở đồ án trước.

1.5. Đối tượng sử dụng

Người dùng: Bao gồm học sinh, sinh viên, lập trình viên và những cá nhân quan tâm đến việc học tập, trao đổi kiến thức lập trình. Người dùng sẽ trực tiếp tham gia đăng bài, trả lời câu hỏi, tham gia thử thách lập trình, sử dụng chatbot AI để nhận hỗ trợ và cập nhật thông tin theo sở thích cá nhân.

Quản trị viên: Bao gồm những cá nhân hoặc nhóm được phân quyền quản lý toàn bộ hoạt động trên website. Quản trị viên chịu trách nhiệm kiểm duyệt nội dung, quản lý người dùng, giám sát chatbot AI, tổ chức các thử thách lập trình và phân tích dữ liệu hoạt động của cộng đồng nhằm đảm bảo diễn đàn hoạt động hiệu quả, an toàn và thân thiện.

1.6. Phương pháp thực hiện

Để thực hiện đề tài, nhóm tiến hành theo các bước chính gồm: tìm hiểu công nghệ, khảo sát và phân tích yêu cầu, thiết kế hệ thống, cài đặt, kiểm thử và hoàn thiện sản phẩm.

Trước tiên, nhóm tìm hiểu các công nghệ phục vụ cho quá trình phát triển hệ thống như SvelteKit, ExpressJS, Python, PostgreSQL, cùng các công cụ hỗ trợ như Figma,

Postman, Docker và GitHub để thiết kế giao diện, kiểm thử API, đóng gói ứng dụng và quản lý mã nguồn.

Tiếp theo, nhóm thu thập yêu cầu thông qua khảo sát người dùng và tham khảo các nền tảng tương tự. Từ đó, nhóm phân tích, phân loại yêu cầu và mô hình hóa các luồng xử lý chính của hệ thống.

Sau đó, nhóm tiến hành thiết kế kiến trúc hệ thống, thiết kế cơ sở dữ liệu và giao diện người dùng. Đối với hệ thống chấm bài tự động, nhóm chú trọng thiết kế cơ chế thực thi an toàn nhằm hạn chế rủi ro khi xử lý mã nguồn do người dùng nộp.

Cuối cùng, nhóm cài đặt các chức năng, kiểm thử hệ thống, hoàn thiện sản phẩm và tổng kết kết quả đạt được, đồng thời đề xuất các hướng phát triển trong tương lai.

1.7. Kết quả mong đợi

Về chức năng, hệ thống website có thể hoạt động ổn định và đáp ứng các chức năng chính của một diễn đàn hỗ trợ lập trình, bao gồm quản lý bài viết, bình luận, tìm kiếm nội dung, kiểm duyệt nội dung tự động, cá nhân hóa cơ bản, chatbot hỗ trợ người dùng và hệ thống chấm bài tự động. Hệ thống chấm bài cho phép người dùng nộp lời giải cho nhiều dạng bài khác nhau như thuật toán, SQL và backend Node.js, đồng thời trả về kết quả chấm chi tiết. Ngoài ra, hệ thống còn hỗ trợ bảng xếp hạng, theo dõi tiến độ cá nhân, trò chuyện thời gian thực và giao diện thân thiện, có khả năng hiển thị tốt trên nhiều thiết bị.

Về mặt tích hợp, hệ thống có thể kết nối và sử dụng các công cụ AI để hỗ trợ kiểm duyệt nội dung và chatbot, giúp nâng cao khả năng hỗ trợ người dùng trong quá trình học tập, trao đổi và thực hành lập trình.

Về chất lượng, hệ thống hướng đến mã nguồn rõ ràng, dễ bảo trì, có cơ chế xác thực người dùng và chú trọng đến an toàn khi thực thi mã nguồn trong hệ thống chấm bài. Website cần mang lại trải nghiệm sử dụng mượt mà, dễ thao tác và có giá trị thực tiễn đối với cộng đồng người học lập trình, đáp ứng được một số tiêu chí đánh giá cơ bản như truy cập đồng thời, tính sẵn sàng và hiệu năng của hệ thống.

Chương 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

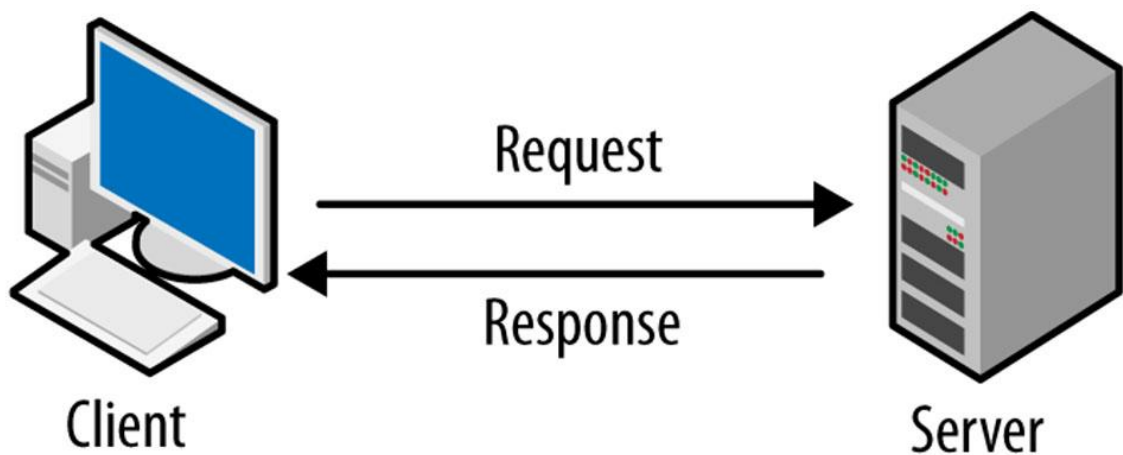
Chương 2 trình bày các kiến thức nền tảng và công nghệ được sử dụng trong quá trình phát triển hệ thống, bao gồm cơ sở lý thuyết về các mô hình Client - Server, kiến trúc nhiều thành phần trong ứng dụng web, RESTful API, sandbox và các công nghệ, ngôn ngữ lập trình, framework, cơ sở dữ liệu để hỗ trợ phát triển hệ thống.

2.1. Cơ sở lý thuyết

2.1.1. Mô hình Client - Server

Mô hình Client - Server là mô hình kiến trúc phổ biến trong phát triển ứng dụng web. Trong mô hình này, Client là phía người dùng, có nhiệm vụ hiển thị giao diện và gửi yêu cầu đến Server. Server là phía xử lý yêu cầu, thực hiện nghiệp vụ, truy xuất dữ liệu và trả kết quả về cho Client [6].

Việc áp dụng mô hình Client - Server giúp hệ thống phân chia rõ ràng giữa giao diện người dùng và phần xử lý nghiệp vụ. Nhờ đó, hệ thống dễ phát triển, dễ bảo trì và thuận tiện khi mở rộng thêm các thành phần hoặc chức năng mới.

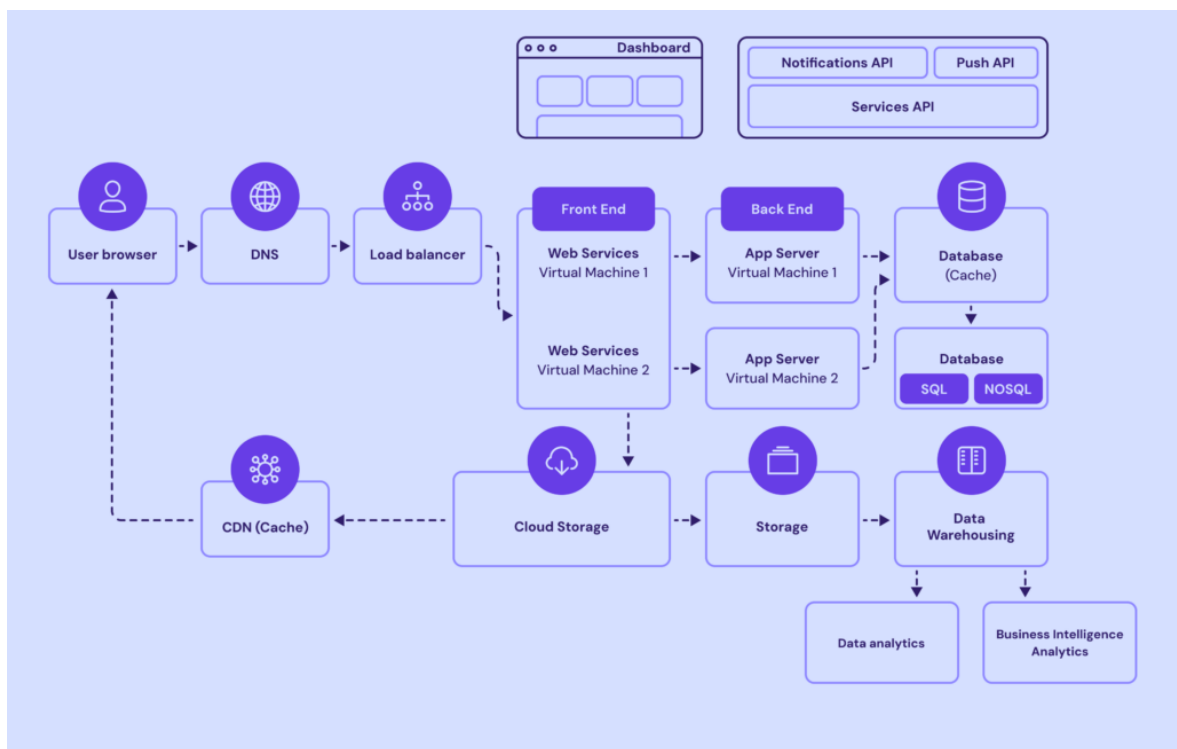


Hình 2.1. Mô hình Client - Server

2.1.2. Kiến trúc nhiều thành phần trong ứng dụng web

Đối với các hệ thống web có nhiều chức năng phức tạp, kiến trúc Client - Server có thể được mở rộng thành kiến trúc nhiều thành phần. Thay vì để một server xử lý toàn bộ nghiệp vụ, hệ thống có thể được chia thành nhiều thành phần riêng như server chính, dịch vụ xử lý AI, worker xử lý tác vụ nền, cơ sở dữ liệu và các dịch vụ hỗ trợ khác.

Cách tổ chức này giúp mỗi thành phần tập trung vào một nhóm nhiệm vụ cụ thể. Server chính có thể đảm nhiệm việc xử lý nghiệp vụ và điều phối yêu cầu, trong khi các thành phần phụ trách xử lý những tác vụ chuyên biệt như chấm bài tự động, xử lý AI hoặc gửi thông báo. Điều này giúp hệ thống dễ mở rộng, giảm tải cho server chính và tăng khả năng bảo trì.

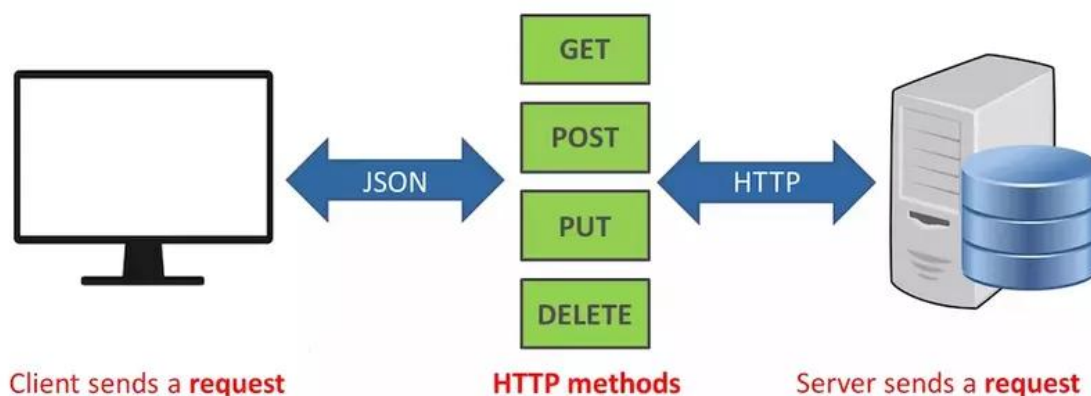


Hình 2.2. Kiến trúc nhiều thành phần trong ứng dụng web

2.1.3. RESTful API

RESTful API là phương thức giao tiếp giữa các thành phần trong hệ thống thông qua giao thức HTTP, dựa trên cách tổ chức tài nguyên và sử dụng các phương thức HTTP để biểu diễn thao tác với tài nguyên [7], [8]. API đóng vai trò là cầu nối giúp Client gửi yêu cầu đến Server và nhận dữ liệu phản hồi.

Các phương thức HTTP thường được sử dụng gồm GET để lấy dữ liệu, POST để tạo dữ liệu mới, PUT hoặc PATCH để cập nhật dữ liệu và DELETE để xóa dữ liệu [8]. RESTful API giúp tách biệt phần giao diện người dùng với phần xử lý nghiệp vụ, từ đó giúp Frontend và Backend có thể phát triển độc lập và dễ kiểm thử hơn.



Hình 2.3. RESTful API

2.1.4. Sandbox

Sandbox là môi trường thực thi cô lập, được sử dụng để chạy chương trình trong phạm vi được kiểm soát. Trong hệ thống chấm bài tự động, sandbox đặc biệt quan trọng vì mã nguồn do người dùng nộp có thể chứa lỗi, vòng lặp vô hạn hoặc các hành vi không an toàn.

Cơ chế sandbox giúp hạn chế quyền truy cập của chương trình, kiểm soát tài nguyên sử dụng, giới hạn thời gian chạy, bộ nhớ và các thao tác có thể ảnh hưởng đến hệ thống chính. Nhờ đó, hệ thống có thể giảm rủi ro bảo mật khi cần thực thi mã nguồn từ người dùng [4], [5].

2.2. Công nghệ sử dụng

2.2.1. Svelte/SvelteKit

Svelte là một framework Frontend hiện đại được sử dụng để xây dựng giao diện người dùng cho ứng dụng web. Khác với một số thư viện Frontend truyền thống, Svelte thực hiện việc biên dịch mã nguồn thành JavaScript tối ưu trong quá trình build, giúp giảm bớt khối lượng xử lý ở phía trình duyệt [9]. SvelteKit là framework được xây dựng trên nền tảng Svelte, hỗ trợ phát triển ứng dụng web hoàn chỉnh với các chức năng như routing, tổ chức trang, xử lý dữ liệu và tối ưu quá trình xây dựng giao diện.

Ưu điểm của Svelte là cú pháp đơn giản, dễ tiếp cận và dễ tổ chức mã nguồn theo từng component. Cơ chế reactive của Svelte giúp giao diện tự động cập nhật khi dữ liệu thay đổi, từ đó hỗ trợ tốt cho việc xây dựng các màn hình có tính tương tác cao. Ngoài ra, do được biên dịch trước khi chạy, ứng dụng Svelte thường có kích thước nhẹ và hiệu năng tốt.

Tuy nhiên, so với các công nghệ phổ biến hơn như React hoặc Vue, hệ sinh thái và cộng đồng của Svelte vẫn còn nhỏ hơn. Vì vậy, trong một số trường hợp, việc tìm kiếm thư viện hỗ trợ hoặc tài liệu tham khảo cho các vấn đề đặc thù có thể gặp khó khăn hơn.

Trong đề tài, Svelte/SvelteKit được sử dụng để xây dựng phần Frontend của hệ thống. Các giao diện như đăng nhập, đăng ký, trang chủ diễn đàn, danh sách bài viết, chi tiết bài viết, bình luận, chatbot, thử thách lập trình, nộp bài và xem kết quả chấm được triển khai bằng Svelte. Việc sử dụng Svelte giúp nhóm tổ chức giao diện theo component, dễ bảo trì và thuận tiện khi mở rộng chức năng.



Hình 2.4. Logo Svelte

2.2.2. Node.js và ExpressJS

Node.js là môi trường thực thi JavaScript phía máy chủ, cho phép xây dựng các ứng dụng Backend bằng JavaScript [10]. ExpressJS là framework chạy trên Node.js, hỗ trợ xây dựng API, định tuyến request và xử lý các nghiệp vụ phía server một cách linh hoạt [11], [12].

Ưu điểm nổi bật của Node.js và ExpressJS là khả năng xử lý nhiều kết nối đồng thời, tốc độ phát triển nhanh, cộng đồng lớn và hệ sinh thái thư viện phong phú thông qua npm. Ngoài ra, do Frontend của hệ thống được xây dựng bằng Svelte và Backend được xây dựng bằng Node.js/ExpressJS, cả hai phía đều sử dụng hệ sinh thái JavaScript. Điều này giúp quá trình phát triển thuận tiện hơn, giảm sự khác biệt về ngôn ngữ lập trình giữa giao diện người dùng và phía máy chủ, đồng thời giúp việc trao đổi dữ liệu giữa Frontend và Backend thông qua API trở nên nhất quán hơn.

Hạn chế của Node.js là không phù hợp để xử lý trực tiếp các tác vụ tính toán nặng trong cùng tiến trình Backend chính như xử lý video hay AI, và dễ bị sập nếu có lỗi chưa được xử lý [10].

Trong đề tài, Node.js và ExpressJS được sử dụng để xây dựng Backend chính của hệ thống. Backend chịu trách nhiệm xử lý xác thực người dùng, quản lý bài viết, bình luận, thông tin cá nhân, thử thách lập trình, bài nộp, kết quả chấm và bảng xếp hạng. Ngoài Backend chính, Node.js cũng được sử dụng để xây dựng Judge Worker nhằm thực thi quá trình chấm bài theo cơ chế tách biệt với tiến trình xử lý request chính, giúp hệ thống ổn định và dễ mở rộng hơn.



Hình 2.5. Logo Node.js và ExpressJS

2.2.3. Python

Python là ngôn ngữ lập trình bậc cao, có cú pháp đơn giản, dễ đọc và được sử dụng rộng rãi trong nhiều lĩnh vực như xử lý dữ liệu, trí tuệ nhân tạo, tự động hóa và xây dựng các dịch vụ hỗ trợ hệ thống [13].

Ưu điểm của Python là có hệ sinh thái thư viện phong phú, đặc biệt mạnh trong các lĩnh vực liên quan đến AI, xử lý dữ liệu và tự động hóa. Cú pháp rõ ràng giúp quá trình phát triển và bảo trì mã nguồn dễ dàng hơn. Python cũng phù hợp để xây dựng các service riêng phục vụ các tác vụ chuyên biệt.

Hạn chế của Python là hiệu năng có thể không cao bằng một số ngôn ngữ biên dịch trong các tác vụ yêu cầu tốc độ xử lý lớn. Ngoài ra, khi triển khai các service Python, cần quản lý môi trường chạy, thư viện phụ thuộc và phiên bản package cẩn thận để tránh lỗi khi chuyển sang môi trường khác.

Điểm nổi bật về mặt kỹ thuật của Python là khả năng tích hợp với nhiều thư viện và công cụ khác nhau. Python có thể được sử dụng để xây dựng các service độc lập, giao tiếp với Backend thông qua API hoặc các cơ chế truyền thông giữa service.

Trong đề tài, Python được sử dụng cho các thành phần xử lý chuyên biệt như AI Server. AI Server đảm nhiệm các xử lý liên quan đến chatbot, vector hóa dữ liệu, phân tích nội dung và hỗ trợ kiểm duyệt tự động.



Hình 2.6. Logo Python

2.2.4. Prisma ORM

Prisma ORM là công cụ ánh xạ đối tượng - quan hệ được sử dụng trong các ứng dụng Backend để làm việc với cơ sở dữ liệu thông qua schema và các truy vấn ở mức mã nguồn. Thay vì viết trực tiếp toàn bộ câu lệnh SQL trong từng chức năng, Prisma cho phép định nghĩa mô hình dữ liệu, quan hệ giữa các bảng và kiểu dữ liệu trong file schema, sau đó sinh Prisma Client để Backend có thể thao tác với cơ sở dữ liệu một cách rõ ràng và an toàn hơn [14].

Ưu điểm của Prisma là giúp mã nguồn thao tác dữ liệu dễ đọc, giảm lỗi khi viết truy vấn thủ công và hỗ trợ quản lý mô hình dữ liệu tập trung. Prisma cũng hỗ trợ migration, giúp quá trình thay đổi cấu trúc cơ sở dữ liệu được kiểm soát tốt hơn trong quá trình phát triển hệ thống.

Hạn chế của Prisma là lập trình viên cần hiểu rõ cách ánh xạ giữa schema và cơ sở dữ liệu thật. Với các truy vấn phức tạp hoặc yêu cầu tối ưu hiệu năng cao, đôi khi vẫn cần sử dụng truy vấn SQL trực tiếp hoặc kiểm tra kỹ câu truy vấn được sinh ra.

Trong đề tài, Prisma ORM được sử dụng ở Backend để kết nối và thao tác với PostgreSQL. Các dữ liệu như người dùng, hồ sơ cá nhân, bài viết, bình luận, thử thách lập trình, bài nộp, kết quả chấm, báo cáo và tin nhắn được định nghĩa thông qua các model trong Prisma schema. Việc sử dụng Prisma giúp nhóm tổ chức tầng truy cập dữ liệu rõ ràng hơn, thuận tiện cho việc phát triển API, bảo trì mã nguồn và mở rộng cơ sở dữ liệu.



Hình 2.7. Logo Prisma

2.2.5. PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, được sử dụng để lưu trữ và quản lý dữ liệu có cấu trúc. PostgreSQL hỗ trợ các thành phần quan trọng của cơ sở dữ liệu quan hệ như bảng, khóa chính, khóa ngoại, ràng buộc dữ liệu, transaction và truy vấn SQL [15].

Ưu điểm của PostgreSQL là độ ổn định cao, hỗ trợ tốt các ràng buộc dữ liệu và transaction, giúp đảm bảo tính toàn vẹn của dữ liệu. PostgreSQL cũng hỗ trợ truy vấn mạnh mẽ, phù hợp với các hệ thống có nhiều bảng dữ liệu liên kết với nhau.

Hạn chế của PostgreSQL là việc thiết kế cơ sở dữ liệu cần được thực hiện cẩn thận ngay từ đầu. Nếu thiết kế bảng, quan hệ hoặc chỉ mục chưa hợp lý, hệ thống có thể gặp khó khăn khi mở rộng hoặc khi truy vấn dữ liệu lớn.

Điểm nổi bật về mặt kỹ thuật của PostgreSQL là khả năng hỗ trợ transaction và đảm bảo tính nhất quán dữ liệu. Điều này đặc biệt quan trọng đối với các hệ thống có nhiều thao tác ghi dữ liệu như đăng bài, bình luận, nộp bài, lưu kết quả chấm và cập nhật bảng xếp hạng.

Trong đề tài, PostgreSQL được sử dụng để lưu trữ các dữ liệu chính của hệ thống như tài khoản người dùng, bài viết, bình luận, thử thách lập trình, test case, bài nộp, kết quả chấm, bảng xếp hạng và dữ liệu trò chuyện. Việc sử dụng PostgreSQL giúp hệ thống quản lý dữ liệu rõ ràng, đảm bảo tính nhất quán và thuận tiện cho quá trình truy vấn.



Hình 2.8. Logo PostgreSQL

2.2.6. Docker

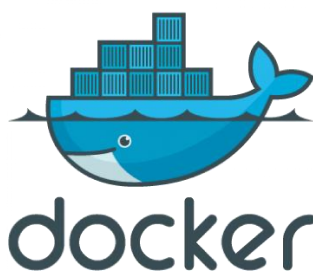
Docker là nền tảng hỗ trợ đóng gói và triển khai ứng dụng dưới dạng container. Container cho phép ứng dụng chạy trong một môi trường tách biệt, bao gồm mã nguồn, thư viện phụ thuộc và các cấu hình cần thiết [16].

Ưu điểm của Docker là giúp ứng dụng dễ triển khai trên nhiều môi trường khác nhau, giảm tình trạng lỗi do khác biệt giữa môi trường phát triển và môi trường chạy thực tế. Docker cũng giúp quản lý các thành phần trong hệ thống một cách rõ ràng hơn, đặc biệt khi hệ thống có nhiều service như Backend, AI Server, Judge Worker và cơ sở dữ liệu.

Hạn chế của Docker là cần có kiến thức về image, container, network, volume và cấu hình môi trường. Nếu cấu hình không đúng, hệ thống có thể gặp lỗi khi giao tiếp giữa các container hoặc khi lưu trữ dữ liệu. Ngoài ra, việc tạo môi trường thực thi tách biệt cũng cần được kiểm soát kỹ để tránh tiêu tốn quá nhiều tài nguyên hệ thống.

Điểm nổi bật về mặt kỹ thuật của Docker trong đề tài là khả năng tạo môi trường sandbox. Việc sử dụng Docker giúp tạo ra môi trường chạy riêng cho quá trình thực thi và kiểm tra bài nộp, từ đó tăng tính an toàn và ổn định cho hệ thống [4], [15].

Trong đề tài, Docker được sử dụng chủ yếu để hỗ trợ Judge System. Đối với Judge System, Docker hỗ trợ tạo môi trường sandbox để chạy mã nguồn của người dùng trong phạm vi tách biệt.



Hình 2.9. Logo Docker

2.2.7. Socket.IO

Socket.IO là thư viện hỗ trợ giao tiếp thời gian thực giữa Client và Server. Công nghệ này cho phép Server gửi dữ liệu đến Client ngay khi có sự kiện mới, thay vì Client phải liên tục gửi yêu cầu để kiểm tra dữ liệu [17].

Ưu điểm của Socket.IO là hỗ trợ tốt các chức năng realtime như trò chuyện trực tuyến, thông báo và cập nhật trạng thái. Socket.IO cũng cung cấp cơ chế kết nối ổn định hơn trong nhiều môi trường mạng khác nhau, giúp việc xây dựng các chức năng thời gian thực trở nên thuận tiện.

Hạn chế của Socket.IO là việc quản lý kết nối realtime phức tạp hơn so với các API thông thường. Hệ thống cần xử lý các trường hợp như người dùng kết nối, mất kết nối, gửi lại dữ liệu, quản lý phòng trò chuyện và đảm bảo dữ liệu được gửi đến đúng người nhận.

Điểm nổi bật về mặt kỹ thuật của Socket.IO là cơ chế giao tiếp hai chiều giữa Client và Server. Nhờ đó, Server có thể chủ động gửi dữ liệu đến Client khi có sự kiện xảy ra, giúp tăng tính tương tác của ứng dụng.

Trong đề tài, Socket.IO được sử dụng cho chức năng trò chuyện thời gian thực giữa người dùng. Khi một người dùng gửi tin nhắn, hệ thống có thể truyền tin nhắn đến người nhận gần như ngay lập tức. Ngoài chức năng chat, Socket.IO cũng có thể được sử dụng để thông báo khi có sự kiện mới như bình luận mới hoặc trạng thái chấm bài được cập nhật.



Hình 2.10. Logo Socket.io

2.2.8. BullMQ

BullMQ là thư viện hàng đợi tác vụ được xây dựng trên nền tảng Redis, thường được sử dụng trong các hệ thống Node.js để xử lý các công việc bất đồng bộ [18]. Thay vì để Backend xử lý trực tiếp những tác vụ tốn thời gian, hệ thống có thể đưa tác vụ vào hàng đợi và để Worker xử lý riêng.

Ưu điểm của BullMQ là giúp quản lý job theo trạng thái như chờ xử lý, đang xử lý, hoàn thành hoặc thất bại. BullMQ cũng hỗ trợ retry khi job lỗi, giúp hạn chế tình trạng mất tác vụ trong quá trình xử lý. Tuy nhiên, BullMQ phụ thuộc vào Redis, nên nếu Redis gặp lỗi hoặc cấu hình không đúng thì quá trình xử lý hàng đợi có thể bị ảnh hưởng.

Hạn chế của BullMQ là hệ thống cần phụ thuộc vào Redis để lưu trữ và quản lý hàng đợi. Nếu Redis gặp sự cố hoặc cấu hình không phù hợp, quá trình xử lý job có thể bị ảnh hưởng.

Điểm nổi bật về mặt kỹ thuật của BullMQ là khả năng hỗ trợ giao tiếp bất đồng bộ giữa Backend và Judge Worker thông qua cơ chế hàng đợi. Backend không cần trực tiếp chờ quá trình chấm bài hoàn tất trong cùng một request, mà chỉ cần tạo job chấm bài và lưu trạng thái bài nộp.

Trong đề tài, BullMQ được sử dụng để giao tiếp bất đồng bộ giữa Backend và Judge Worker. Khi người dùng nộp bài, Backend lưu thông tin bài nộp và đưa yêu cầu chấm vào hàng đợi. Judge Worker lấy job từ hàng đợi để chấm bài, sau đó gửi kết quả về. Cách làm này giúp hệ thống xử lý bài nộp ổn định hơn, tránh bỏ sót hoặc mất bài khi có nhiều người dùng nộp cùng lúc.



Hình 2.11. Logo BullMQ

2.2.9. Redis

Redis là hệ quản trị dữ liệu dạng key-value có tốc độ truy xuất cao, thường được sử dụng để lưu trữ dữ liệu tạm thời, cache và hỗ trợ các hệ thống hàng đợi. Redis phù hợp với các loại dữ liệu cần xử lý nhanh, có thời gian tồn tại ngắn hoặc không cần lưu lâu dài trong cơ sở dữ liệu chính [19].

Ưu điểm của Redis là tốc độ xử lý cao, cấu trúc dữ liệu linh hoạt và hỗ trợ cơ chế đặt thời gian hết hạn cho dữ liệu. Cơ chế này phù hợp với các chức năng như lưu mã OTP trong một khoảng thời gian ngắn, lưu trạng thái hoạt động của người dùng, hoặc lưu các thông tin tạm thời không cần tồn tại lâu dài trong cơ sở dữ liệu chính [20]. Ngoài ra, Redis cũng được nhiều thư viện hàng đợi sử dụng làm nền tảng lưu trữ job, trong đó có BullMQ.

Tuy nhiên, Redis cũng có một số hạn chế. Vì Redis thường lưu dữ liệu trên bộ nhớ, dung lượng lưu trữ phụ thuộc vào tài nguyên RAM của máy chủ. Nếu sử dụng Redis để lưu quá nhiều dữ liệu hoặc không cấu hình thời gian hết hạn hợp lý, hệ thống có thể tiêu tốn nhiều tài nguyên.

Trong đề tài, Redis được sử dụng để lưu mã OTP trong quá trình xác thực, lưu trạng thái hoạt động của người dùng và làm nền tảng cho BullMQ trong hệ thống chấm bài. Nhờ Redis, hệ thống có thể xử lý nhanh các dữ liệu tạm thời, đồng thời hỗ trợ quá trình giao tiếp bất đồng bộ giữa Backend và Judge Worker một cách ổn định hơn. Redis không thay thế cơ sở dữ liệu chính, mà đóng vai trò hỗ trợ cho các dữ liệu ngắn hạn và các tác vụ cần tốc độ xử lý cao.



Hình 2.12. Logo Redis

2.2.10. k6

k6 là công cụ kiểm thử tải mã nguồn mở, được sử dụng để mô phỏng nhiều người dùng ảo truy cập hệ thống trong cùng một khoảng thời gian. Công cụ này cho phép xây dựng các kịch bản kiểm thử bằng JavaScript, gửi request đến các API cần đánh giá và thu thập các chỉ số như tổng số request, tỷ lệ lỗi, thời gian phản hồi trung bình, thời gian phản hồi trung vị và p95 response time [21].

Ưu điểm của k6 là cú pháp kịch bản kiểm thử tương đối dễ tiếp cận, phù hợp với các hệ thống web API và có thể chạy trực tiếp bằng dòng lệnh hoặc thông qua Docker. Nhờ đó, nhóm có thể kiểm tra khả năng chịu tải ban đầu của hệ thống mà không cần xây dựng thêm công cụ kiểm thử riêng.

Hạn chế của k6 là kết quả kiểm thử phụ thuộc nhiều vào kịch bản được thiết kế, môi trường chạy kiểm thử và tài nguyên máy chủ tại thời điểm thực hiện. Do đó, kết quả kiểm thử tải trong phạm vi đề tài chỉ mang ý nghĩa đánh giá bước đầu, chưa thể đại diện hoàn toàn cho hiệu năng của hệ thống trong môi trường triển khai thực tế với dữ liệu lớn và số lượng người dùng cao hơn.

Trong đề tài, k6 được sử dụng để kiểm thử tải hệ thống bằng cách mô phỏng 50 người dùng ảo đồng thời truy cập vào một số API thường dùng như danh sách bài viết, danh sách thử thách lập trình và dữ liệu gợi ý cá nhân hóa. Các kết quả thu được như tổng số request, tỷ lệ request lỗi, thời gian phản hồi trung bình và p95 response time được sử dụng làm cơ sở đánh giá khả năng xử lý đồng thời ban đầu của hệ thống.



Hình 2.13. Logo k6

Chương 3. PHÂN TÍCH THIẾT KẾ HỆ THỐNG

Chương 3 tập trung xác định các tác nhân sử dụng hệ thống, yêu cầu chức năng, yêu cầu phi chức năng, mô hình use case, luồng xử lý nghiệp vụ, kiến trúc tổng thể, thiết kế cơ sở dữ liệu và thiết kế các thành phần chính của hệ thống. Đây là cơ sở quan trọng cho quá trình xây dựng và triển khai ứng dụng ở chương tiếp theo.

3.1. Khảo sát và phân tích yêu cầu

Trong giai đoạn Đồ án 1, nhóm đã thực hiện khảo sát người dùng nhằm tìm hiểu nhu cầu sử dụng diễn đàn hỗ trợ lập trình, mức độ tham gia các cộng đồng hỏi đáp lập trình, các hạn chế của những nền tảng hiện có và những chức năng mà người dùng mong muốn. Nội dung khảo sát chi tiết đã được trình bày trong báo cáo Đồ án 1, bao gồm thông tin về đối tượng khảo sát, hành vi sử dụng diễn đàn và nhu cầu đối với một nền tảng hỗ trợ học lập trình.

Chi tiết kết quả khảo sát được trình bày trong báo cáo Đồ án 1: [DoAn1](#).

Trong Đồ án 2, nhóm kế thừa kết quả khảo sát từ giai đoạn trước làm cơ sở phân tích hiện trạng và xác định hướng phát triển tiếp theo cho hệ thống. Kết quả khảo sát cho thấy người dùng có nhu cầu cao đối với một nền tảng hỗ trợ trao đổi kiến thức lập trình, có khả năng tìm kiếm nội dung hiệu quả, hỗ trợ thông minh bằng chatbot AI, cá nhân hóa trải nghiệm và cải thiện chất lượng nội dung trên diễn đàn.

Bên cạnh việc kế thừa kết quả khảo sát người dùng từ Đồ án 1, trong Đồ án 2 nhóm tiếp tục xem xét và so sánh một số nền tảng tương tự đang được sử dụng phổ biến như Stack Overflow, LeetCode, HackerRank, Reddit, Discord và các công cụ hỗ trợ bằng AI. Việc so sánh này nhằm làm rõ ưu điểm, hạn chế của các nền tảng hiện có, từ đó xác định khoảng trống mà hệ thống cần tập trung phát triển.

Bảng 3.1. Bảng so sánh các diễn đàn có trên thị trường

Tính năng	Stack Overflow 	Leetcode 	Reddit 	WindFlow 
Đăng bài, hỏi đáp lập trình	X	X	X	X
Tổ chức nội dung theo chủ đề	X	X	X	X
Chăm bài thuật toán/DSA		X		X
Chăm bài SQL/Database		X		X
Chăm bài backend/API				X
Cuộc thi lập trình		X		
Chatbot AI hỗ trợ hỏi đáp				X
AI phân tích lỗi bài nộp				X
Thực quan hóa thuật toán				X
Nhắn tin cá nhân, nhóm	X			X
Gọi video thời gian thực				X
Luyện phỏng vấn kỹ thuật		X		

Dựa trên hệ thống đã xây dựng ở Đề án 1, nhóm nhận thấy sản phẩm đã đáp ứng được các chức năng nền tảng như đăng bài, bình luận, trao đổi kiến thức, chatbot hỗ trợ cơ bản, thử thách lập trình và trò chuyện thời gian thực. Tuy nhiên, hệ thống vẫn còn

một số hạn chế cần tiếp tục phát triển, đặc biệt là khả năng chấm bài còn giới hạn ở dạng bài thuật toán cơ bản, chatbot chưa hỗ trợ phân tích sâu theo ngữ cảnh, chức năng kiểm duyệt nội dung chưa được tự động hóa rõ ràng và trải nghiệm người dùng vẫn cần được cải thiện.

Vì vậy, trong Đề án 2, nhóm tập trung phát triển hệ thống theo hướng mở rộng và nâng cấp các chức năng đã có. Trọng tâm của giai đoạn này là xây dựng Judge System đa kỹ năng hỗ trợ các dạng bài thuật toán, SQL và backend Node.js; cải tiến chatbot AI; bổ sung kiểm duyệt nội dung tự động; cá nhân hóa cơ bản; đồng thời tái cấu trúc giao diện và nâng cao tính an toàn khi thực thi mã nguồn của người dùng.

3.2. Xác định yêu cầu của hệ thống

Các yêu cầu được chia thành hai nhóm chính gồm yêu cầu chức năng và yêu cầu phi chức năng. Yêu cầu chức năng mô tả những chức năng mà hệ thống cần cung cấp cho người dùng và quản trị viên. Yêu cầu phi chức năng mô tả các tiêu chí chất lượng của hệ thống như hiệu năng, bảo mật, tính an toàn, khả năng mở rộng và trải nghiệm người dùng.

3.2.1. Yêu cầu chức năng

Hệ thống cần hỗ trợ nhóm chức năng quản lý tài khoản người dùng, bao gồm đăng ký, đăng nhập, đăng xuất, cập nhật thông tin cá nhân và xem hồ sơ người dùng. Các chức năng này giúp người dùng có thể sử dụng hệ thống với tài khoản riêng, đồng thời tạo cơ sở để lưu trữ lịch sử hoạt động, bài viết, bài nộp và tiến độ cá nhân.

Đối với chức năng diễn đàn, hệ thống cần cho phép người dùng tạo bài viết, chỉnh sửa bài viết, xem danh sách bài viết, xem chi tiết bài viết, bình luận, trả lời bình luận và tìm kiếm nội dung theo từ khóa hoặc chủ đề. Đây là nhóm chức năng nền tảng nhằm hỗ trợ người dùng trao đổi kiến thức, đặt câu hỏi và thảo luận các vấn đề liên quan đến lập trình.

Hệ thống cần có chức năng kiểm duyệt nội dung tự động nhằm hỗ trợ phát hiện các bài viết hoặc bình luận không phù hợp. Chức năng này giúp giảm bớt khối lượng kiểm duyệt thủ công và góp phần duy trì môi trường trao đổi an toàn, lành mạnh.

Đối với chức năng chatbot hỗ trợ, hệ thống cần cho phép người dùng gửi câu hỏi hoặc nội dung cần hỗ trợ, sau đó nhận phản hồi từ chatbot. Chatbot được định hướng hỗ trợ người dùng trong quá trình học lập trình, giải thích lỗi, gợi ý hướng xử lý và cung cấp thông tin tham khảo liên quan đến các vấn đề kỹ thuật.

Đối với hệ thống chấm bài tự động, hệ thống cần cho phép người dùng xem danh sách thử thách lập trình, xem chi tiết đề bài, nộp lời giải, theo dõi trạng thái bài nộp và xem kết quả chấm chi tiết. Hệ thống chấm bài cần hỗ trợ nhiều dạng bài khác nhau, bao gồm bài thuật toán, bài SQL và bài backend Node.js. Đây là chức năng trọng tâm của Đề án 2 nhằm giúp người dùng rèn luyện nhiều kỹ năng lập trình thực tế hơn.

Hệ thống cần hỗ trợ bảng xếp hạng và tiến độ cá nhân. Người dùng có thể theo dõi kết quả luyện tập, số bài đã hoàn thành, điểm số hoặc thứ hạng của mình trong hệ thống. Chức năng này giúp tăng động lực học tập và tạo môi trường cạnh tranh tích cực giữa người dùng.

Bên cạnh đó, hệ thống tiếp tục duy trì chức năng trò chuyện thời gian thực đã được xây dựng ở giai đoạn trước. Người dùng có thể gửi và nhận tin nhắn trực tuyến, giúp việc trao đổi kiến thức và hỗ trợ lẫn nhau diễn ra nhanh chóng hơn.

Đối với quản trị viên, hệ thống cần cung cấp các chức năng quản lý người dùng, quản lý bài viết, quản lý bình luận, quản lý thử thách lập trình, quản lý test case, theo dõi bài nộp, xem kết quả chấm và giám sát các nội dung được hệ thống kiểm duyệt. Các chức năng này giúp quản trị viên kiểm soát hoạt động của hệ thống và đảm bảo hệ thống vận hành ổn định.

3.2.2. Yêu cầu phi chức năng

Về hiệu năng, hệ thống cần đảm bảo phản hồi trong thời gian hợp lý đối với các chức năng thường xuyên được sử dụng như đăng nhập, xem bài viết, bình luận, tìm kiếm

nội dung và xem kết quả chấm. Đối với quá trình chấm bài tự động, hệ thống cần xử lý bài nộp theo cơ chế phù hợp để tránh làm ảnh hưởng đến các chức năng chính của Backend.

Về bảo mật, hệ thống cần có cơ chế xác thực và phân quyền người dùng. Người dùng chỉ được thực hiện các thao tác phù hợp với quyền hạn của mình. Các chức năng quản trị cần được giới hạn cho tài khoản có quyền quản trị. Ngoài ra, dữ liệu người dùng và các thông tin quan trọng cần được bảo vệ trong quá trình lưu trữ và xử lý.

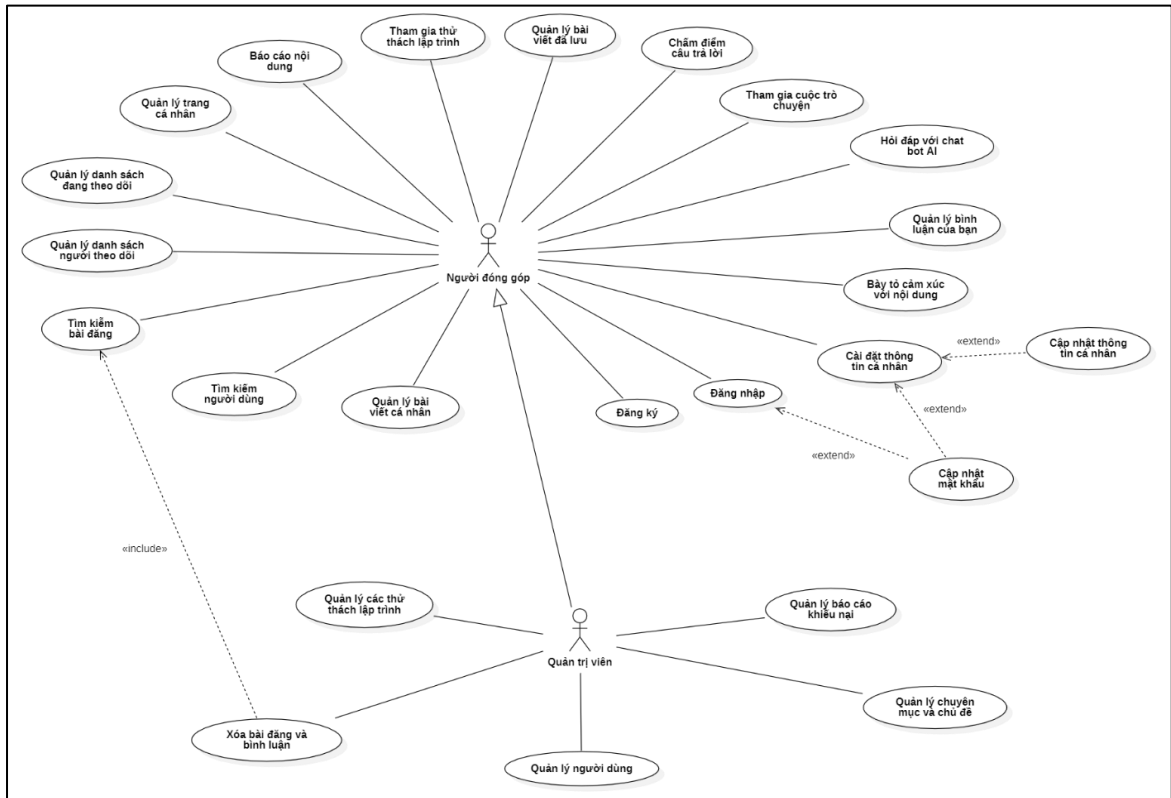
Về an toàn khi chấm bài, hệ thống cần hạn chế rủi ro khi thực thi mã nguồn do người dùng nộp. Các bài nộp cần được xử lý trong môi trường được kiểm soát, tránh ảnh hưởng trực tiếp đến Backend chính và cơ sở dữ liệu thật của hệ thống. Đối với bài SQL, hệ thống cần có cơ chế bảo vệ dữ liệu kiểm thử và khôi phục dữ liệu sau quá trình chấm nếu cần thiết.

Về khả năng mở rộng, hệ thống cần được thiết kế theo hướng có thể bổ sung thêm chức năng trong tương lai. Các thành phần như AI Server và Judge Worker nên được tách biệt tương đối với Backend chính để thuận tiện cho việc nâng cấp chatbot, mở rộng dạng bài chấm mới hoặc tăng khả năng xử lý khi số lượng người dùng tăng lên.

Về tính dễ sử dụng, giao diện hệ thống cần rõ ràng, dễ thao tác và phù hợp với nhóm người dùng chính là học sinh, sinh viên và người học lập trình. Các chức năng như đăng bài, đặt câu hỏi, nộp bài, xem kết quả chấm và sử dụng chatbot cần được thiết kế dễ tiếp cận, tránh gây khó khăn cho người dùng mới.

3.3. Sơ đồ use case

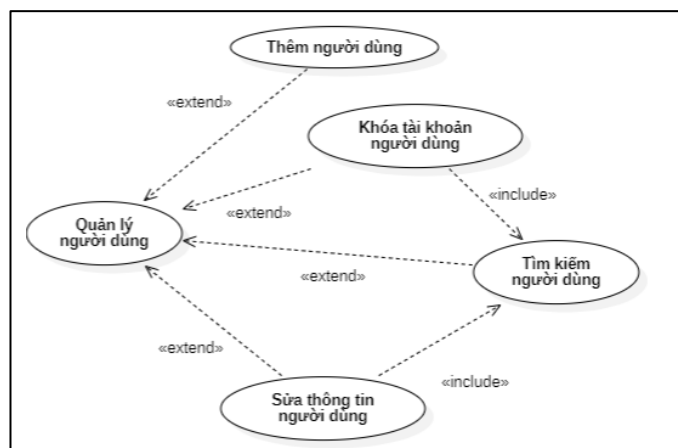
Sơ đồ Use Case được sử dụng để mô tả tổng quát các chức năng của hệ thống và sự tương tác giữa các tác nhân với hệ thống. Dựa trên các yêu cầu đã xác định, hệ thống gồm hai nhóm tác nhân chính là người dùng và quản trị viên. Sơ đồ Use Case tổng quát của hệ thống được trình bày như sau:



Hình 3.1. Sơ đồ use case tổng quát

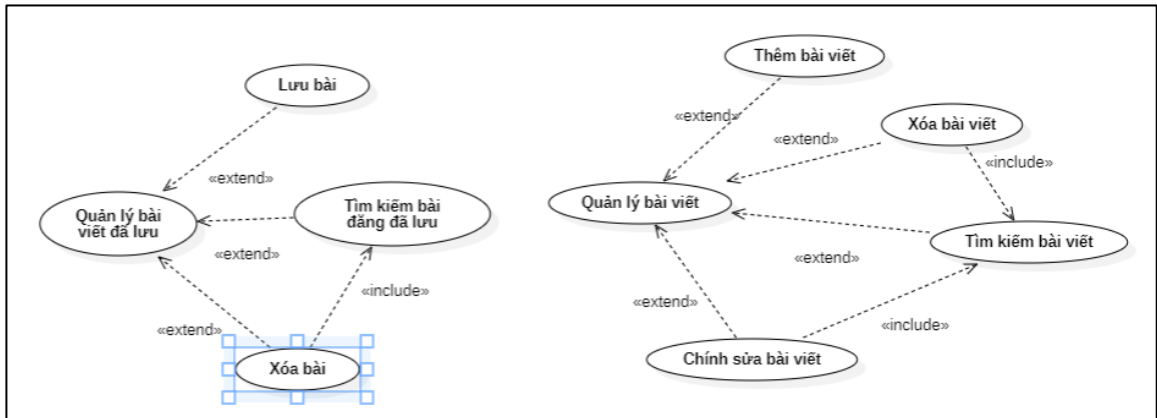
Để làm rõ hơn phạm vi chức năng của từng nhóm tác nhân, các use case trong sơ đồ tổng quát được phân rã thành các sơ đồ use case con theo từng nhóm chức năng chính.

3.3.1. Sơ đồ Use case quản lý người dùng



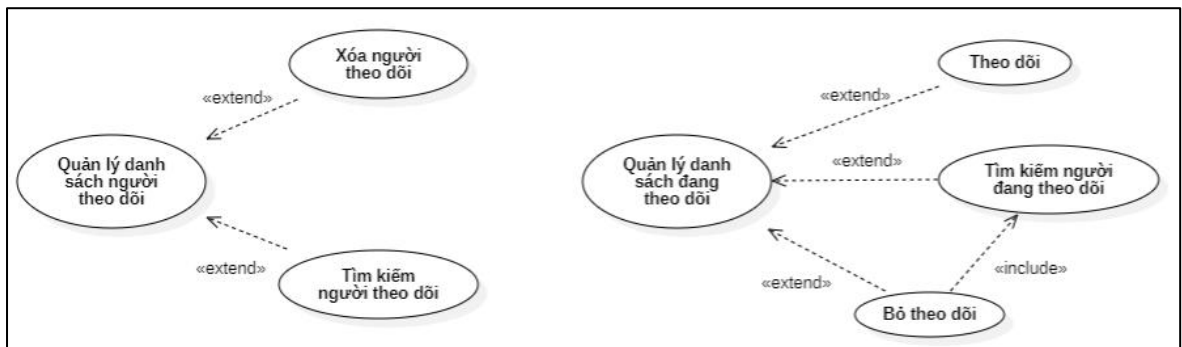
Hình 3.2. Sơ đồ use case quản lý người dùng

3.3.2. Sơ đồ Use case quản lý bài viết và bài viết đã lưu



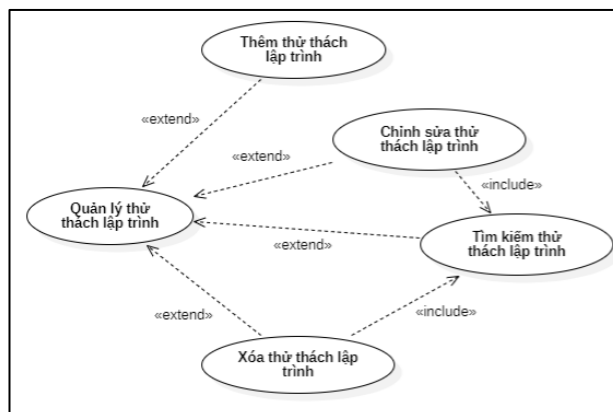
Hình 3.3. Sơ đồ use case quản lý bài viết và bài viết đã lưu

3.3.3. Sơ đồ Use case quản lý danh sách người theo dõi và đang theo dõi



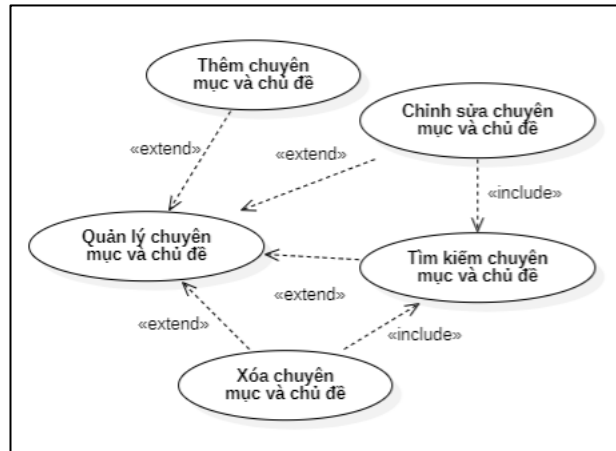
Hình 3.4. Sơ đồ use case quản lý danh sách người theo dõi và đang theo dõi

3.3.4. Sơ đồ Use case quản lý thử thách lập trình



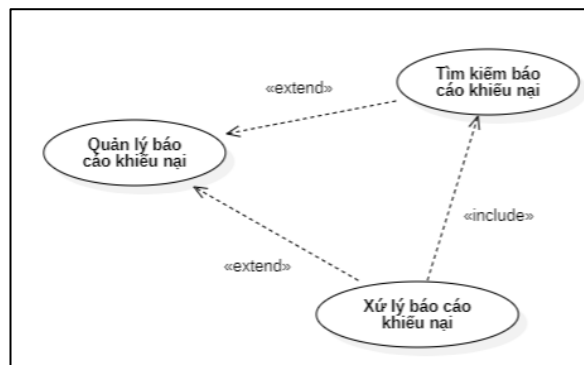
Hình 3.5. Sơ đồ use case quản lý thử thách lập trình

3.3.5. Sơ đồ Use case quản lý chuyên mục và chủ đề



Hình 3.6. Sơ đồ use case quản lý chuyên mục và chủ đề

3.3.6. Sơ đồ Use case quản lý báo cáo và khiếu nại



Hình 3.7. Sơ đồ use case quản lý báo cáo và khiếu nại

3.4. Đặc tả use case

Do số lượng use case của hệ thống tương đối nhiều, báo cáo chỉ trình bày chi tiết một số use case tiêu biểu và có vai trò quan trọng trong phạm vi Đồ án 2. Các use case nền tảng đã được nhóm đặc tả trong giai đoạn Đồ án 1, trong khi các use case còn lại được tổng hợp trong tài liệu tham chiếu riêng nhằm giúp người đọc có thể theo dõi đầy đủ hơn khi cần.

Liên kết tài liệu đặc tả use case: [TongHopDacTaUseCase](#)

3.4.1. Đặc tả Use case thêm bài viết

Bảng 3.2. Bảng đặc tả use case thêm bài viết

Tên use case	Thêm bài viết
Mô tả use case	Tính năng cho phép Người dùng có thể đăng bài viết mới lên diễn đàn.
Kích hoạt (Trigger)	Người dùng nhấn vào nút “Đăng bài” từ giao diện người dùng.
Actors	Người dùng
Use-case liên quan	Quản lý bài viết cá nhân
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống, tài khoản có quyền đăng bài viết.
Hậu điều kiện	Hệ thống thông báo “Đăng bài thành công”. Dữ liệu được cập nhật trong cơ sở dữ liệu.
Luồng sự kiện chính	1. Người dùng truy cập chức năng “Đăng bài viết mới”. 2. Hiện thị trang nhập chủ đề, nội dung của bài viết mới. 3. Người dùng nhập chủ đề, nội dung, ... cho bài viết. 4. Người dùng nhấn nút “Đăng bài” 5. Hệ thống kiểm tra tính hợp lệ của dữ liệu. 6. Hệ thống lưu bài viết mới vào cơ sở dữ liệu. Bài viết được xuất hiện trên diễn đàn
Luồng sự kiện thay thế	5a. Hệ thống thông báo “Nội dung nhập vào không hợp lệ”. Trong trường hợp này hệ thống yêu cầu thực hiện lại bước 3.

Luồng sự kiện ngoại lệ	Người dùng bấm nút “Hủy” hoặc đóng màn hình “Đăng bài viết mới”, use-case thêm bài viết dừng lại. Hệ thống gặp lỗi bất thường trong lúc xử lý dữ liệu, không thể kết nối cơ sở dữ liệu. Hiện thị thông báo “đã xảy ra lỗi hệ thống” và use-case dừng lại.
-------------------------------	---

3.4.2. Đặc tả Use case tham gia thử thách lập trình

Bảng 3.3. Bảng đặc tả use case tham gia thử thách lập trình

Tên use case	Tham gia thử thách lập trình
Mô tả use case	Tính năng cho phép người dùng có thể tham gia thử thách lập trình trên diễn đàn
Kích hoạt (Trigger)	Người dùng tìm thấy thử thách muốn tham gia và nhấn nút vào thử thách đó trong màn hình “Thử thách lập trình”
Actors	Người dùng
Use-case liên quan	Xem danh sách thử thách lập trình, Xem bảng xếp hạng
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống. Thử thách lập trình tồn tại và đang ở trạng thái cho phép tham gia.
Hậu điều kiện	Hệ thống ghi nhận bài nộp của người dùng. Kết quả chấm được lưu lại sau khi quá trình chấm hoàn tất. Thành tích của người dùng được cập nhật nếu bài nộp hợp lệ.

Luồng sự kiện chính	1. Người dùng truy cập vào màn hình thử thách lập trình và chọn một thử thách muốn tham gia. 2. Hệ thống hiển thị thông tin chi tiết của thử thách, bao gồm đề bài, loại thử thách, yêu cầu đầu vào, đầu ra hoặc yêu cầu xử lý tương ứng. 3. Người dùng thực hiện bài làm theo loại thử thách bằng cách nhập lời giải vào vùng lời giải và nhấn nút nộp bài. 4. Hệ thống tiếp nhận bài nộp và lưu thông tin bài nộp với trạng thái đang chấm. 5. Hệ thống gửi yêu cầu chấm bài đến Judge Worker. 6. Judge Worker xác định loại thử thách và thực hiện quá trình chấm tương ứng: a. Với bài thuật toán, hệ thống chạy mã nguồn với các test case và so sánh kết quả đầu ra. b. Với bài SQL, hệ thống thực thi truy vấn trên cơ sở dữ liệu kiểm thử, so sánh kết quả trả về và khôi phục dữ liệu sau khi chấm. c. Với bài backend Node.js/API, hệ thống chạy môi trường kiểm thử, gửi request đến các endpoint cần đánh giá và kiểm tra response theo yêu cầu. 7. Judge Worker gửi kết quả chấm về Backend. 8. Hệ thống cập nhật trạng thái bài nộp, lưu kết quả chấm và cập nhật điểm số hoặc thành tích của người dùng nếu phù hợp.
-----------------------------------	---

	9. Hệ thống hiển thị kết quả chấm cho người dùng, bao gồm trạng thái bài làm, số test case đạt, lỗi nếu có và thông tin điểm số.
Luồng sự kiện thay thế	Không có
Luồng sự kiện ngoại lệ	Người dùng thoát khỏi màn hình “Thử thách lập trình” hoặc nhấn nút “Hủy”, Use-case “Tham gia thử thách” dừng lại. Hệ thống gặp lỗi bất thường trong lúc xử lý dữ liệu, không thể kết nối cơ sở dữ liệu. Hiển thị thông báo “đã xảy ra lỗi hệ thống” và use-case dừng lại.

3.4.3. Đặc tả Use case thêm thử thách lập trình

Bảng 3.4. Bảng đặc tả use case thêm thử thách lập trình

Tên use case	Thêm thử thách lập trình
Mô tả use case	Cho phép quản trị viên tạo thử thách lập trình cho người dùng tham gia
Kích hoạt (Trigger)	Quản trị viên nhấn vào nút “Tạo thử thách lập trình” trên trang quản lý thử thách lập trình
Actors	Quản trị viên
Use-case liên quan	Quản lý thử thách lập trình
Tiền điều kiện	Người thực hiện đã đăng nhập vào hệ thống. Người thực hiện có quyền thêm thử thách lập trình. Hệ thống đang hoạt động bình thường.

Hậu điều kiện	Thử thách lập trình mới được tạo và lưu vào hệ thống. Thử thách được hiển thị trong danh sách quản lý. Nếu thử thách được đặt ở trạng thái công khai, người dùng có thể xem và tham gia thử thách.
Luồng sự kiện chính	1. Người thực hiện truy cập vào màn hình quản lý thử thách lập trình và chọn chức năng thêm. 2. Hệ thống hiển thị biểu mẫu thêm thử thách lập trình. 3. Người thực hiện nhập các thông tin cơ bản của thử thách, bao gồm tên thử thách, mô tả đề bài, mức độ, chủ đề, thời gian giới hạn, bộ nhớ giới hạn và trạng thái hiển thị. a. Với bài thuật toán, người thực hiện cấu hình ngôn ngữ hỗ trợ, dữ liệu đầu vào, kết quả mong đợi hoặc các test case tương ứng. b. Với bài SQL, người thực hiện cấu hình dữ liệu kiểm thử, câu truy vấn mẫu hoặc kết quả mong đợi để hệ thống so sánh khi chấm. c. Với bài backend Node.js/API, người thực hiện cấu hình yêu cầu API, endpoint cần kiểm thử, phương thức request, dữ liệu gửi lên và kết quả phản hồi mong đợi. 4. Người thực hiện kiểm tra lại thông tin và nhấn nút lưu thử thách. 5. Hệ thống kiểm tra tính hợp lệ của dữ liệu được nhập. 6. Hệ thống lưu thông tin thử thách và cấu hình chấm bài vào cơ sở dữ liệu.

	7. Hệ thống thông báo “Đã tạo thử thách thành công” và hiển thị trang quản lý thử thách lập trình.
Luồng sự kiện thay thế	5a. Hệ thống hiển thị thông báo người dùng nhập thông tin sai định dạng. Trường hợp này sẽ yêu cầu người dùng thực hiện lại bước 3.
Luồng sự kiện ngoại lệ	Quản trị viên bấm vào nút “hủy” hoặc đóng màn hình. Use case thêm thử thách lập trình dừng lại. Hệ thống gặp lỗi bất thường trong lúc xử lý dữ liệu, không thể kết nối cơ sở dữ liệu. Hiển thị thông báo “đã xảy ra lỗi hệ thống” và use case thêm thử thách lập trình dừng lại

3.4.4. Đặc tả Use case hỏi đáp với AI

Bảng 3.5. Bảng đặc tả use case hỏi đáp với AI

Tên use case	Hỏi đáp với AI
Mô tả use case	Cho phép người dùng đặt câu hỏi cho chatbot AI để nhận hỗ trợ trong quá trình học tập và thực hành lập trình.
Kích hoạt (Trigger)	Người dùng nhấn vào biểu tượng chatbot và nhập câu hỏi vào khung chat AI.
Actors	Người dùng
Use-case liên quan	Không có
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống. Hệ thống chatbot AI đang hoạt động bình thường.
Hậu điều kiện	Người dùng nhận được phản hồi từ AI.

	Lịch sử hội thoại được lưu lại nếu hệ thống có hỗ trợ lưu lịch sử.
Luồng sự kiện chính	1. Người dùng truy cập vào chức năng chatbot AI trên hệ thống. 2. Hệ thống hiển thị giao diện hội thoại với chatbot. 3. Người dùng nhập câu hỏi hoặc nội dung cần được hỗ trợ và nhấn nút gửi. 4. Hệ thống kiểm tra tính hợp lệ của câu hỏi. 5. Hệ thống phân tích câu hỏi và sinh ra câu trả lời. 6. Hệ thống lưu câu hỏi và câu trả lời vào cơ sở dữ liệu. 7. Hệ thống trả câu trả lời cho người dùng.
Luồng sự kiện thay thế	4a. Hệ thống hiển thị thông báo người dùng nhập tin nhắn không hợp lệ. Trường hợp này sẽ yêu cầu người dùng thực hiện lại bước 3.
Luồng sự kiện ngoại lệ	Quản trị viên bấm vào nút “hủy” hoặc đóng màn hình. Use case hỏi đáp với AI dừng lại. Hệ thống gặp lỗi bất thường trong lúc xử lý dữ liệu, không thể kết nối cơ sở dữ liệu. Hiển thị thông báo “đã xảy ra lỗi hệ thống” và use case hỏi đáp với AI dừng lại

3.4.5. Đặc tả Use case xử lý báo cáo khiếu nại

Bảng 3.6. Bảng đặc tả use case xử lý báo cáo khiếu nại

Tên use case	Xử lý báo cáo khiếu nại
Mô tả use case	Cho phép quản trị viên tiếp nhận, xem xét và xử lý các báo cáo/khiếu nại do người dùng gửi lên hệ thống.

Kích hoạt (Trigger)	Quản trị viên truy cập vào màn hình quản lý báo cáo khiếu nại và chọn một báo cáo cần xử lý.
Actors	Quản trị viên
Use-case liên quan	Quản lý báo cáo khiếu nại
Tiền điều kiện	Quản trị viên đã đăng nhập vào hệ thống. Quản trị viên có quyền xử lý báo cáo/khiếu nại. Báo cáo khiếu nại đã tồn tại trong hệ thống.
Hậu điều kiện	Báo cáo khiếu nại được cập nhật trạng thái xử lý. Hệ thống lưu lại kết quả xử lý của quản trị viên. Nội dung hoặc tài khoản liên quan được xử lý theo quyết định của quản trị viên nếu cần.
Luồng sự kiện chính	1. Quản trị viên truy cập vào màn hình quản lý báo cáo khiếu nại. 2. Hệ thống hiển thị danh sách báo cáo khiếu nại. 3. Quản trị viên chọn một báo cáo khiếu nại cần xem xét. 4. Hệ thống hiển thị chi tiết báo cáo, bao gồm nội dung báo cáo, đối tượng bị báo cáo và các thông tin liên quan. 5. Hệ thống hiển thị các lựa chọn xử lý phù hợp. 6. Quản trị viên đưa ra quyết định xử lý báo cáo. 7. Hệ thống cập nhật trạng thái báo cáo khiếu nại và lưu thông tin xử lý. 8. Hệ thống hiển thị thông báo xử lý thành công.

Luồng sự kiện thay thế	Không có.
Luồng sự kiện ngoại lệ	Quản trị viên bấm vào nút “hủy” hoặc đóng màn hình. Use case xử lý báo cáo khiếu nại dừng lại. Hệ thống gặp lỗi bất thường trong lúc xử lý dữ liệu, không thể kết nối cơ sở dữ liệu. Hiện thị thông báo “đã xảy ra lỗi hệ thống” và use case xử lý báo cáo khiếu nại dừng lại.

3.4.6. Đặc tả Use case tham gia cuộc trò chuyện

Bảng 3.7. Bảng đặc tả use case tham gia cuộc trò chuyện

Tên use case	Tham gia cuộc trò chuyện
Mô tả use case	Cho phép người dùng tham gia vào một cuộc trò chuyện để gửi và nhận tin nhắn với người dùng khác trong hệ thống
Kích hoạt (Trigger)	Người dùng chọn một cuộc trò chuyện trong danh sách cuộc trò chuyện hoặc bắt đầu cuộc trò chuyện với người dùng khác.
Actors	Người dùng
Use-case liên quan	Không có
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống. Cuộc trò chuyện tồn tại hoặc người dùng có quyền tạo cuộc trò chuyện mới. Người dùng là thành viên của cuộc trò chuyện.
Hậu điều kiện	Người dùng truy cập được vào giao diện cuộc trò chuyện.

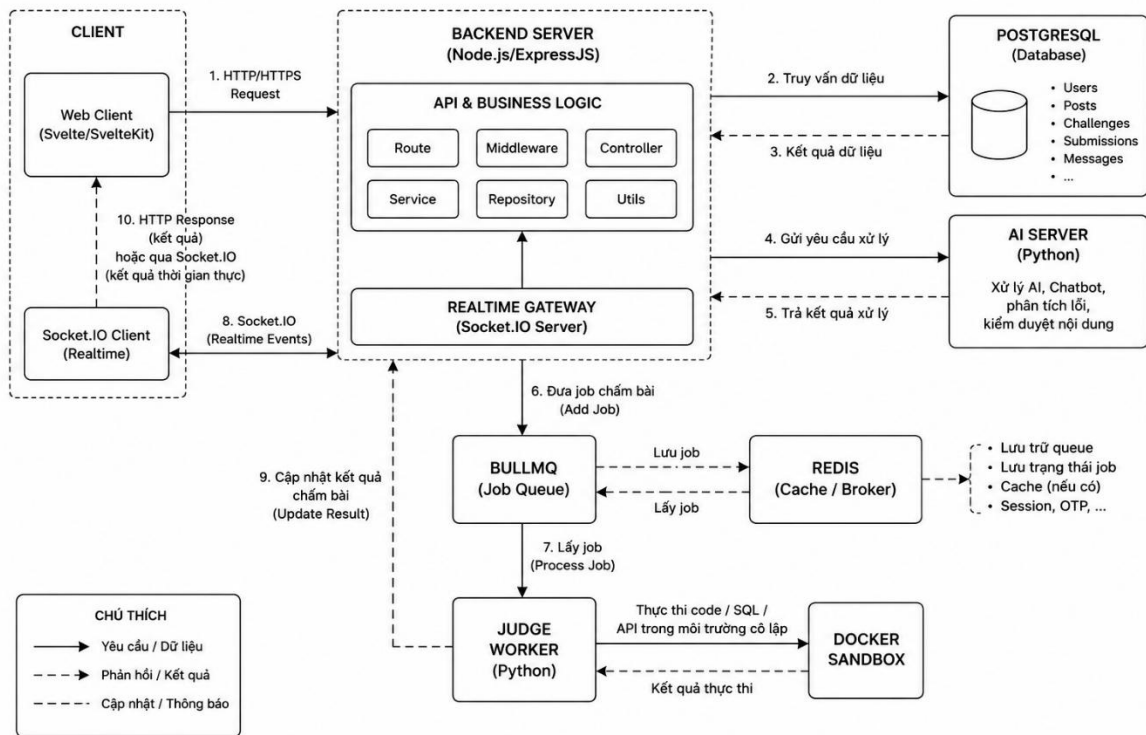
	Tin nhắn mới được gửi và hiển thị trong cuộc trò chuyện nếu thao tác gửi tin nhắn thành công. Lịch sử tin nhắn được cập nhật trong hệ thống.
Luồng sự kiện chính	1. Người dùng truy cập vào chức năng nhắn tin của hệ thống. 2. Hệ thống hiển thị danh sách các cuộc trò chuyện mà người dùng đang tham gia. 3. Người dùng chọn một cuộc trò chuyện muốn tham gia. 4. Hệ thống hiển thị giao diện và chi tiết cuộc trò chuyện. 5. Người dùng nhập nội dung tin nhắn vào vùng nhập tin nhắn và nhấn nút gửi. 6. Hệ thống kiểm tra tính hợp lệ của tin nhắn. 7. Hệ thống lưu tin nhắn và cơ sở dữ liệu và gửi tin nhắn trực tuyến cho các thành viên trong cuộc trò chuyện.
Luồng sự kiện thay thế	5a. Người dùng bấm vào cuộc trò chuyện khác. Trường hợp này hệ thống thực hiện lại bước 4. 6a. Hệ thống hiển thị thông báo người dùng nhập tin nhắn không hợp lệ. Trường hợp này sẽ yêu cầu người dùng thực hiện lại bước 5.
Luồng sự kiện ngoại lệ	Quản trị viên bấm vào nút “hủy” hoặc đóng màn hình. Use case tham gia cuộc trò chuyện dừng lại. Hệ thống gặp lỗi bất thường trong lúc xử lý dữ liệu, không thể kết nối cơ sở dữ liệu. Hiển thị thông báo “đã xảy ra lỗi hệ thống” và use case tham gia cuộc trò chuyện dừng lại.

3.5. Thiết kế kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình Client - Server, kết hợp với kiến trúc nhiều thành phần nhằm tách biệt rõ trách nhiệm giữa giao diện người dùng, xử lý nghiệp vụ, xử lý AI, chấm bài tự động và lưu trữ dữ liệu. Trong kiến trúc này, Frontend đóng vai trò là phía Client, Backend đóng vai trò là Server trung tâm, còn AI Server và Judge Worker là các thành phần xử lý chuyên biệt được tách riêng khỏi Backend chính.

3.5.1. Kiến trúc tổng thể

Sơ đồ kiến trúc tổng thể của hệ thống được trình bày ở hình sau.



Hình 3.8. Sơ đồ kiến trúc tổng thể

Các thành phần chính của hệ thống bao gồm:

Bảng 3.8. Bảng các thành phần chính của hệ thống

Thành phần	Công nghệ	Vai trò
Frontend	Svelte/SvelteKit	Cung cấp giao diện người dùng, tiếp nhận thao tác và gửi yêu cầu đến Backend
Backend	Node.js/ExpressJS	Xử lý nghiệp vụ chính, xác thực, phân quyền, cung cấp API và điều phối các thành phần khác
Database	PostgreSQL	Lưu trữ dữ liệu người dùng, bài viết, bình luận, cuộc trò chuyện, thử thách, bài nộp và kết quả chấm
AI Server	Python	Xử lý các chức năng liên quan đến chatbot AI, phân tích lỗi và kiểm duyệt nội dung
Redis	Redis	Lưu trữ dữ liệu tạm thời và làm nền tảng lưu trữ cho hàng đợi xử lý tác vụ nền
Job Queue	BullMQ	Quản lý hàng đợi chấm bài, giúp Backend gửi bài nộp sang Judge Worker theo cơ chế bất đồng bộ
Judge Worker	Javascript/ Docker	Lấy job chấm bài từ hàng đợi, thực thi quá trình chấm bài và cập nhật kết quả về Backend hoặc cơ sở dữ liệu

Docker Sandbox	Docker	Cô lập môi trường chạy mã nguồn hoặc truy vấn do người dùng nộp
Realtime	Socket.IO	Hỗ trợ nhắn tin và cập nhật dữ liệu theo thời gian thực

3.5.2. Mô hình phân lớp trong Backend

Backend được tổ chức theo hướng phân lớp nhằm giúp mã nguồn rõ ràng, dễ bảo trì và dễ mở rộng. Cấu trúc xử lý chính gồm các lớp:

Bảng 3.9. Bảng mô tả các lớp xử lý ở Backend

Lớp xử lý	Vai trò
Route	Định nghĩa endpoint và tiếp nhận request từ Frontend
Middleware	Kiểm tra xác thực, phân quyền và xử lý dữ liệu trung gian
Controller	Nhận request, gọi service phù hợp và trả response
Service	Xử lý logic nghiệp vụ chính của hệ thống và thao tác cơ sở dữ liệu

3.6. Thiết kế luồng xử lý chính

3.6.1. Luồng đăng nhập và xác thực người dùng

Luồng này mô tả quá trình người dùng đăng nhập vào hệ thống và được xác thực trước khi sử dụng các chức năng yêu cầu tài khoản.

Các bước xử lý:

1. Người dùng nhập thông tin đăng nhập trên giao diện Frontend.
2. Frontend gửi thông tin đăng nhập đến Backend qua API.

3. Backend kiểm tra tính hợp lệ của dữ liệu.
4. Backend truy vấn PostgreSQL để lấy thông tin tài khoản tương ứng.
5. Backend kiểm tra thông tin đăng nhập của người dùng. Nếu hợp lệ, Backend tạo token hoặc phiên đăng nhập cho người dùng.
6. Backend trả kết quả đăng nhập về cho Frontend.
7. Frontend lưu thông tin xác thực cần thiết và chuyển người dùng vào hệ thống.

3.6.2. Luồng đăng bài và kiểm duyệt nội dung

Luồng này mô tả quá trình người dùng tạo bài viết mới và hệ thống kiểm duyệt nội dung trước khi bài viết được hiển thị hoặc lưu ở trạng thái phù hợp.

Các bước xử lý:

1. Người dùng nhập nội dung bài viết trên Frontend.
2. Frontend gửi dữ liệu bài viết đến Backend.
3. Backend kiểm tra trạng thái đăng nhập và tính hợp lệ của nội dung.
4. Backend gửi nội dung bài viết đến AI Server để phân tích kiểm duyệt.
5. AI Server xử lý nội dung và trả kết quả đánh giá về Backend.
6. Backend dựa trên kết quả kiểm duyệt để xác định trạng thái bài viết.
7. Nếu nội dung hợp lệ, Backend lưu bài viết vào PostgreSQL và cho phép hiển thị.
8. Nếu nội dung có dấu hiệu vi phạm, Backend có thể từ chối bài viết hoặc lưu bài viết ở trạng thái chờ xử lý.
9. Backend trả kết quả xử lý về Frontend.
10. Frontend hiển thị thông báo tương ứng cho người dùng.

3.6.3. Luồng hỏi đáp với AI

Luồng này mô tả quá trình người dùng gửi câu hỏi đến chatbot AI và nhận phản hồi từ hệ thống.

Các bước xử lý:

1. Người dùng nhập câu hỏi trong giao diện chatbot.

2. Frontend gửi nội dung câu hỏi đến Backend.
3. Backend kiểm tra trạng thái đăng nhập và tính hợp lệ của nội dung.
4. Backend gửi câu hỏi cùng ngữ cảnh cần thiết đến AI Server.
5. AI Server phân tích câu hỏi và tạo phản hồi.
6. AI Server trả phản hồi về Backend.
7. Backend lưu lịch sử hỏi đáp vào PostgreSQL nếu cần.
8. Backend trả phản hồi về Frontend.
9. Frontend hiển thị câu trả lời của chatbot cho người dùng.

Trong trường hợp AI Server không phản hồi hoặc gặp lỗi, Backend trả về thông báo lỗi phù hợp để Frontend hiển thị cho người dùng.

3.6.4. Luồng nộp bài và đưa vào hàng đợi chấm bài

Luồng này mô tả quá trình người dùng nộp bài cho một thử thách lập trình và Backend đưa bài nộp vào hàng đợi chấm bài.

Các bước xử lý:

1. Người dùng chọn thử thách lập trình và nhập lời giải.
2. Người dùng nhấn nút nộp bài trên Frontend.
3. Frontend gửi bài nộp đến Backend.
4. Backend kiểm tra người dùng, thử thách, loại bài và dữ liệu bài nộp.
5. Backend tạo bản ghi bài nộp trong PostgreSQL với trạng thái đang chờ chấm hoặc đang chấm.
6. Backend tạo job chấm bài và đưa job vào BullMQ Queue.
7. BullMQ sử dụng Redis để lưu thông tin hàng đợi và trạng thái job.
8. Backend trả kết quả tiếp nhận bài nộp về Frontend.
9. Frontend hiển thị trạng thái bài nộp đang được xử lý.

3.6.5. Luồng xử lý chấm bài tự động

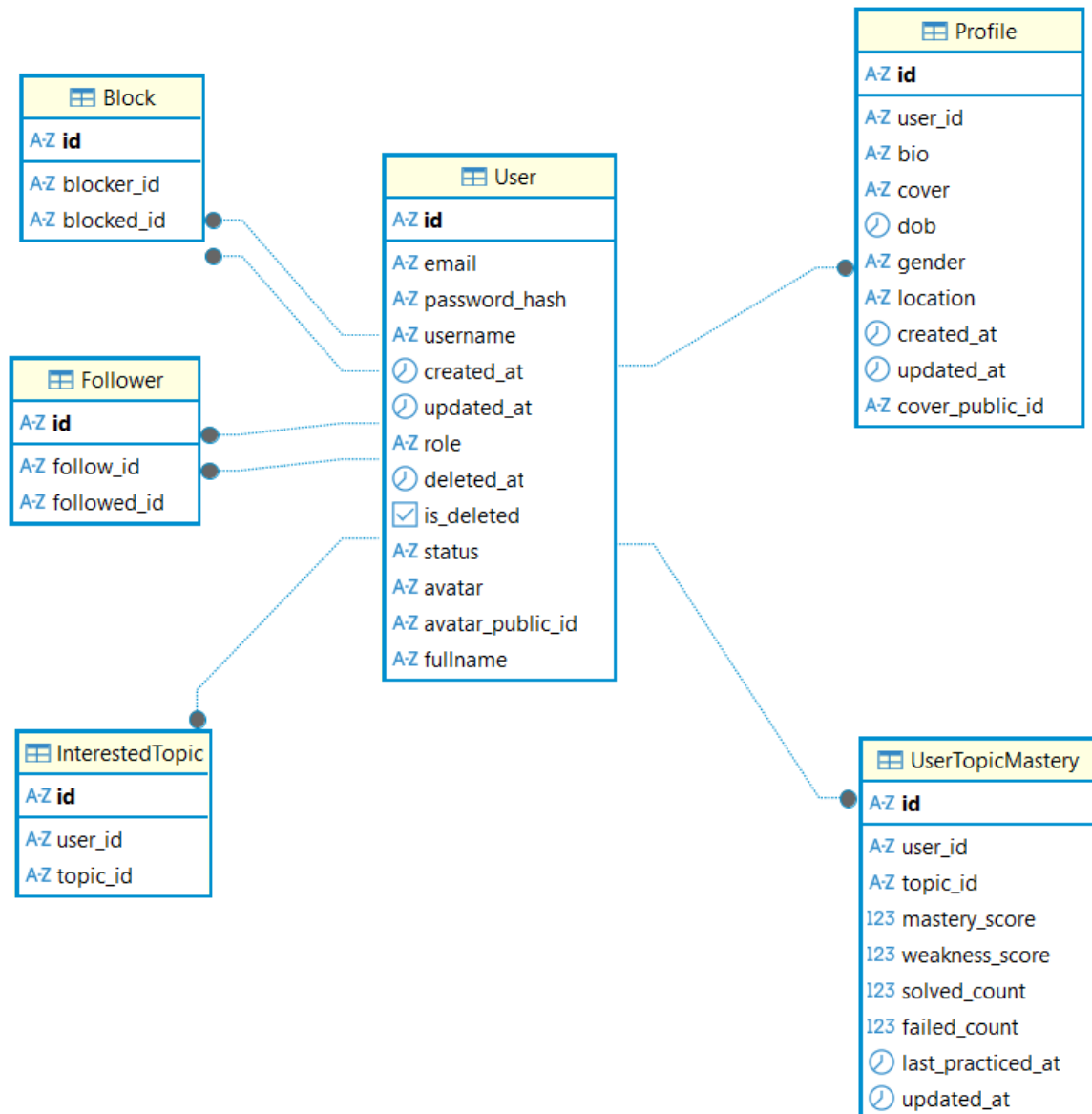
Luồng này mô tả quá trình Judge Worker lấy job từ hàng đợi, thực thi bài nộp trong môi trường cách ly và tạo kết quả chấm.

Các bước xử lý:

1. Judge Worker lấy job chấm bài từ BullMQ Queue.
2. Judge Worker đọc thông tin bài nộp, loại thử thách và cấu hình chấm bài.
3. Judge Worker chuẩn bị môi trường thực thi thông qua Docker Sandbox.
4. Judge Worker thực hiện chấm bài theo từng loại thử thách:
 - a. Đối với bài thuật toán, hệ thống chạy mã nguồn với bộ test case và so sánh kết quả đầu ra.
 - b. Đối với bài SQL, hệ thống thực thi truy vấn trên dữ liệu kiểm thử, so sánh kết quả trả về và khôi phục dữ liệu sau khi chấm.
 - c. Đối với bài backend Node.js/API, hệ thống khởi chạy môi trường kiểm thử, gửi request đến các endpoint và kiểm tra response theo yêu cầu.
5. Judge Worker tổng hợp kết quả chấm, bao gồm trạng thái, số test case đạt, lỗi nếu có, thời gian chạy và điểm số.
6. Judge Worker gửi kết quả chấm về Backend.
7. Backend cập nhật trạng thái bài nộp và lưu kết quả chi tiết vào PostgreSQL.
8. Backend có thể phát sự kiện realtime để thông báo cho Frontend rằng bài nộp đã có kết quả.

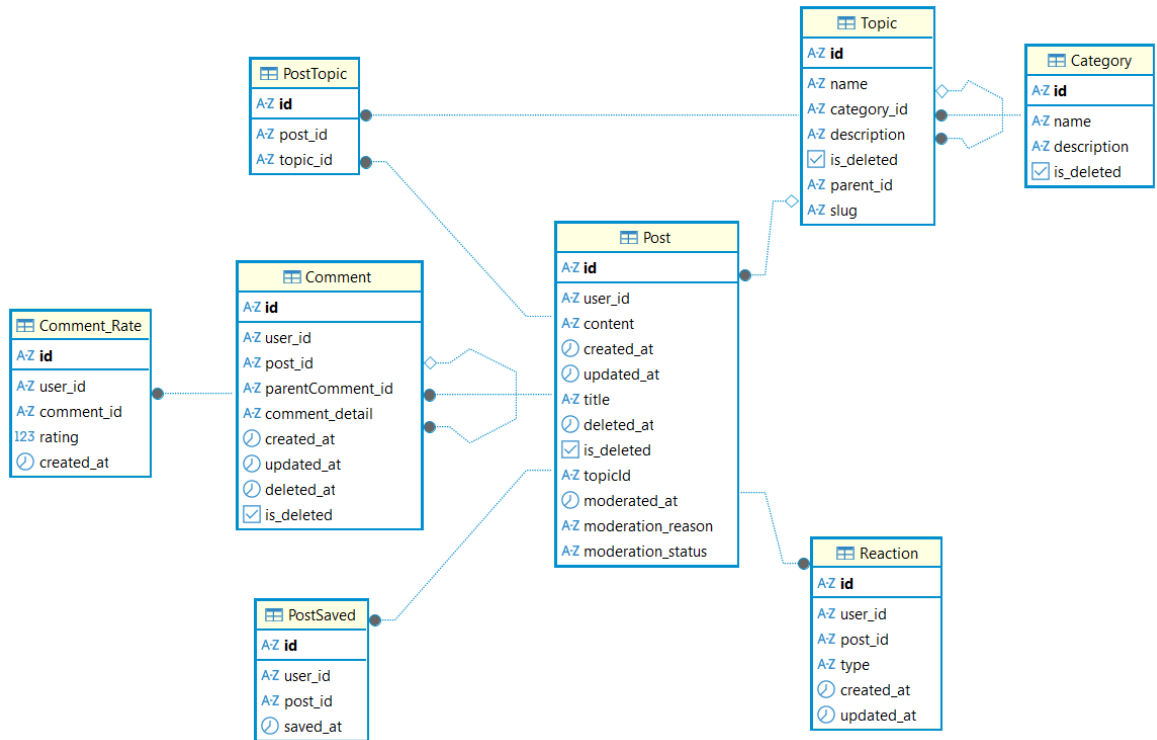
3.7. Thiết kế cơ sở dữ liệu

3.7.1. Nhóm bảng người dùng và cá nhân hóa



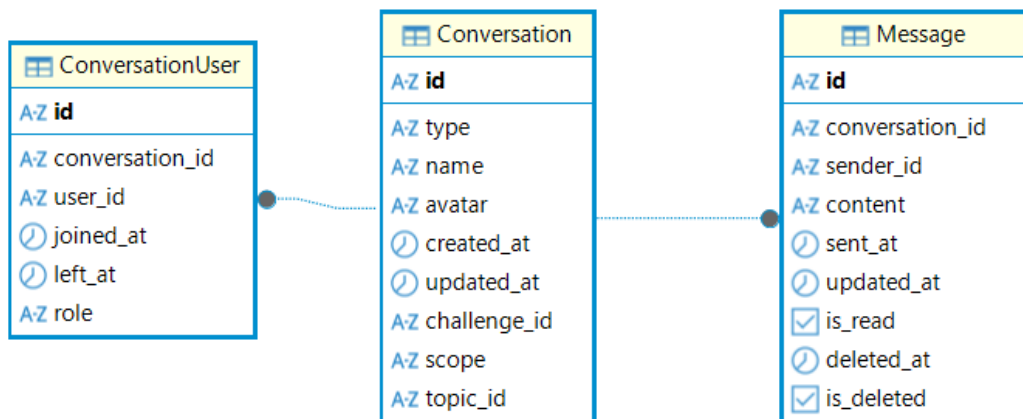
Hình 3.9. Sơ đồ cơ sở dữ liệu nhóm người dùng và cá nhân hóa

3.7.2. Nhóm bảng diễn đàn và tương tác nội dung



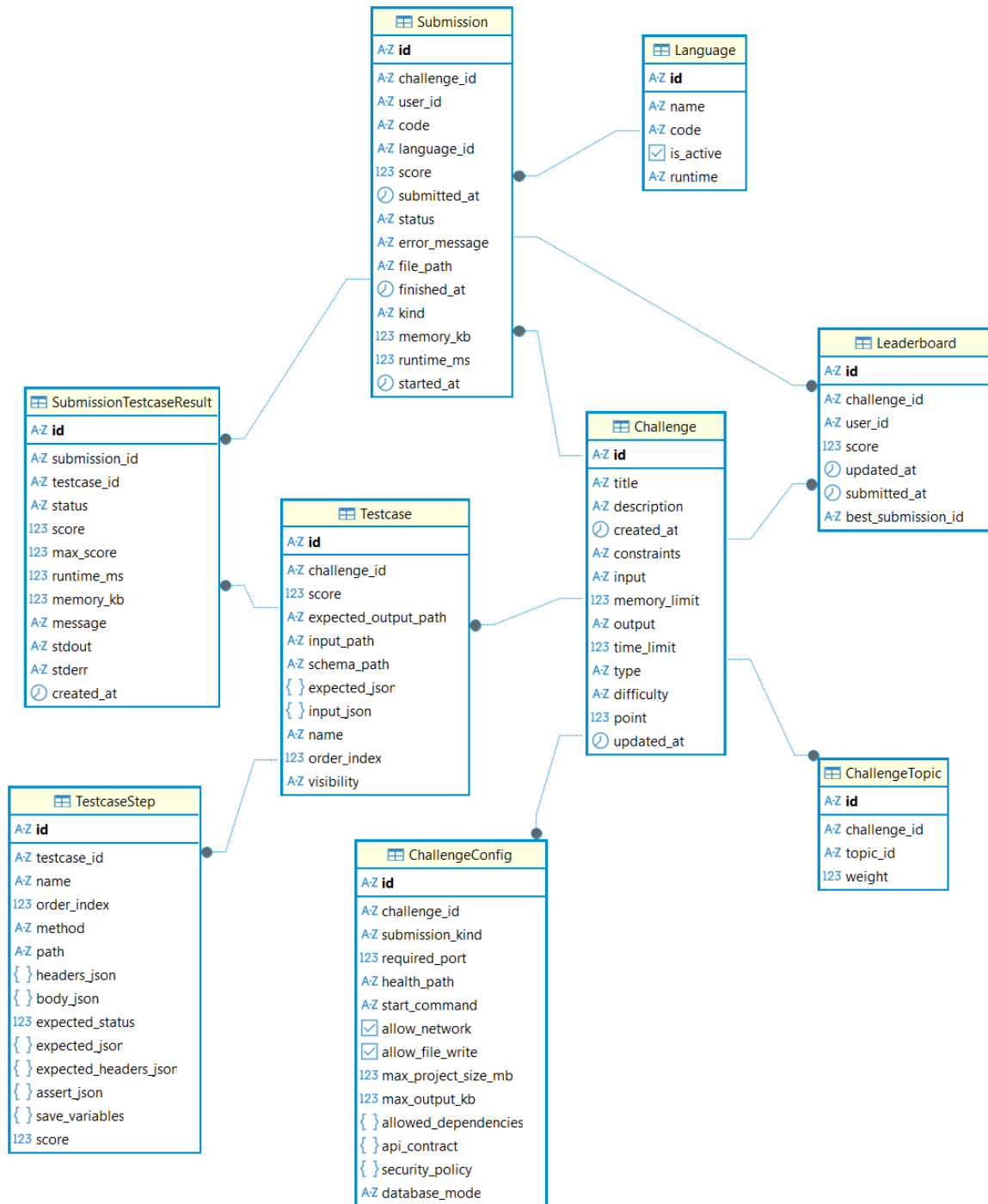
Hình 3.10. Sơ đồ cơ sở dữ liệu nhóm bảng diễn đàn và tương tác nội dung

3.7.3. Nhóm bảng trò chuyện thời gian thực



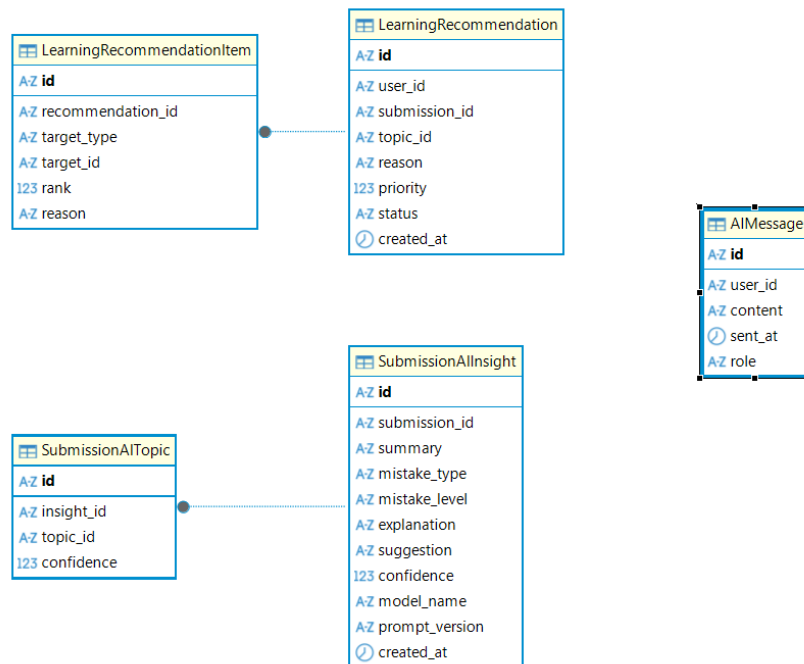
Hình 3.11. Sơ đồ cơ sở dữ liệu nhóm bảng trò chuyện thời gian thực

3.7.4. Nhóm bảng thử thách lập trình và chấm bài tự động



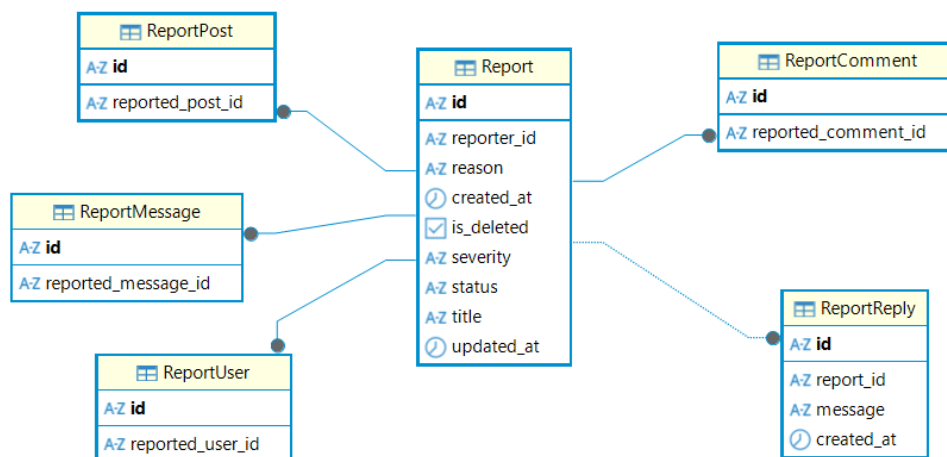
Hình 3.12. Sơ đồ cơ sở dữ liệu nhóm bảng thử thách lập trình

3.7.5. Nhóm bảng AI và gợi ý học tập



Hình 3.13. Sơ đồ cơ sở dữ liệu nhóm bảng AI và gợi ý học tập

3.7.6. Nhóm bảng báo cáo và kiểm duyệt



Hình 3.14. Sơ đồ cơ sở dữ liệu nhóm bảng báo cáo và kiểm duyệt

3.7.7. Mô tả chi tiết các bảng

Do số lượng bảng dữ liệu của hệ thống tương đối nhiều, báo cáo chỉ trình bày chi tiết các bảng chính có vai trò quan trọng trong các nghiệp vụ cốt lõi như người dùng, diễn đàn, trò chuyện thời gian thực, thử thách lập trình, chấm bài tự động và báo cáo khiếu nại. Các bảng phụ trợ hoặc bảng trung gian còn lại được mô tả đầy đủ trong tài liệu đặc tả cơ sở dữ liệu riêng nhằm giúp người đọc có thể tham khảo khi cần.

Liên kết tài liệu đặc tả chi tiết cơ sở dữ liệu: [TaiLieuMoTaChiTietCSDL](#)

3.7.7.1. Bảng User

Bảng 3.10. Bảng User

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh duy nhất của người dùng
2	email	String	Duy nhất, không null	Email đăng ký tài khoản
3	password_hash	String	Không null	Mật khẩu đã được mã hóa
4	username	String	Duy nhất, không null	Tên đăng nhập của người dùng
5	fullname	String	Có thể null	Họ tên đầy đủ của người dùng
6	avatar	String	Có thể null	Đường dẫn ảnh đại diện

7	avatar_public_id	String	Có thể null	Mã public_id của ảnh đại diện nếu lưu trên dịch vụ bên ngoài
8	created_at	DateTime	Mặc định thời điểm tạo	Thời điểm tạo tài khoản
9	updated_at	DateTime	Tự động cập nhật	Thời điểm cập nhật thông tin gần nhất
10	role	ROLE	Mặc định USER	Vai trò của người dùng trong hệ thống
11	is_deleted	Boolean	Mặc định false	Trạng thái xóa mềm của tài khoản
12	deleted_at	DateTime	Có thể null	Thời điểm tài khoản bị xóa mềm
13	status	USER_STATUS	Mặc định ACTIVE	Trạng thái hoạt động của tài khoản

3.7.7.2. Bảng Profile

Bảng 3.11. Bảng Profile

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh hồ sơ
2	user_id	String	Duy nhất, khóa ngoại tham chiếu User(id), không null	Người dùng sở hữu hồ sơ
3	bio	String	Có thể null	Giới thiệu ngắn về người dùng
4	cover	String	Có thể null	Đường dẫn ảnh bìa
5	cover_public_id	String	Có thể null	Mã public_id của ảnh bìa nếu lưu trên dịch vụ bên ngoài
6	dob	DateTime	Có thể null	Ngày sinh của người dùng
7	gender	GENDER_TYPE	Có thể null	Giới tính của người dùng

8	location	String	Có thể null	Vị trí hoặc địa chỉ khái quát của người dùng
9	created_at	DateTime	Mặc định thời điểm tạo	Thời điểm tạo hồ sơ
10	updated_at	DateTime	Tự động cập nhật	Thời điểm cập nhật hồ sơ gần nhất

3.7.7.3. Bảng Category

Bảng 3.12. Bảng Category

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh danh mục
2	name	String	Không null	Tên danh mục
3	description	String	Có thể null	Mô tả ngắn về danh mục
4	is_deleted	Boolean	Mặc định false	Trạng thái xóa mềm của danh mục

3.7.7.4. Bảng Topic

Bảng 3.13. Bảng Topic

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh chủ đề
2	name	String	Không null	Tên chủ đề
3	slug	String	Duy nhất, không null	Chuỗi định danh dùng cho URL hoặc tìm kiếm
4	category_id	String	Khóa ngoại tham chiếu Category(id), không null	Danh mục chứa chủ đề
5	parent_id	String	Có thể null, khóa ngoại tự tham chiếu Topic(id)	Chủ đề cha nếu có cấu trúc phân cấp
6	description	String	Có thể null	Mô tả chủ đề
7	is_deleted	Boolean	Mặc định false	Trạng thái xóa mềm của chủ đề

3.7.7.5. Bảng Post

Bảng 3.14. Bảng Post

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh bài viết
2	user_id	String	Khóa ngoại tham chiếu User(id), không null	Người dùng tạo bài viết
3	title	String	Không null	Tiêu đề bài viết
4	content	String	Không null	Nội dung bài viết
5	created_at	DateTime	Mặc định thời điểm tạo	Thời điểm tạo bài viết
6	updated_at	DateTime	Tự động cập nhật	Thời điểm cập nhật bài viết gần nhất
7	is_deleted	Boolean	Mặc định false	Trạng thái xóa mềm của bài viết
8	deleted_at	DateTime	Có thể null	Thời điểm bài viết bị xóa mềm

9	moderation_status	MODERATION STATUS	Mặc định NOT_REQUIRED	Trạng thái kiểm duyệt nội dung
10	moderation_reason	String	Có thể null	Lý do kiểm duyệt nếu có
11	moderated_at	DateTime	Có thể null	Thời điểm bài viết được kiểm duyệt
12	topicId	String	Có thể null, khóa ngoại tham chiếu Topic(id)	Chủ đề chính của bài viết nếu có

3.7.7.6. Bảng Comment

Bảng 3.15. Bảng Comment

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh bình luận
2	user_id	String	Khóa ngoại tham chiếu User(id), không null	Người dùng tạo bình luận
3	post_id	String	Khóa ngoại tham chiếu Post(id), không null	Bài viết chứa bình luận

4	parentComment_id	String	Có thể null, khóa ngoại tự tham chiếu Comment(id)	Bình luận cha nếu đây là phản hồi
5	comment_detail	String	Không null	Nội dung bình luận
6	created_at	DateTime	Mặc định thời điểm tạo	Thời điểm tạo bình luận
7	updated_at	DateTime	Tự động cập nhật	Thời điểm cập nhật bình luận gần nhất
8	is_deleted	Boolean	Mặc định false	Trạng thái xóa mềm của bình luận
9	deleted_at	DateTime	Có thể null	Thời điểm bình luận bị xóa mềm

3.7.7.7. Bảng Reaction

Bảng 3.16. Bảng Reaction

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh phản ứng
2	user_id	String	Khóa ngoại tham chiếu User(id), không null	Người dùng thực hiện phản ứng
3	post_id	String	Khóa ngoại tham chiếu Post(id), không null	Bài viết được phản ứng
4	type	REACTION_TYPE	Không null	Loại phản ứng như LOVE, HAHA, SAD, ANGRY
5	created_at	DateTime	Mặc định thời điểm tạo	Thời điểm tạo phản ứng
6	updated_at	DateTime	Tự động cập nhật	Thời điểm cập nhật phản ứng gần nhất

3.7.7.8. Bảng Conversation

Bảng 3.17. Bảng Conversation

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh cuộc trò chuyện
2	type	CONVERSATION TYPE	Mặc định CHAT	Loại cuộc trò chuyện
3	scope	CONVERSATION SCOPE	Mặc định GENERAL	Phạm vi
4	name	String	Có thể null	Tên cuộc trò chuyện nhóm
5	avatar	String	Có thể null	Ảnh đại diện
6	topic_id	String	Có thể null, tham chiếu Topic(id)	Chủ đề liên quan
7	challenge_id	String	Có thể null, tham chiếu Challenge(id)	Thử thách liên quan
8	created_at	DateTime	Mặc định thời điểm tạo	Thời điểm tạo cuộc trò chuyện
9	updated_at	DateTime	Tự động cập nhật	Thời điểm cập nhật cuộc trò chuyện gần nhất

3.7.7.9. Bảng Message

Bảng 3.18. Bảng Message

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh tin nhắn
2	conversation_id	String	Không null, tham chiếu Conversation(id)	Cuộc trò chuyện chứa tin nhắn
3	sender_id	String	Không null, tham chiếu User(id)	Người gửi tin nhắn
4	content	String	Có thể null	Nội dung tin nhắn
5	sent_at	DateTime	Mặc định thời điểm gửi	Thời điểm gửi tin nhắn
6	updated_at	DateTime	Tự động cập nhật	Thời điểm cập nhật tin nhắn gần nhất
7	is_read	Boolean	Mặc định false	Trạng thái đã đọc
8	is_deleted	Boolean	Mặc định false	Trạng thái xóa mềm của tin nhắn
9	deleted_at	DateTime	Có thể null	Thời điểm tin nhắn bị xóa mềm

3.7.7.10. Bảng Challenge

Bảng 3.19. Bảng Challenge

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh thử thách
2	title	String	Không null	Tên thử thách lập trình
3	description	String	Có thể null	Mô tả đề bài
4	input	String	Có thể null	Mô tả dữ liệu đầu vào
5	output	String	Có thể null	Mô tả dữ liệu đầu ra
6	constraints	String	Có thể null	Ràng buộc của bài toán
7	difficulty	ChallengeDifficulty	Mặc định EASY	Độ khó của thử thách
8	type	ChallengeType	Mặc định DSA	Loại thử thách: DSA, SQL, BACKEND hoặc CONTEST

9	point	Int	Mặc định 100	Số điểm tối đa của thử thách
10	time_limit	Int	Có thể null	Giới hạn thời gian chạy
11	memory_limit	Int	Có thể null	Giới hạn bộ nhớ
12	created_at	DateTime	Mặc định thời điểm tạo	Thời điểm tạo thử thách
13	updated_at	DateTime	Tự động cập nhật	Thời điểm cập nhật thử thách gần nhất

3.7.7.11. Bảng Language

Bảng 3.20. Bảng Language

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh	Mã định danh
2	name	String	Không null	Tên ngôn ngữ
3	code	String	Duy nhất, không null	Mã định danh ngôn ngữ
4	runtime	String	Có thể null	Môi trường runtime
5	is_active	Boolean	Mặc định true	Trạng thái cho phép sử dụng ngôn ngữ

3.7.7.12. Bảng Testcase

Bảng 3.21. Bảng Testcase

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh test case
2	challenge_id	String	Không null, tham chiếu Challenge(id)	Thử thách chứa test case
3	name	String	Có thể null	Tên test case
4	score	Int	Mặc định 1	Điểm của test case
5	visibility	TestcaseVisibility	Mặc định HIDDEN	Trạng thái hiển thị của test case
6	order_index	Int	Mặc định 0	Thứ tự thực hiện test case
7	input_json	Json	Có thể null	Dữ liệu đầu vào
8	expected_json	Json	Có thể null	Kết quả mong đợi
9	input_path	String	Có thể null	Đường dẫn file dữ liệu đầu vào

10	expected_output_path	String	Có thể null	Đường dẫn file kết quả mong đợi
11	schema_path	String	Có thể null	Đường dẫn schema kiểm thử nếu có

3.7.7.13. Bảng Submission

Bảng 3.22. Bảng Submission

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh bài nộp
2	challenge_id	String	Không null, tham chiếu Challenge(id)	Thử thách được nộp bài
3	user_id	String	Không null, tham chiếu User(id)	Người dùng nộp bài
4	language_id	String	Không null, tham chiếu Language(id)	Ngôn ngữ của bài nộp
5	kind	SubmissionKind	Mặc định CODE	Loại bài nộp
6	code	String	Có thể null	Mã nguồn được nộp

7	file_path	String	Có thể null	Đường dẫn file nộp nếu dạng file
8	status	STATUS	Mặc định PENDING	Trạng thái chấm bài
9	score	Int	Mặc định 0	Điểm số
10	submitted_at	DateTime	Mặc định thời điểm tạo	Thời điểm nộp bài
11	started_at	DateTime	Có thể null	Thời điểm bắt đầu chấm
12	finished_at	DateTime	Có thể null	Thời điểm chấm xong
13	runtime_ms	Int	Có thể null	Thời gian chạy của bài nộp
14	memory_kb	Int	Có thể null	Bộ nhớ sử dụng
15	error_message	String	Có thể null	Thông báo lỗi nếu quá trình chấm thất bại

3.7.7.14. Bảng Report

Bảng 3.23. Bảng Report

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	String	Khóa chính, tự sinh UUID	Mã định danh báo cáo
2	reporter_id	String	Khóa ngoại tham chiếu User(id), không null	Người dùng gửi báo cáo
3	title	String	Có thể null	Tiêu đề báo cáo
4	reason	String	Không null	Lý do báo cáo
5	status	ReportStatus	Mặc định OPEN	Trạng thái xử lý báo cáo
6	severity	ReportSeverity	Mặc định MEDIUM	Mức độ nghiêm trọng của báo cáo
7	created_at	DateTime	Mặc định thời điểm tạo	Thời điểm gửi báo cáo
8	updated_at	DateTime	Tự động cập nhật	Thời điểm cập nhật báo cáo gần nhất

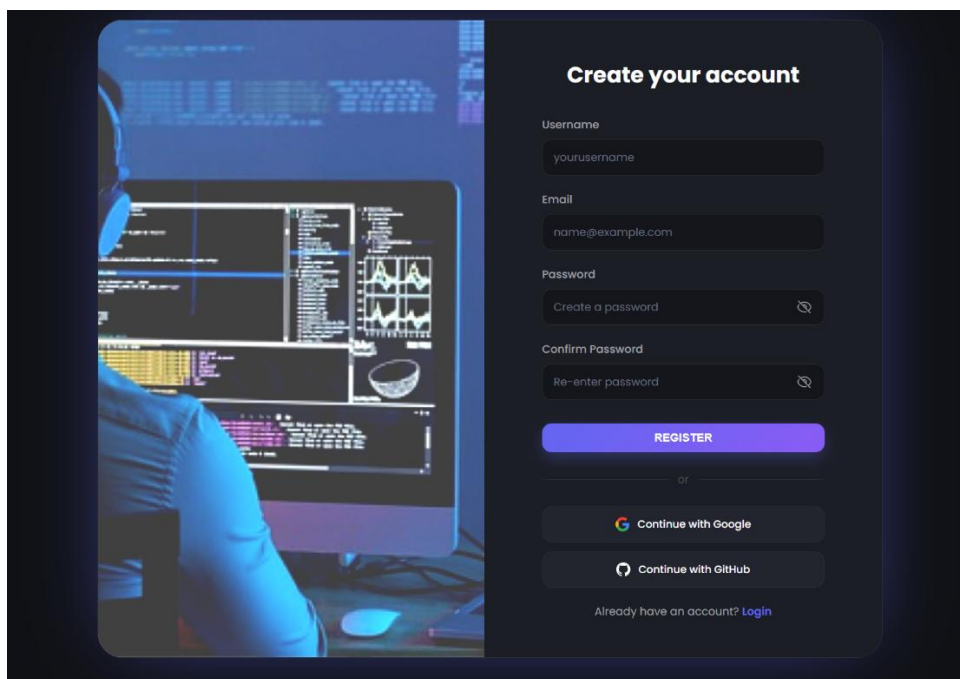
Chương 4. XÂY DỰNG VÀ TRIỂN KHAI ỨNG DỤNG

Chương 4 trình bày quá trình hiện thực hóa hệ thống dựa trên các phân tích và thiết kế đã được xác định ở chương trước. Nội dung chương mô tả chi tiết giao diện người dùng cùng các chức năng tương ứng, đồng thời làm rõ cách backend xử lý nghiệp vụ bên trong. Qua đó, chương cung cấp cái nhìn toàn diện về sản phẩm hoàn chỉnh, từ trải nghiệm phía người dùng đến logic vận hành của hệ thống..

4.1. Xây dựng giao diện người dùng

Nhóm thiết kế giao diện theo phong cách hiện đại, sử dụng tông màu tối. Mỗi trang được nhóm phân chia bố cục rõ ràng và có các biểu mẫu, gợi ý, thanh điều hướng phù hợp giúp cho người dùng dễ dàng thao tác và chuyển hướng giữa các trang.

4.1.1. Màn hình đăng ký



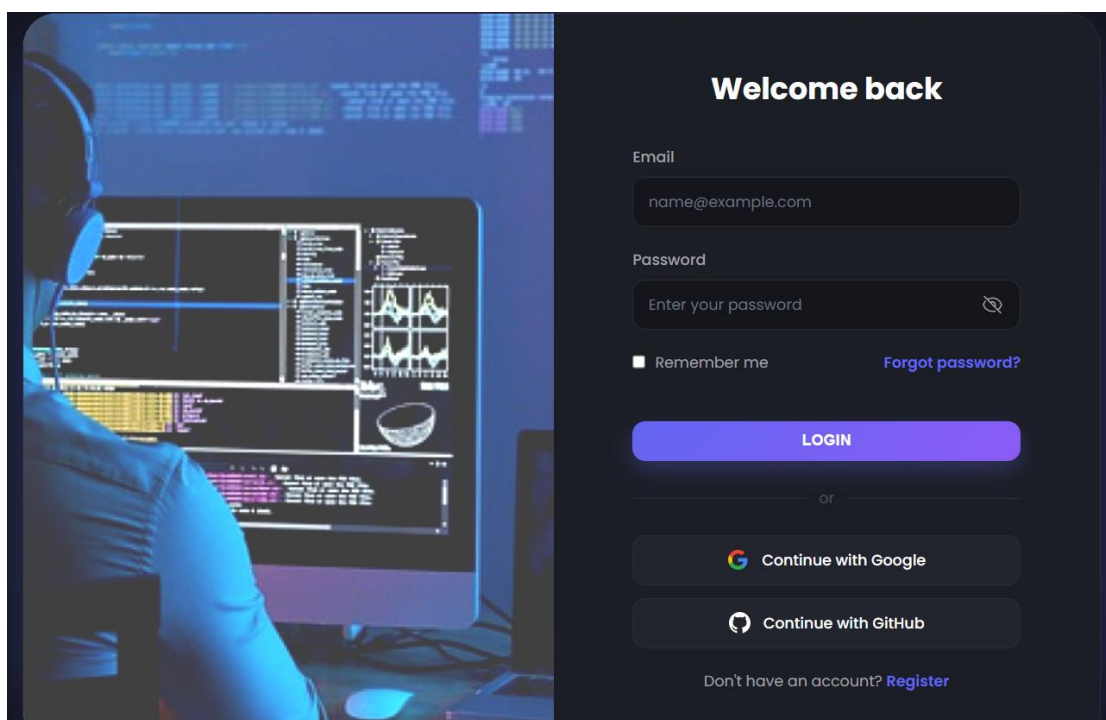
Hình 4.1. Màn hình đăng ký

Giao diện trang đăng ký được chia làm hai phần, bên trái là hình minh họa môi trường làm việc công nghệ nhằm gợi liên tưởng đến cộng đồng lập trình, bên phải là khu vực

biểu mẫu đăng ký với các trường thông tin cần thiết như tên người dùng, email, mật khẩu và xác nhận mật khẩu.

Phân dưới biểu mẫu, hệ thống thiết kế thêm hai nút là “Continue with Google” và “Continue with Github” hỗ trợ người dùng đăng ký nhanh thông qua Google và GitHub, giúp người dùng có thêm lựa chọn linh hoạt khi tạo tài khoản. Đồng thời là button chữ “Login” giúp người dùng chuyển sang trang Login một cách dễ dàng.

4.1.2. Màn hình đăng nhập

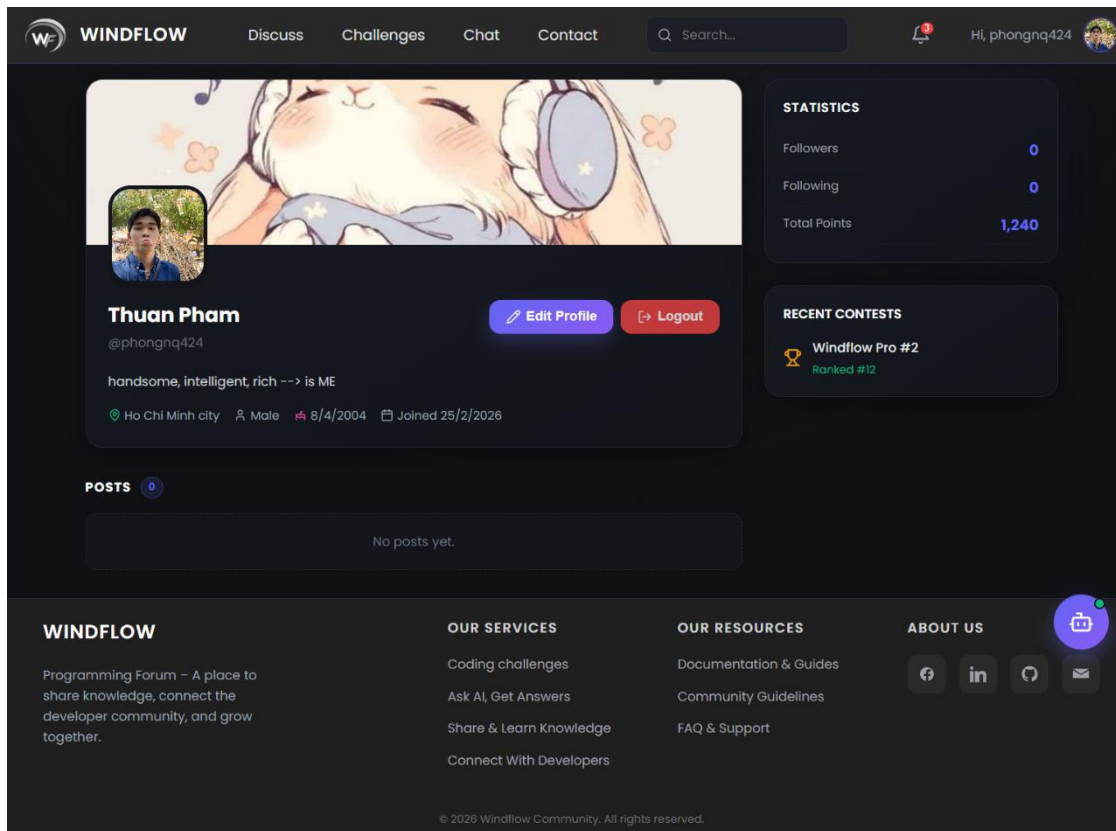


Hình 4.2. Màn hình đăng nhập

Tương tự với màn hình đăng ký, màn hình đăng nhập cũng có thiết kế bố cục tương tự, với hai phần được chia ra rõ rệt, đồng thời hỗ trợ đăng nhập nhanh và chuyển hướng sang trang đăng ký khi cần.

Biểu mẫu đăng nhập yêu cầu người dùng nhập hai trường thông tin là email và password.

4.1.3. Màn hình trang hồ sơ cá nhân



Hình 4.3. Màn hình trang hồ sơ cá nhân

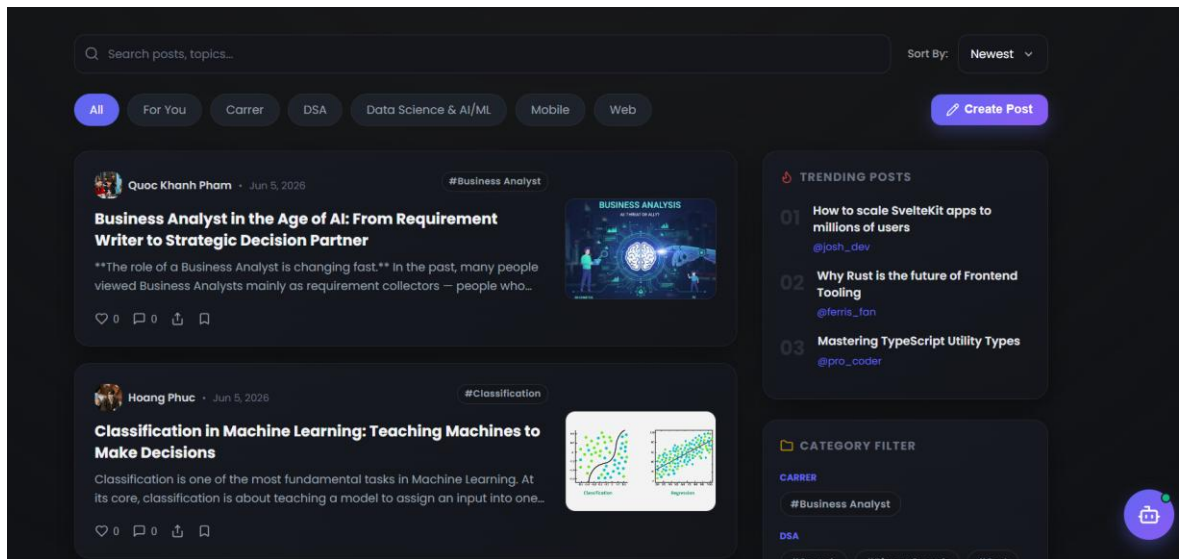
Màn hình trang hồ sơ được thiết kế giúp cho người dùng quản lý và theo dõi thông tin cá nhân một cách trực quan với bố cục gồm các khối với chức năng khác nhau.

Phần trung tâm hiển thị thẻ hồ sơ với ảnh bìa, ảnh đại diện, họ tên, tên tài khoản, mô tả ngắn và các thông tin cơ bản như vị trí, giới tính, ngày sinh và ngày tham gia hệ thống. Màn hình cung cấp các button “Edit Profile” và “Logout” giúp người dùng nhanh chóng chỉnh sửa thông tin cá nhân và đăng xuất khi cần.

Bên cạnh đó, giao diện còn có khu vực thống kê ở phía phải, hiển thị số người theo dõi, số người đang theo dõi và tổng điểm tích lũy, các cuộc thi và thứ hạng đạt được, giúp người dùng dễ dàng theo dõi mức độ hoạt động của mình trong cộng đồng.

Bên dưới hồ sơ là khu vực bài viết cá nhân, nơi hiển thị các nội dung mà người dùng đã đăng.

4.1.4. Màn hình trang thảo luận



Hình 4.4. Màn hình trang thảo luận

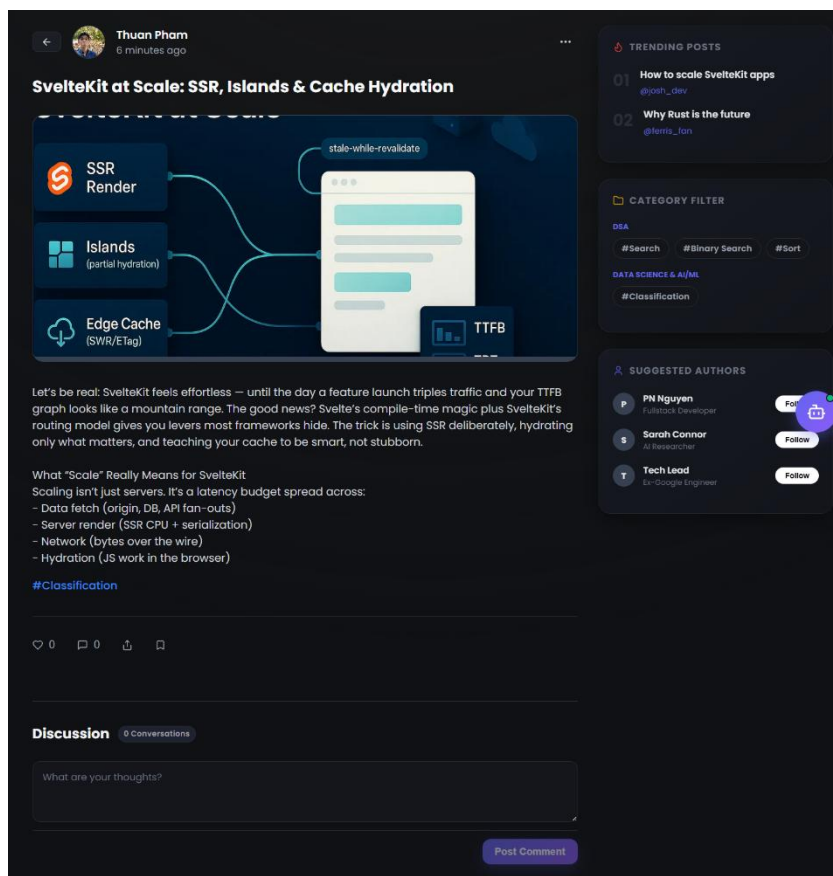
Màn hình trang thảo luận được thiết kế là khu vực chính để người dùng theo dõi, tìm kiếm và tương tác với các bài viết trong cộng đồng. Giao diện sử dụng bố cục gồm hai phần chính, trong đó phần bên trái là danh sách bài viết và phần bên phải là các khối thông tin hỗ trợ.

Ở phía trên danh sách bài viết có thanh tìm kiếm giúp người dùng tìm nhanh bài viết hoặc chủ đề quan tâm, bên dưới là các tag lọc nội dung nhanh giúp người dùng dễ dàng xem bài viết theo từng nhóm chủ đề. Ngoài ra, Góc phải phía trên còn có chức năng sắp xếp bài viết theo tiêu chí như mới nhất, đồng thời cung cấp button “Create Post” để người dùng tạo bài viết mới.

Phần danh sách bài viết được hiển thị dưới dạng các thẻ nội dung. Mỗi bài viết gồm thông tin người đăng, thời gian đăng, tiêu đề bài viết, mô tả ngắn, các hashtag liên quan và hình ảnh minh họa nếu có. Bên dưới mỗi bài viết có các biểu tượng tương tác như thích, bình luận, chia sẻ hoặc lưu bài viết, giúp người dùng thao tác nhanh mà không cần vào trang chi tiết ngay lập tức.

Bên phải màn hình là các khối thông tin bổ sung như “Trending Posts” hiển thị những bài viết đang nổi bật, “Category Filter” cho phép lọc nhanh theo danh mục và hashtag, cùng với “Suggested Authors” gợi ý những tác giả mà người dùng có thể theo dõi.

4.1.5. Màn hình chi tiết bài đăng



Hình 4.5. Màn hình chi tiết bài đăng

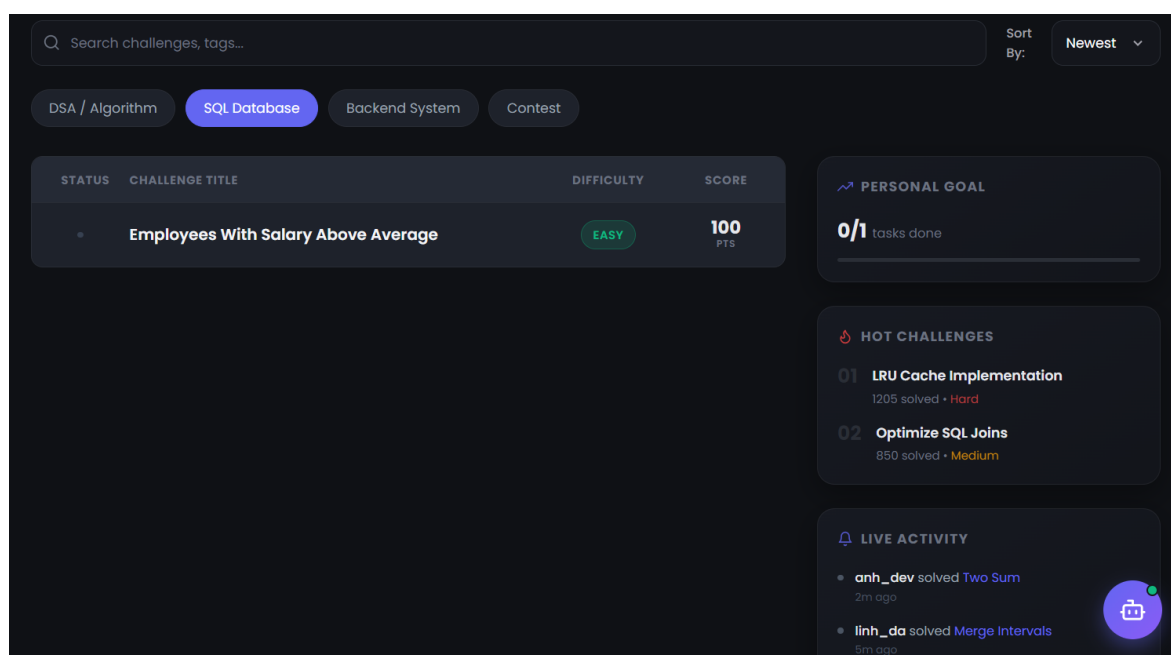
Màn hình chi tiết bài viết được thiết kế để người dùng xem đầy đủ nội dung của một bài đăng và thực hiện các thao tác tương tác liên quan. Ở phần đầu màn hình, hệ thống hiển thị thông tin người đăng kèm theo nút quay lại giúp người dùng trở về trang trước đó. Bên dưới là tiêu đề bài viết được trình bày nổi bật, tiếp theo là hình ảnh minh họa và phần nội dung chi tiết của bài viết.

Phần nội dung bài viết được bố trí theo dạng đọc tập trung, giúp người dùng dễ theo dõi nội dung chính mà không bị rối bởi quá nhiều thành phần phụ. Các hashtag liên

quan được hiển thị bên dưới nội dung để thể hiện chủ đề của bài viết và hỗ trợ việc phân loại. Sau phần bài viết là nhóm biểu tượng tương tác như thích, bình luận, chia sẻ và lưu bài viết, cho phép người dùng thao tác nhanh trực tiếp trên bài đăng.

Bên dưới khu vực tương tác là phần “Discussion”, nơi người dùng có thể nhập bình luận vào ô nội dung và gửi thông qua button “Post Comment”. Bố cục của phần thông tin bổ sung bên phải tương tự trang thảo luận.

4.1.6. Màn hình trang thử thách lập trình



Hình 4.6. Màn hình trang thử thách lập trình

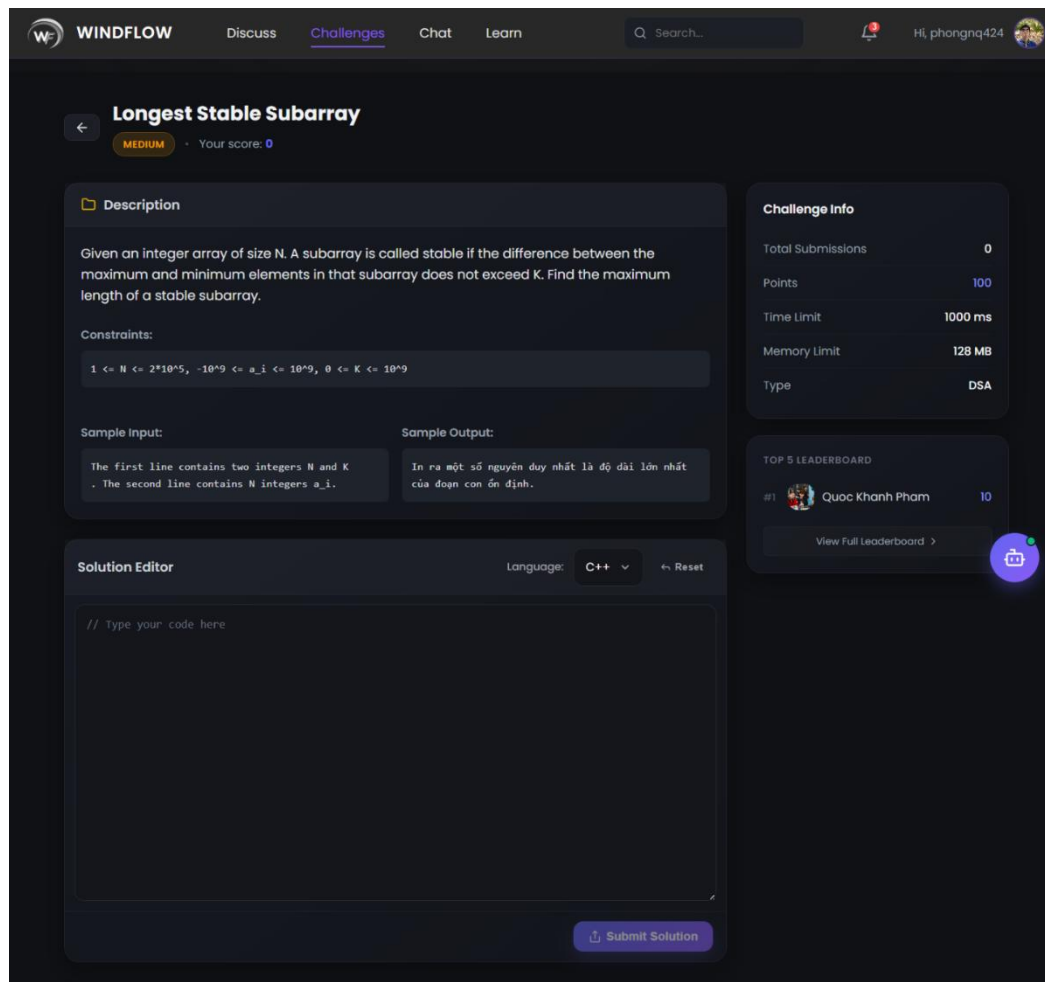
Màn hình trang thử thách lập trình được thiết kế giúp người dùng dễ dàng tìm kiếm, lọc và theo dõi các thử thách lập trình trong hệ thống. Giao diện sử dụng bố cục hai cột, trong đó cột trái là danh sách thử thách và cột phải là các khối thông tin hỗ trợ.

Phía trên danh sách có thanh tìm kiếm cho phép người dùng tìm kiếm theo tên thử thách hoặc tag, kết hợp với bộ lọc theo loại thử thách gồm DSA/Algorithm, SQL Database, Backend System và Contest. Góc phải phía trên cung cấp chức năng sắp xếp theo tiêu chí như mới nhất, giúp người dùng dễ dàng tìm thấy thử thách phù hợp.

Danh sách thử thách được hiển thị dạng bảng với các cột thông tin gồm trạng thái hoàn thành, tên thử thách, độ khó và điểm số tương ứng. Mỗi thử thách được gắn nhãn độ khó như Easy, Medium hoặc Hard với màu sắc phân biệt rõ ràng, giúp người dùng nhanh chóng đánh giá mức độ phù hợp trước khi tham gia.

Bên phải màn hình là các khối thông tin bổ sung gồm "Personal Goal" hiển thị tiến độ hoàn thành mục tiêu cá nhân, "Hot Challenges" liệt kê các thử thách đang được nhiều người tham gia nhất, và "Live Activity" cập nhật hoạt động thời gian thực của cộng đồng như thành viên vừa giải xong thử thách nào, giúp tạo không khí học tập sôi nổi và cạnh tranh lành mạnh.

4.1.7. Màn hình chi tiết thử thách lập trình



Hình 4.7. Màn hình chi tiết thử thách lập trình

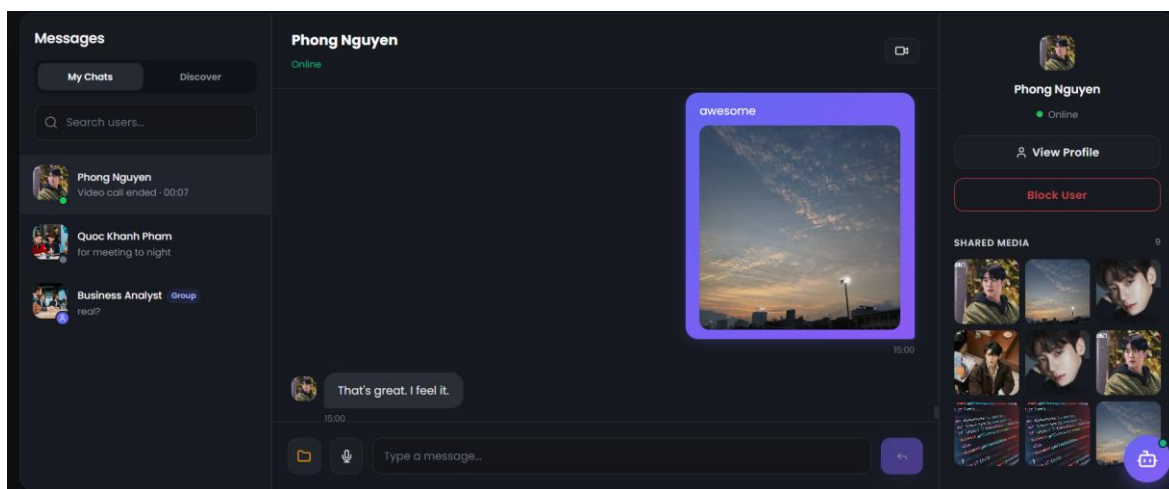
Màn hình chi tiết thử thách lập trình được thiết kế giúp người dùng nắm đầy đủ thông tin đề bài và thực hiện nộp lời giải trực tiếp trên cùng một giao diện. Bố cục gồm hai cột, trong đó cột trái chứa nội dung đề bài và trình soạn thảo code, cột phải hiển thị thông tin thử thách và bảng xếp hạng.

Phần đầu màn hình hiển thị tên thử thách, nhãn độ khó và điểm số hiện tại của người dùng. Bên dưới là khối mô tả đề bài bao gồm nội dung bài toán, ràng buộc, dữ liệu đầu vào mẫu và kết quả đầu ra mong đợi, giúp người dùng hiểu rõ yêu cầu trước khi bắt đầu làm bài.

Phần Solution Editor cho phép người dùng chọn ngôn ngữ lập trình và nhập lời giải trực tiếp vào trình soạn thảo code tích hợp. Sau khi hoàn thành, người dùng nhấn nút "Submit Solution" để nộp bài.

Cột bên phải hiển thị khối "Challenge Info" gồm các thông tin như tổng số lượt nộp, điểm tối đa, giới hạn thời gian, giới hạn bộ nhớ và loại thử thách. Bên dưới là bảng "Top 5 Leaderboard" hiển thị những người dùng đạt điểm cao nhất, kết hợp với liên kết xem toàn bộ bảng xếp hạng.

4.1.8. Màn hình trang nhắn tin



Hình 4.8. Màn hình trang nhắn tin

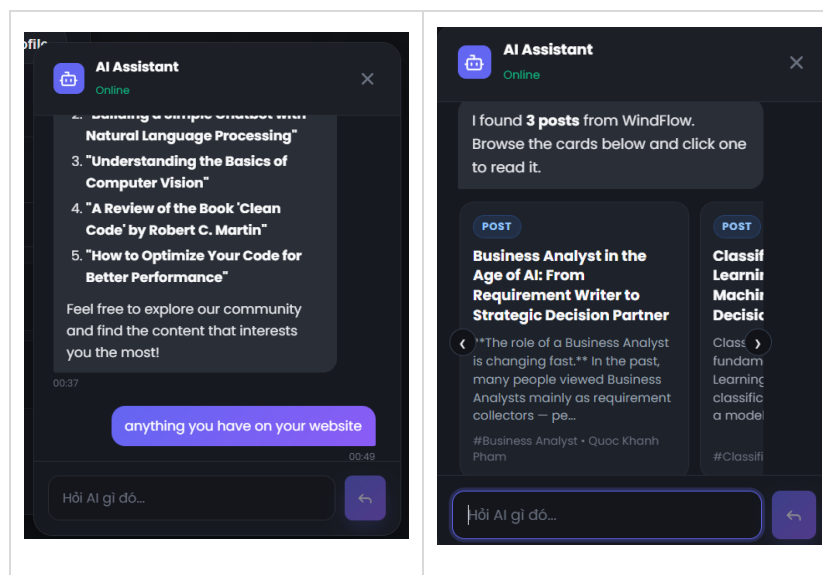
Màn hình trang nhắn tin được thiết kế hỗ trợ người dùng trao đổi trực tiếp với nhau theo thời gian thực. Giao diện được chia thành ba cột rõ ràng gồm danh sách cuộc trò chuyện, khu vực hiển thị tin nhắn và thông tin người dùng đối diện.

Cột bên trái hiển thị danh sách các cuộc trò chuyện hiện có, kèm theo thanh tìm kiếm để người dùng tìm nhanh theo tên. Mỗi cuộc trò chuyện hiển thị ảnh đại diện, tên người dùng và nội dung tin nhắn gần nhất.

Khu vực trung tâm là vùng hiển thị lịch sử tin nhắn, hỗ trợ gửi cả văn bản lẫn hình ảnh. Phía dưới là ô nhập tin nhắn cùng nút gửi và nút đính kèm tệp, giúp người dùng trao đổi thuận tiện. Ngoài ra hệ thống, còn cho phép người dùng call video với nhau qua button “Call video”.

Cột bên phải hiển thị thông tin người dùng, nút "View Profile" để xem hồ sơ, nút "Block User" để chặn người dùng và khối "Shared Media" hiển thị các tệp và hình ảnh đã được chia sẻ trong cuộc trò chuyện.

4.1.9. Màn hình hỏi đáp với chatbot



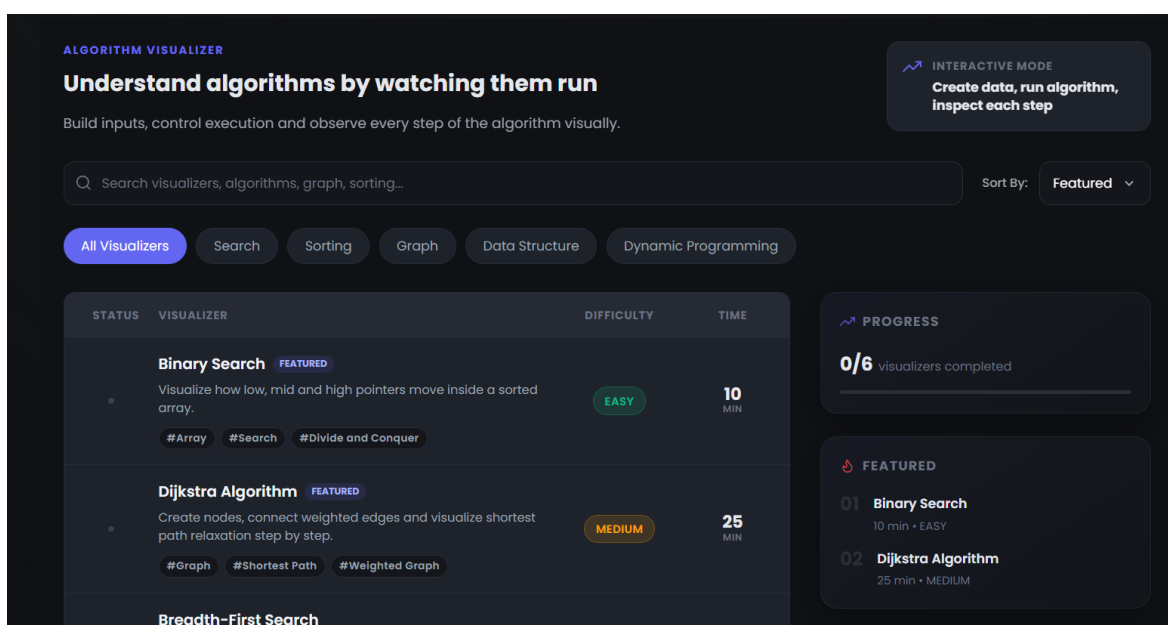
Hình 4.9. Màn hình hỏi đáp với chatbot

Màn hình hỏi đáp với chatbot được xây dựng nhằm hỗ trợ người dùng tương tác trực tiếp với hệ thống AI trong quá trình sử dụng diễn đàn. Giao diện cung cấp vùng nhập

tin nhắn để người dùng đặt các câu hỏi thông thường, câu hỏi liên quan đến lập trình hoặc các câu hỏi gắn với dữ liệu riêng của hệ thống.

Đối với các câu hỏi mang tính trao đổi thông thường, chatbot sẽ trả về phản hồi dạng văn bản trong khung hội thoại. Đối với các câu hỏi liên quan đến nội dung có sẵn trên diễn đàn, chẳng hạn như gợi ý bài viết, thử thách lập trình hoặc thông tin học tập, hệ thống có thể hiển thị thêm các thẻ gợi ý dưới dạng card. Mỗi card đại diện cho một thực thể cụ thể trong hệ thống và cho phép người dùng nhấn vào để chuyển trực tiếp đến trang chi tiết tương ứng.

4.1.10. Màn hình trang học tập



Hình 4.10. Màn hình trang học tập

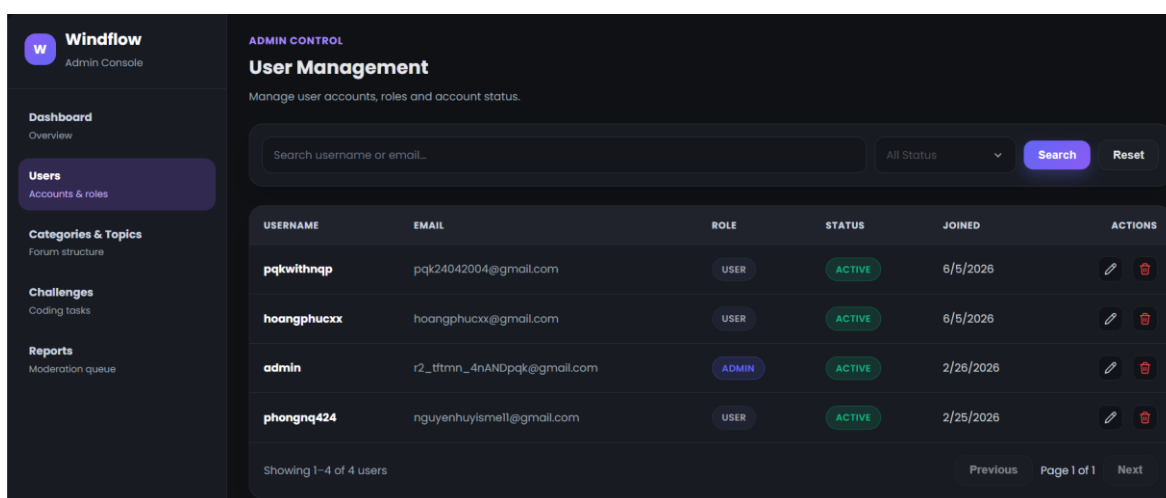
Màn hình trang học tập cung cấp công cụ trực quan hóa thuật toán giúp người dùng hiểu cách các thuật toán hoạt động thông qua việc quan sát từng bước thực thi.

Phía trên danh sách có thanh tìm kiếm và bộ lọc theo nhóm thuật toán gồm Search, Sorting, Graph, Data Structure và Dynamic Programming, giúp người dùng dễ dàng tìm kiếm bài học phù hợp.

Danh sách visualizer được hiển thị dạng bảng với các cột gồm trạng thái hoàn thành, tên và mô tả ngắn, độ khó và thời gian học ước tính. Mỗi visualizer được gắn các hashtag liên quan giúp người dùng nắm nhanh nội dung trước khi bắt đầu.

Cột bên phải hiển thị khối "Progress" theo dõi số lượng visualizer đã hoàn thành và khối "Featured" gợi ý các bài học nổi bật, giúp người dùng có định hướng học tập rõ ràng hơn.

4.1.11. Màn hình trang quản lý người dùng

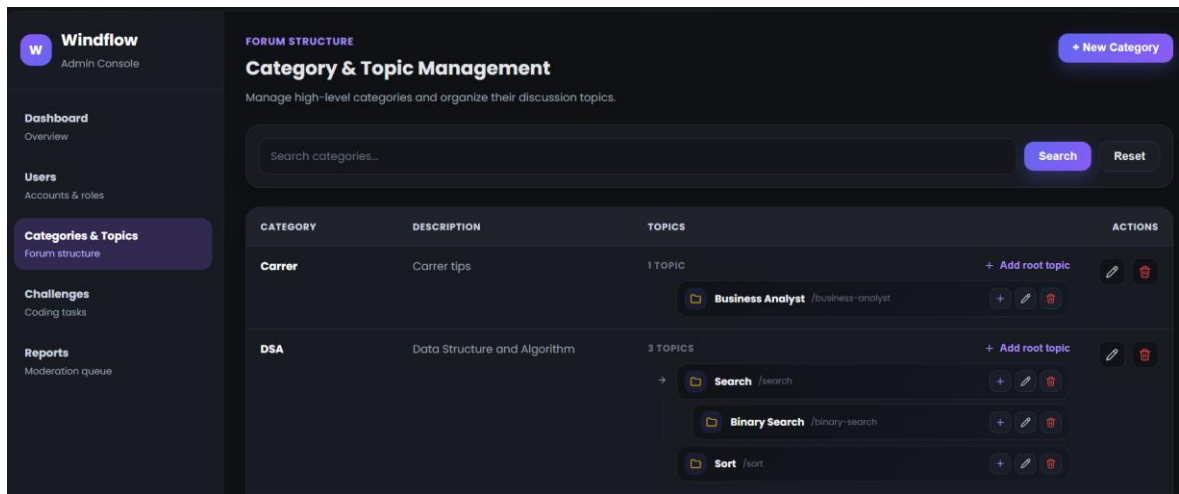


Hình 4.11. Màn hình trang quản lý người dùng

Màn hình trang quản lý người dùng thuộc khu vực Admin Console, cho phép quản trị viên theo dõi và quản lý toàn bộ tài khoản trong hệ thống. Giao diện bao gồm thanh điều hướng bên trái với các mục quản lý chính như Dashboard, Users, Categories, Topics, Challenges và Reports, giúp quản trị viên dễ dàng chuyển đổi giữa các phân hệ quản trị.

Khu vực chính hiển thị bảng danh sách người dùng với các cột thông tin gồm tên đăng nhập, email, vai trò, trạng thái tài khoản, ngày tham gia và các thao tác quản lý. Phía trên bảng có thanh tìm kiếm theo tên đăng nhập hoặc email, kết hợp với bộ lọc theo trạng thái tài khoản, giúp quản trị viên nhanh chóng tìm kiếm người dùng cần xử lý. Mỗi dòng trong bảng cung cấp nút chỉnh sửa và xóa tài khoản, hỗ trợ quản trị viên thao tác trực tiếp trên danh sách mà không cần chuyển sang trang khác.

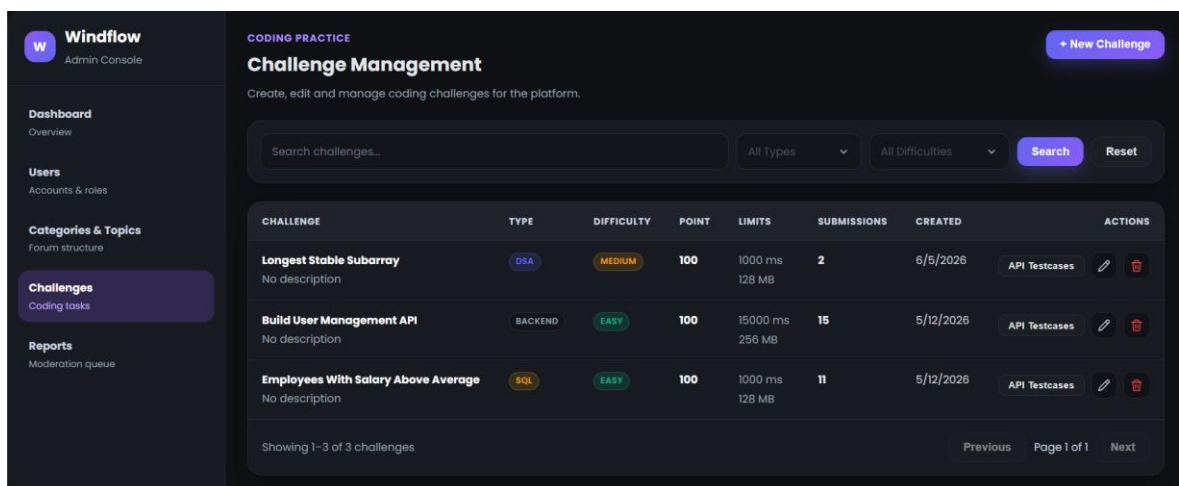
4.1.12. Màn hình trang quản lý chuyên mục và chủ đề



Hình 4.12. Màn hình trang quản lý chuyên mục và chủ đề

Tương tự với trang quản lý người dùng, khu vực chính hiển thị bảng danh sách các chuyên mục gồm tên danh mục, mô tả, danh sách chủ đề con và các thao tác quản lý. Với các chuyên mục ta có một cây chủ đề thuộc chuyên mục đó, quản trị viên có thể thêm chủ đề gốc mới thông qua nút "Add root topic", đồng thời chỉnh sửa hoặc xóa từng chủ đề trực tiếp trên danh sách thông qua các nút thao tác ở cuối mỗi dòng.

4.1.13. Màn hình trang quản lý thử thách lập trình



Hình 4.13. Màn hình trang quản lý thử thách lập trình

Màn hình quản lý thử thách lập trình cho phép quản trị viên theo dõi, tìm kiếm và quản lý toàn bộ các thử thách lập trình trong hệ thống. Phía trên có thanh tìm kiếm theo tên thử thách, kết hợp bộ lọc theo loại và độ khó, cùng nút "New Challenge" để tạo thử thách mới.

Danh sách thử thách được hiển thị dạng bảng với các cột gồm tên thử thách, loại bài, độ khó, điểm số, giới hạn thời gian và bộ nhớ, số lượt nộp bài, ngày tạo và các thao tác quản lý. Mỗi thử thách cung cấp nút "API Testcases" để quản lý bộ test case, nút chỉnh sửa và nút xóa, giúp quản trị viên kiểm soát nội dung thử thách một cách linh hoạt.

4.1.14. Màn hình thêm thử thách lập trình

Create Challenge [X]

Manage coding challenge content, scoring, difficulty and execution limits.

Title *

Enter challenge title

Type * **Difficulty ***

DSA Easy

Topics *

☐ Binary Search ☐ Business Analyst ☐ Classification ☐ Frontend ☐ RealTimeML

☐ RecommendationSystem ☐ Search ☐ Sort

Description

Describe the challenge

Cancel Create

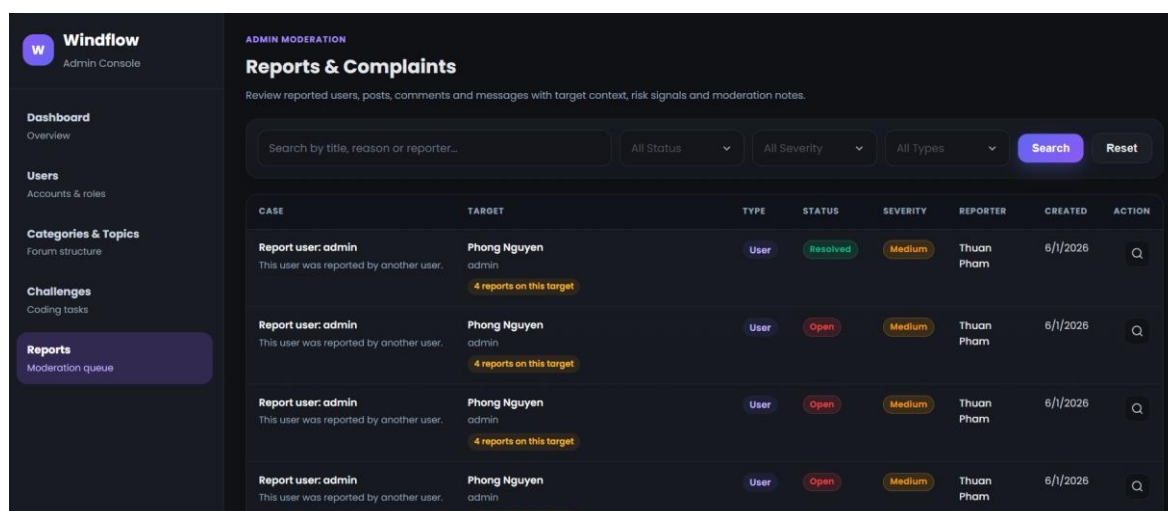
Hình 4.14. Màn hình thêm thử thách lập trình

Màn hình quản lý báo cáo khiếu nại cho phép quản trị viên tiếp nhận và xử lý các báo cáo do người dùng gửi lên liên quan đến tài khoản, bài viết, bình luận hoặc tin nhắn

vi phạm. Phía trên có thanh tìm kiếm theo tiêu đề, lý do hoặc người báo cáo, kết hợp bộ lọc theo trạng thái, mức độ nghiêm trọng và loại báo cáo.

Danh sách báo cáo được hiển thị dạng bảng với các cột gồm nội dung báo cáo, đối tượng bị báo cáo, loại, trạng thái xử lý, mức độ nghiêm trọng, người báo cáo, ngày tạo và nút xem chi tiết. Mỗi báo cáo được gắn nhãn trạng thái như Open hoặc Resolved với màu sắc phân biệt rõ ràng, giúp quản trị viên nhanh chóng nhận biết các trường hợp cần ưu tiên xử lý. Ngoài ra, hệ thống còn hiển thị thông tin tổng hợp về số lượng báo cáo trùng lặp trên cùng một đối tượng, hỗ trợ quản trị viên đánh giá mức độ vi phạm.

4.1.15. Màn hình quản lý báo cáo khiếu nại



CASE	TARGET	TYPE	STATUS	SEVERITY	REPORTER	CREATED	ACTION
Report user: admin This user was reported by another user.	Phong Nguyen admin 4 reports on this target	User	Resolved	Medium	Thuan Pham	6/1/2026	Q
Report user: admin This user was reported by another user.	Phong Nguyen admin 4 reports on this target	User	Open	Medium	Thuan Pham	6/1/2026	Q
Report user: admin This user was reported by another user.	Phong Nguyen admin 4 reports on this target	User	Open	Medium	Thuan Pham	6/1/2026	Q
Report user: admin This user was reported by another user.	Phong Nguyen admin 4 reports on this target	User	Open	Medium	Thuan Pham	6/1/2026	Q

Hình 4.15. Màn hình quản lý báo cáo khiếu nại

Màn hình xử lý báo cáo khiếu nại hiển thị dưới dạng hộp thoại chi tiết khi quản trị viên chọn xem một báo cáo cụ thể. Giao diện được chia thành hai phần chính, phần bên trái hiển thị thông tin đối tượng bị báo cáo bao gồm tên, mã định danh, loại đối tượng và nút "Open Target" để truy cập trực tiếp vào nội dung bị báo cáo. Phần bên phải hiển thị thông tin người gửi báo cáo gồm ảnh đại diện, tên tài khoản và mã định danh.

Bên dưới là khối "Risk Signal" cung cấp thông tin phân tích tự động của hệ thống, bao gồm tần suất báo cáo trên đối tượng, mức độ nghiêm trọng hiện tại và mức độ được hệ thống đề xuất nâng lên dựa trên số lần bị báo cáo. Hệ thống cũng hiển thị gợi ý để quản trị viên xem xét lại mức độ xử lý, đồng thời nhấn mạnh rằng quyết định cuối cùng vẫn thuộc về quản trị viên.

4.1.16. Màn hình xử lý báo cáo khiếu nại

The screenshot displays a 'Report Case' interface with a dark theme. It is divided into several sections:

- Reported Target:** Includes the name 'Phong Nguyen', a 'TARGET ID' (723a945d-fa76-4189-b652-a540a4c2624d), a 'TARGET TYPE' (User), and a username 'admin'. An 'Open Target' button is located at the bottom right of this section.
- Reporter:** Includes a profile picture, the name 'Thuan Pham', a handle '@phongnq424', and a 'REPORTER ID' (a20b8ebf-6831-41c1-98e9-757ec8a9c51e).
- Risk Signal:** A section with a yellow header. It contains the text 'Target report frequency' and 'This target has repeated reports.' To the right, a badge indicates '4 reports'. Below this, there are two boxes for severity: 'CURRENT SEVERITY' (Medium) and 'RECOMMENDED' (High). A note at the bottom states: 'System signal suggests reviewing severity. Admin should still make the final moderation decision.'

Hình 4.16. Màn hình xử lý báo cáo khiếu nại

4.2. Xử lý nghiệp vụ hệ thống

Bên cạnh phần giao diện người dùng, hệ thống còn được xây dựng các luồng xử lý nghiệp vụ ở Backend nhằm tiếp nhận yêu cầu từ Frontend, kiểm tra dữ liệu, thực hiện xử lý cần thiết và trả kết quả về cho người dùng. Backend được tổ chức theo mô hình phân lớp gồm Route, Middleware, Controller và Service. Route định nghĩa các endpoint, Middleware xử lý xác thực và các kiểm tra trung gian, Controller tiếp nhận request và điều phối xử lý, còn Service đảm nhiệm phần logic nghiệp vụ chính, truy

xuất cơ sở dữ liệu và giao tiếp với các thành phần chuyên biệt như AI Server hoặc Judge Worker.

4.2.1. Xử lý đăng nhập và xác thực người dùng

Nhóm API xác thực được sử dụng để hỗ trợ người dùng đăng ký, đăng nhập, xác minh OTP, đăng xuất và sử dụng các chức năng yêu cầu tài khoản. Đây là nhóm API nền tảng vì hầu hết các chức năng chính như đăng bài, bình luận, sử dụng chatbot, nộp bài và xem tiến độ cá nhân đều cần xác định người dùng đang đăng nhập.

Bảng 4.1. Nhóm API chức năng xử lý đăng nhập và xác thực người dùng

Chức năng	Phương thức	Endpoint
Gửi OTP	POST	/api/auth/send-otp
Xác thực OTP	POST	/api/auth/verify-otp
Đăng ký	POST	/api/auth/register
Đăng nhập	POST	/api/auth/login
Đăng xuất	POST	/api/auth/logout

Đối với chức năng đăng ký, email dùng để đăng ký tài khoản mới cần được xác thực trước khi đăng ký.

Đối với chức năng đăng nhập, kết quả trả về khi đăng nhập thành công gồm thông tin người dùng và thông tin xác thực cần thiết để Frontend lưu trạng thái đăng nhập.

4.2.2. Xử lý bài viết và kiểm duyệt nội dung

Chức năng bài viết là một trong những chức năng cốt lõi của diễn đàn, cho phép người dùng tạo bài viết, xem danh sách bài viết, xem chi tiết bài viết, chỉnh sửa, xóa mềm và tương tác với nội dung. Trong Đề án 2, nghiệp vụ bài viết được mở rộng

thêm bước kiểm duyệt nội dung tự động bằng AI nhằm hạn chế các nội dung vi phạm như spam, nội dung độc hại hoặc nội dung không phù hợp xuất hiện trên hệ thống.

Các API chính của nhóm chức năng bài viết gồm:

Bảng 4.2. Nhóm API backend chức năng xử lý bài viết và kiểm duyệt nội dung

Chức năng	Phương thức	Endpoint
Tạo bài viết	POST	/api/posts
Lấy danh sách bài viết	GET	/api/posts
Lấy chi tiết bài viết	GET	/api/posts/:id
Cập nhật bài viết	PATCH	/api/posts/:id
Xóa bài viết	DELETE	/api/posts/:id

Dữ liệu đầu vào của API tạo bài viết gồm tiêu đề, nội dung, chủ đề, danh mục và danh sách tệp đính kèm nếu có. Với nội dung văn bản, Backend gửi dữ liệu sang AI Server thông qua endpoint kiểm duyệt văn bản. Với hình ảnh, Backend gửi file ảnh sang endpoint kiểm duyệt hình ảnh.

Bảng 4.3. Nhóm API kiểm duyệt của AI Server

Chức năng	Phương thức	Endpoint
Kiểm duyệt nội dung văn bản	POST	/moderate/text
Kiểm duyệt hình ảnh	POST	/moderate/image

AI Server trả về kết quả đánh giá an toàn của nội dung thông qua trường `is_safe`.

4.2.3. Xử lý hỏi đáp với chatbot AI

Nhóm xử lý chatbot AI cho phép người dùng gửi câu hỏi, nhận phản hồi từ AI, xem lại lịch sử hội thoại và yêu cầu AI phân tích lỗi bài nộp. Frontend không gọi trực tiếp AI Server mà gửi request đến Backend. Backend kiểm tra xác thực, lấy ngữ cảnh hội thoại, gọi AI Server bằng internal token, lưu lịch sử và trả kết quả về Frontend.

Các API của backend xử lý hỏi đáp với AI bao gồm:

Bảng 4.4. Nhóm API backend chức năng hỏi đáp với chatbot AI

Chức năng	Phương thức	Endpoint
Gửi câu hỏi	POST	/api/ai/chat
Lấy lịch sử	GET	/api/ai/history

Dữ liệu đầu vào của API /api/ai/chat gồm message và lastCards. Trường message là câu hỏi hiện tại của người dùng. Trường lastCards chứa danh sách card đã được hiển thị trước đó, dùng để hỗ trợ các câu hỏi tiếp nối hoặc yêu cầu lọc lại gợi ý.

Các API trong AI Server dùng để phân tích câu hỏi từ người dùng bao gồm:

Bảng 4.5. Nhóm API chức năng hỏi đáp với chatbot AI của AI Server

Chức năng	Phương thức	Endpoint
AI sinh phản hồi	POST	/chat
AI phân tích ý định câu hỏi	POST	/chat/intent
AI chọn card phù hợp trong danh sách ứng viên	POST	/chat/select-cards

4.2.4. Xử lý nộp bài và chấm bài tự động

Nhóm xử lý nộp bài và chấm bài tự động tiếp nhận lời giải của người dùng, lưu bài nộp, đưa bài nộp vào hàng đợi và để Judge Worker xử lý bất đồng bộ.

Bảng 4.6. Nhóm API backend chức năng xử lý nộp bài và chấm bài tự động

Chức năng	Phương thức	Endpoint
Lấy danh sách thử thách	GET	/api/challenges
Lấy chi tiết thử thách	GET	/api/challenges/[id]
Nộp lời giải	POST	/api/submissions
Lấy kết quả bài nộp	GET	/api/submissions/
Lấy bảng xếp hạng thử thách	GET	/api/challenges/[id]/leaderboard
Nhận kết quả từ Judge Worker	POST	/api/submissions/result

Sau khi người dùng nộp bài, backend tạo bản ghi Submission và tạo job chấm bài rồi đưa vào BullMQ Queue, sau đó Judge Worker lấy job từ BullMQ Queue, xác định thông tin thử thách và bài nộp để tạo sandbox phù hợp.

- Với bài thuật toán, Worker chạy mã nguồn trong Docker Sandbox và so sánh output với expected output.
- Với bài SQL, Worker thực thi truy vấn trên cơ sở dữ liệu kiểm thử, so sánh kết quả trả về và rollback dữ liệu sau khi chấm.
- Với bài backend Node.js/API, Worker khởi chạy môi trường kiểm thử, gửi request đến endpoint cần đánh giá và kiểm tra response.

Cách xử lý bất đồng bộ giúp Backend không bị chặn trong thời gian chấm bài, đồng thời tăng khả năng xử lý khi có nhiều người dùng nộp bài cùng lúc. Việc sử dụng Docker Sandbox giúp hạn chế rủi ro khi thực thi mã nguồn do người dùng gửi lên.

Docker Sandbox được cấu hình với nhiều giới hạn an toàn. Container chạy bằng user riêng judgeuser, tắt quyền truy cập mạng, giới hạn bộ nhớ, giới hạn CPU, giới hạn số lượng process, loại bỏ toàn bộ Linux capabilities và chặn leo thang đặc quyền bằng no-new-privileges. Các cấu hình này giúp giảm khả năng bài nộp truy cập tài nguyên không cần thiết hoặc ảnh hưởng đến hệ thống chính.

Sau khi chấm xong, Worker tổng hợp kết quả gồm trạng thái bài nộp, kết quả testcase, điểm số, thời gian chạy, bộ nhớ sử dụng và thông báo lỗi nếu có. Kết quả được gửi về Backend thông qua API nội bộ kèm internal token. Kết quả nộp sẽ được thông báo đến Frontend để hiển thị.

Đồng thời, bài nộp sau khi chấm còn được đưa qua AI Server để phân tích lỗi. Kết quả phân tích lỗi được dùng để hỗ trợ người dùng hiểu nguyên nhân sai và phục vụ cho các chức năng cá nhân hóa sau này.

Bảng 4.7. Nhóm API phân tích lỗi bài nộp của AI Server

Chức năng	Phương thức	Endpoint
Phân tích lỗi bài nộp	POST	/api/ai/submission/analyze

Sau khi phân tích, hệ thống cập nhật mức độ thành thạo của người dùng theo từng topic trong bảng UserTopicMastery nhằm phục vụ mục đích cá nhân hóa người dùng.

Bảng 4.8. Quy tắc cập nhật năng lực topic sau khi chấm bài

Trạng thái bài nộp	mastery_score	weakness_score	solved_count
ACCEPTED	+0.08	-0.05	+1
Không accepted	-0.04	+0.08	+0

Nếu người dùng chưa có bản ghi năng lực ở topic đó, hệ thống tạo mới bản ghi với giá trị khởi tạo như sau:

Bảng 4.9. Giá trị khởi tạo năng lực topic

Trạng thái bài nộp	mastery_score ban đầu	weakness_score ban đầu
ACCEPTED	0.15	0.05
Không accepted	0.02	0.15

Các giá trị mastery_score và weakness_score được giới hạn trong khoảng từ 0 đến 1. Điều này giúp điểm năng lực không bị vượt quá miền giá trị hợp lệ trong quá trình cập nhật nhiều lần.

4.2.5. Xử lý báo cáo khiếu nại

Nhóm xử lý báo cáo khiếu nại cho phép người dùng gửi báo cáo đối với người dùng, bài viết hoặc bình luận có dấu hiệu vi phạm. Quản trị viên có thể xem danh sách báo cáo, kiểm tra chi tiết và cập nhật trạng thái xử lý.

Bảng 4.10. Nhóm API backend xử lý báo cáo khiếu nại

Chức năng	Phương thức	Endpoint
Tạo báo cáo	POST	/api/reports
Lấy danh sách báo cáo	GET	/api/admin/reports
Lấy chi tiết báo cáo	GET	/api/admin/reports/[id]
Xử lý báo cáo	PATCH	/api/admin/reports/[id]

Request tạo báo cáo gồm type, targetId, title, reason và severity. Trong đó type xác định đối tượng bị báo cáo, targetId là mã đối tượng, reason là lý do báo cáo và severity là mức độ nghiêm trọng.

4.2.6. Hệ thống cá nhân hóa người dùng

Hệ thống cá nhân hóa được xây dựng nhằm điều chỉnh nội dung hiển thị theo hồ sơ quan tâm và hoạt động của từng người dùng. Ở mức hiện tại, hệ thống được triển khai theo hướng rule-based recommendation, tập trung vào các dữ liệu như chủ đề quan tâm, bài viết đã tương tác, kết quả bài nộp, năng lực theo chủ đề và các gợi ý học tập được sinh ra sau quá trình phân tích lỗi bài nộp.

Bảng 4.11. Nhóm API chức năng gợi ý, cá nhân hóa người dùng

Chức năng	Phương thức	Endpoint
Lấy danh sách chủ đề quan tâm	GET	/api/interested-topics/me
Lấy danh sách bài viết gợi ý cho bạn	GET	/api/recommendations/posts
Lấy danh sách người dùng gợi ý	GET	/api/recommendations/users
Lấy danh sách thử thách gợi ý	GET	/api/recommendations/challenges

Luồng xử lý cá nhân hóa bắt đầu từ các hoạt động của người dùng trong hệ thống. Khi người dùng theo dõi một chủ đề, hệ thống lưu cặp user_id và topic_id vào bảng InterestedTopic. Khi người dùng bỏ theo dõi, hệ thống xóa bản ghi tương ứng dựa trên cặp khóa user_id_topic_id. Đây là nguồn dữ liệu thể hiện sở thích trực tiếp của người dùng.

Ngoài dữ liệu chủ đề quan tâm, hệ thống còn sử dụng dữ liệu phát sinh từ các chức năng khác. Khi người dùng lưu bài viết, reaction bài viết hoặc nộp bài cho thử thách lập trình, các dữ liệu này đều được dùng làm tín hiệu để xác định người dùng đang quan tâm hoặc còn yếu ở những topic nào. Vì vậy, phân hệ cá nhân hóa không hoạt

động tách rời mà sử dụng lại dữ liệu từ các phần bài viết, tương tác, chấm bài tự động và phân tích lỗi bằng AI.

Khi người dùng gọi các API gợi ý, Backend không lấy dữ liệu ngẫu nhiên mà trước tiên xây dựng hồ sơ chủ đề của người dùng. Hồ sơ này được tạo trong bước buildUserTopicProfile. Hệ thống lấy dữ liệu từ nhiều nguồn gồm topic đã theo dõi, năng lực topic, bài viết đã lưu, reaction bài viết và lịch sử nộp bài gần đây. Mỗi nguồn dữ liệu được cộng điểm vào topic tương ứng theo trọng số khác nhau.

Bảng 4.12. Trọng số xây dựng hồ sơ chủ đề người dùng

Nguồn dữ liệu	Cách tính điểm	Ý nghĩa
Topic người dùng theo dõi	+5	Chủ đề người dùng chọn trực tiếp
Bài viết đã lưu	+3	Chủ đề từ bài viết người dùng muốn lưu lại
Reaction bài viết	+2	Chủ đề từ bài viết người dùng đã tương tác
Bài nộp đã accepted	+1	Chủ đề người dùng đã luyện tập và làm đúng
Bài nộp chưa accepted	+3	Chủ đề người dùng cần luyện tập lại
Năng lực topic	$\text{weakness_score} * 6 + \max(0, 1 - \text{mastery_score}) * 2$	Ưu tiên topic người dùng còn yếu

Từ các điểm trên, hệ thống tạo ra danh sách weightedTopics. Mỗi phần tử trong danh sách gồm topicId, điểm score, danh sách nguồn dữ liệu sources và thông tin topic.

Danh sách này được sắp xếp giảm dần theo điểm. Các topic có điểm cao hơn sẽ được xem là quan trọng hơn trong quá trình gợi ý.

Điểm gợi ý bài viết:

$$\text{recommendationScore} = \text{topicScore} + \text{engagementScore} + \text{recencyScore}$$

Trong đó:

Bảng 4.13. Công thức tính điểm gợi ý bài viết

Thành phần	Công thức
topicScore	Tổng điểm các topic của bài viết trong hồ sơ người dùng
engagementScore	$\text{reactionCount} * 0.3 + \text{commentCount} * 0.5$
recencyScore	$\max(0, 3 - \text{số_tuần_từ_khi_đăng})$

Hệ thống ưu tiên lấy các bài viết có topic trùng với hồ sơ chủ đề của người dùng. Nếu số lượng bài viết khớp topic chưa đủ, hệ thống lấy thêm các bài viết mới khác làm fallback. Sau đó, hệ thống gộp hai nhóm bài viết, loại bỏ bài viết trùng id, tính điểm gợi ý và sắp xếp giảm dần theo recommendationScore.

Đối với gợi ý thử thách, hệ thống ưu tiên các challenge có topic nằm trong hồ sơ chủ đề của người dùng. Nếu chưa đủ số lượng ứng viên, hệ thống lấy thêm challenge khác làm fallback. Các challenge mà người dùng đã làm đúng có thể được loại khỏi danh sách gợi ý để tránh đề xuất lại nội dung đã hoàn thành.

Đối với gợi ý người dùng, hệ thống tính điểm theo công thức:

$$\text{recommendationScore} = \text{topicScore} + \text{activityScore} + \text{followerScore}$$

Trong đó:

Bảng 4.14. Công thức tính điểm gợi ý người dùng

Thành phần	Công thức
topicScore	Tổng điểm các topic của bài viết trong hồ sơ người dùng
activityScore	$\text{số_bài_viết_phù_hợp} * 0.8$
followerScore	$\text{min}(\text{số_người_theo_dõi}, 20) * 0.1$

Nhìn chung, luồng cá nhân hóa của hệ thống được triển khai như sau: người dùng đăng nhập vào hệ thống, phát sinh dữ liệu trong quá trình sử dụng như theo dõi topic, lưu bài viết, reaction, nộp bài và nhận phân tích lỗi. Backend tổng hợp các dữ liệu này thành hồ sơ chủ đề có trọng số. Từ hồ sơ đó, hệ thống tính điểm cho từng loại nội dung và trả về danh sách gợi ý đã sắp xếp cho Frontend. Cách triển khai này đơn giản, dễ kiểm soát và có thể giải thích được lý do gợi ý thông qua các nguồn dữ liệu cụ thể như topic đã theo dõi, bài viết đã lưu, reaction, bài nộp sai hoặc điểm yếu theo topic.

Chương 5. TỔNG KẾT

Chương 5 - chương cuối cùng sẽ tổng kết những kết quả đạt được, đánh giá những điểm mạnh và hạn chế của hệ thống. Đồng thời, nhóm sẽ đề xuất các hướng phát triển trong tương lai nhằm nâng cấp và mở rộng tính năng hệ thống để đáp ứng tốt hơn nhu cầu của người dùng.

5.1. Kết quả đạt được

Sau quá trình phân tích, thiết kế, xây dựng và kiểm thử, đồ án đã hoàn thiện được một nền tảng diễn đàn hỗ trợ lập trình có tích hợp hệ thống chấm bài tự động và chatbot hỗ trợ người dùng. Hệ thống được phát triển trên nền tảng web, gồm các thành phần chính là Frontend, Backend, AI Server, Judge Worker, cơ sở dữ liệu PostgreSQL, Redis/BullMQ và môi trường Docker Sandbox phục vụ quá trình chấm bài.

Về tổng thể, hệ thống đã đáp ứng được định hướng chính của Đồ án 2 là tiếp tục phát triển sản phẩm từ giai đoạn Đồ án 1, đồng thời mở rộng thêm các chức năng có tính ứng dụng thực tế hơn. Nếu ở giai đoạn trước hệ thống chủ yếu tập trung vào diễn đàn, chatbot cơ bản và thử thách lập trình ở mức đơn giản, thì trong Đồ án 2, hệ thống đã được phát triển theo hướng đầy đủ hơn với các chức năng: kiểm duyệt nội dung tự động bằng AI, cá nhân hóa nội dung người dùng, cải tiến chatbot, mở rộng Judge System cho nhiều dạng bài và tái cấu trúc giao diện người dùng.

5.1.1. Kết quả về chức năng diễn đàn và quản lý người dùng

Hệ thống đã xây dựng được các chức năng nền tảng của một diễn đàn hỗ trợ lập trình. Người dùng có thể đăng ký tài khoản, xác thực email bằng OTP, đăng nhập, đăng xuất và sử dụng các chức năng yêu cầu xác thực. Đây là nhóm chức năng cơ sở, giúp hệ thống xác định người dùng đang thao tác và lưu lại lịch sử hoạt động của từng tài khoản.

Đối với chức năng diễn đàn, hệ thống hỗ trợ người dùng tạo bài viết, xem danh sách bài viết, xem chi tiết bài viết, cập nhật bài viết và xóa mềm bài viết. Các bài viết được tổ chức theo chủ đề và danh mục, giúp người dùng dễ dàng theo dõi các nội dung liên

quan đến từng lĩnh vực lập trình. Ngoài ra, hệ thống còn hỗ trợ các thao tác tương tác như bình luận, reaction và lưu bài viết, tạo điều kiện cho người dùng trao đổi kiến thức và tham gia thảo luận trong cộng đồng.

Hệ thống cũng xây dựng các chức năng hồ sơ cá nhân, giúp người dùng quản lý thông tin cá nhân, xem các chỉ số hoạt động và theo dõi các nội dung liên quan đến tài khoản của mình. Đây là cơ sở để phát triển các chức năng nâng cao như tiến độ học tập, bảng xếp hạng và cá nhân hóa nội dung.

5.1.2. Kết quả về kiểm duyệt nội dung tự động bằng AI

Một trong những điểm mở rộng của Đề án 2 là bổ sung chức năng kiểm duyệt nội dung tự động bằng AI. Khi người dùng tạo bài viết hoặc gửi nội dung có hình ảnh, Backend có thể gửi dữ liệu sang AI Server để phân tích mức độ an toàn của nội dung.

Đối với nội dung văn bản, hệ thống gửi dữ liệu đến endpoint kiểm duyệt văn bản của AI Server. Đối với hình ảnh, hệ thống gửi file ảnh sang endpoint kiểm duyệt hình ảnh. AI Server trả về kết quả đánh giá thông qua trường thể hiện nội dung có an toàn hay không. Dựa trên kết quả này, Backend quyết định bài viết có thể được hiển thị, bị từ chối hoặc cần được xử lý thêm.

Chức năng này giúp giảm bớt khối lượng kiểm duyệt thủ công cho quản trị viên và góp phần hạn chế các nội dung không phù hợp xuất hiện trên diễn đàn. Tuy nhiên, hệ thống vẫn duy trì vai trò của quản trị viên trong việc theo dõi và xử lý các trường hợp cần đánh giá thủ công.

5.1.3. Kết quả về chatbot AI hỗ trợ người dùng

Hệ thống đã tích hợp chatbot AI nhằm hỗ trợ người dùng trong quá trình học tập, thảo luận và thực hành lập trình. Người dùng có thể gửi câu hỏi thông qua giao diện chatbot, sau đó Backend tiếp nhận request, kiểm tra xác thực, lấy ngữ cảnh cần thiết và gửi dữ liệu sang AI Server để sinh phản hồi.

Chatbot được xây dựng theo hướng hỗ trợ các câu hỏi liên quan đến lập trình, nội dung học tập, lỗi thường gặp và các chức năng trong hệ thống. Ngoài việc trả lời bằng văn bản, chatbot còn có khả năng phân tích ý định câu hỏi và lựa chọn các card phù hợp trong danh sách ứng viên để hỗ trợ người dùng chuyển đến bài viết, thử thách hoặc nội dung liên quan.

Bên cạnh chức năng hỏi đáp, AI Server còn được sử dụng để phân tích lỗi bài nộp. Sau khi người dùng nộp bài và hệ thống có kết quả chấm, bài nộp có thể được gửi sang AI Server để phân tích nguyên nhân lỗi và tạo gợi ý sửa lỗi. Kết quả phân tích này giúp người dùng hiểu rõ hơn vì sao bài làm chưa đạt, đồng thời tạo dữ liệu phục vụ cho quá trình cá nhân hóa học tập trong tương lai.

5.1.4. Kết quả về hệ thống chấm bài tự động

Hệ thống chấm bài tự động là một phân hệ trọng tâm của Đề án 2. Hệ thống cho phép người dùng xem danh sách thử thách lập trình, xem chi tiết đề bài, nộp lời giải, theo dõi trạng thái bài nộp, xem kết quả chấm chi tiết và theo dõi bảng xếp hạng.

So với giai đoạn trước, hệ thống chấm bài đã được mở rộng theo hướng hỗ trợ nhiều dạng bài hơn. Cụ thể, hệ thống hướng đến ba nhóm bài chính:

- Bài thuật toán truyền thống, trong đó hệ thống chạy mã nguồn với các test case và so sánh output thực tế với expected output.
- Bài SQL, trong đó hệ thống thực thi truy vấn trên cơ sở dữ liệu kiểm thử, so sánh kết quả trả về và rollback dữ liệu sau khi chấm.
- Bài backend Node.js/API, trong đó hệ thống khởi chạy môi trường kiểm thử, gửi request đến các endpoint cần đánh giá và kiểm tra response theo yêu cầu.

Về luồng xử lý, khi người dùng nộp bài, Backend tạo bản ghi Submission trong cơ sở dữ liệu, sau đó tạo job chấm bài và đưa vào BullMQ Queue. Redis được sử dụng để hỗ trợ quản lý hàng đợi. Judge Worker lấy job từ hàng đợi, xác định loại thử thách, chuẩn bị môi trường Docker Sandbox và tiến hành chấm bài. Sau khi chấm xong, Worker tổng hợp kết quả và gửi lại cho Backend để cập nhật trạng thái bài nộp.

Cách xử lý bất đồng bộ giúp Backend không bị chặn trong thời gian chờ bài, đồng thời phù hợp hơn khi số lượng bài nộp tăng. Việc sử dụng Docker Sandbox giúp cô lập môi trường chạy code của người dùng, giảm rủi ro ảnh hưởng đến hệ thống chính khi xử lý mã nguồn không tin cậy.

5.1.5. Kết quả về cá nhân hóa người dùng

Hệ thống đã xây dựng được cơ chế cá nhân hóa người dùng ở mức cơ bản theo hướng rule-based recommendation. Phân hệ này không sử dụng mô hình học máy phức tạp, mà dựa trên các luật tính điểm và trọng số được định nghĩa sẵn.

Dữ liệu dùng cho cá nhân hóa được lấy từ nhiều nguồn trong quá trình người dùng sử dụng hệ thống. Khi người dùng theo dõi topic, lưu bài viết, reaction bài viết, nộp bài hoặc nhận kết quả phân tích lỗi, các dữ liệu này có thể được sử dụng làm tín hiệu để xây dựng hồ sơ chủ đề của người dùng. Ngoài ra, hệ thống còn cập nhật mức độ thành thạo và điểm yếu của người dùng theo từng topic thông qua bảng UserTopicMastery.

Từ hồ sơ chủ đề có trọng số, hệ thống tạo các danh sách gợi ý như bài viết gợi ý, thử thách lập trình gợi ý, người dùng gợi ý phù hợp. Các nội dung được sắp xếp theo điểm gợi ý dựa trên mức độ phù hợp với topic người dùng quan tâm, hành vi tương tác, kết quả nộp bài và các tiêu chí phụ như độ mới hoặc mức độ hoạt động.

Cách tiếp cận này giúp hệ thống bước đầu điều chỉnh nội dung hiển thị theo từng người dùng thay vì chỉ hiển thị dữ liệu chung. Đồng thời, vì cơ chế gợi ý dựa trên luật và trọng số rõ ràng, hệ thống có thể giải thích lý do gợi ý thông qua các nguồn dữ liệu cụ thể như topic đã theo dõi, bài viết đã lưu, reaction, bài nộp sai hoặc điểm yếu theo topic.

5.1.6. Kết quả trò chuyện về thời gian thực

Hệ thống tiếp tục duy trì chức năng trò chuyện thời gian thực đã được thực hiện ở giai đoạn trước. Người dùng có thể tham gia cuộc trò chuyện, gửi và nhận tin nhắn với các người dùng khác trong hệ thống. Ngoài ra, người dùng còn có thể gọi điện

thời gian thực. Chức năng này hỗ trợ quá trình trao đổi kiến thức nhanh hơn, đặc biệt trong các trường hợp người dùng cần thảo luận trực tiếp về bài viết, thử thách lập trình hoặc vấn đề kỹ thuật.

Việc sử dụng Socket.IO giúp hệ thống hỗ trợ giao tiếp hai chiều giữa Client và Server. Khi có tin nhắn mới, Server có thể gửi dữ liệu đến các thành viên trong cuộc trò chuyện mà không cần người dùng tải lại trang hoặc liên tục gửi request kiểm tra dữ liệu.

5.1.7. Kết quả về giao diện người dùng

Frontend của hệ thống đã được tái cấu trúc và xây dựng bằng Svelte/SvelteKit. Các màn hình chính đã được thiết kế để phục vụ các nhóm chức năng của hệ thống, bao gồm đăng ký, đăng nhập, hồ sơ cá nhân, trang thảo luận, chi tiết bài viết, thử thách lập trình, chi tiết thử thách, nhắn tin, chatbot, học tập và các trang quản trị.

Giao diện được tổ chức theo hướng hiện đại, có bố cục rõ ràng và hỗ trợ hiển thị trên nhiều kích thước màn hình. Việc tái cấu trúc Frontend giúp mã nguồn dễ tổ chức theo component hơn, thuận tiện cho quá trình bảo trì và phát triển tiếp theo.

5.1.8. Đánh giá kiểm thử theo tiêu chí chất lượng

Sau khi hoàn thiện các chức năng chính, nhóm tiến hành kiểm thử một số tiêu chí chất lượng đã đặt ra trong phạm vi đề tài. Nội dung kiểm thử tập trung vào ba tiêu chí chính: khả năng xử lý người dùng đồng thời, thời gian chấm bài trung bình và độ chính xác của chức năng kiểm duyệt nội dung bằng AI.

5.1.8.1. Kiểm thử tải hệ thống

Đối với khả năng xử lý người dùng đồng thời, nhóm sử dụng công cụ k6 để mô phỏng 50 người dùng đồng thời truy cập vào các API thường dùng của hệ thống. Các API được chọn kiểm thử gồm danh sách bài viết, danh sách thử thách lập trình và dữ liệu gợi ý cá nhân hóa. Kết quả kiểm thử được đánh giá thông qua số lượng request, tỷ lệ request lỗi, thời gian phản hồi trung bình và thời gian phản hồi p95.

```

TOTAL RESULTS

checks_total.....: 5342    29.38656/s
checks_succeeded...: 99.45% 5313 out of 5342
checks_failed.....: 0.54%  29 out of 5342

X posts status 200
  ↳ 98% - ✓ 2642 / X 29
✓ challenges status 200

HTTP
http_req_duration.....: avg=574.57ms min=65.58ms med=339.23ms max=25.76s p(90)=845.77ms p(95)=999.79ms
{ expected_response:true }...: avg=521.69ms min=65.58ms med=334.84ms max=25.76s p(90)=834.69ms p(95)=978.46ms
http_req_failed.....: 0.54%  29 out of 5342
http_reqs.....: 5342    29.38656/s

EXECUTION
iteration_duration.....: avg=2.14s    min=1.32s    med=1.71s    max=26.9s    p(90)=2.14s    p(95)=2.42s
iterations.....: 2671    14.69328/s
vus.....: 1    min=1    max=50
vus_max.....: 50    min=50    max=50

NETWORK
data_received.....: 43 MB    238 kB/s
data_sent.....: 879 kB    4.8 kB/s

```

Hình 5.1. Kết quả kiểm thử tải 50 người dùng đồng thời bằng k6

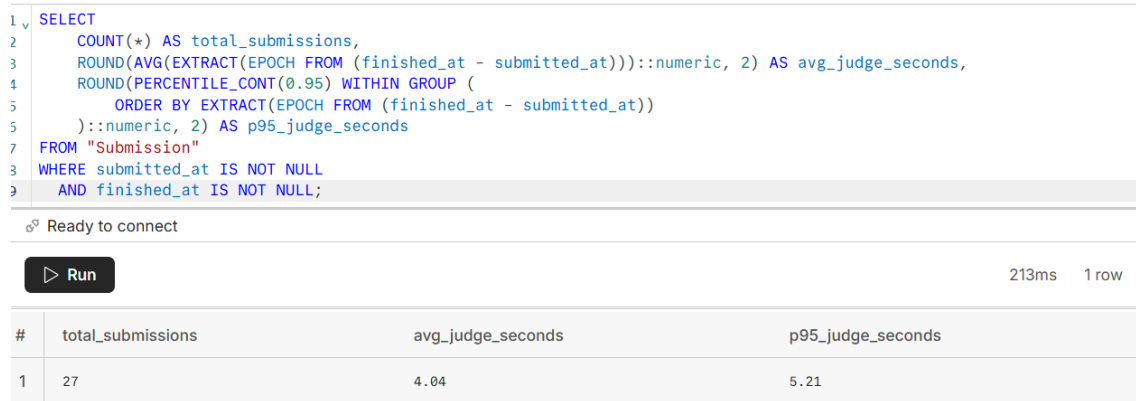
Bảng 5.1. Kết quả kiểm thử tải hệ thống bằng k6

Tiêu chí	Kết quả ghi nhận
Số người dùng đồng thời tối đa	50 VUs
Tổng số request	5342
Số lượt kiểm tra thành công	5313
Tỷ lệ kiểm tra thành công	99.45%
Tỷ lệ request lỗi	0.54%
Thời gian phản hồi trung bình	574.57 ms
Thời gian phản hồi trung vị	339.23 ms
p95 response time	999.79 ms
Nhận xét	Hệ thống đạt trong quá trình kiểm thử

Dựa trên kết quả kiểm thử tải bằng k6, với kịch bản cụ thể là gọi list posts và challenges, ta có thể kết luận rằng hệ thống có thể xử lý tối đa 50 người dùng ảo đồng thời trong thời gian kiểm thử mà không xảy ra crash.

5.1.8.2. Hiệu năng chấm bài

Đối với thời gian chấm bài tự động, nhóm đánh giá dựa trên dữ liệu được lưu trong bảng Submission. Thời gian chấm được tính từ thời điểm người dùng nộp bài đến thời điểm hệ thống hoàn tất chấm và cập nhật kết quả. Nhóm thực hiện truy vấn dữ liệu trên các bài nộp đã có kết quả để tính thời gian chấm trung bình và p95.



```

1 SELECT
2   COUNT(*) AS total_submissions,
3   ROUND(AVG(EXTRACT(EPOCH FROM (finished_at - submitted_at))))::numeric, 2) AS avg_judge_seconds,
4   ROUND(PERCENTILE_CONT(0.95) WITHIN GROUP (
5     ORDER BY EXTRACT(EPOCH FROM (finished_at - submitted_at))
6   ))::numeric, 2) AS p95_judge_seconds
7 FROM "Submission"
8 WHERE submitted_at IS NOT NULL
9    AND finished_at IS NOT NULL;

```

Ready to connect

Run 213ms 1 row

#	total_submissions	avg_judge_seconds	p95_judge_seconds
1	27	4.04	5.21

Hình 5.2. Kết quả truy vấn thời gian chấm bài trong cơ sở dữ liệu

Bảng 5.2. Kết quả đo thời gian chấm bài tự động

Tiêu chí	Kết quả ghi nhận
Tổng số bài nộp đã hoàn tất chấm	27
Thời gian chấm bài trung bình	4.04 giây
p95 thời gian chấm bài	5.21 giây
Ngưỡng đánh giá ban đầu	Dưới 10 giây
Kết quả đánh giá	Đạt

5.1.8.3. Độ chính xác của kiểm duyệt bằng AI

Đối với chức năng kiểm duyệt nội dung bằng AI, nhóm chuẩn bị một tập mẫu văn bản có nhãn mong đợi trước. Tập mẫu gồm nội dung hợp lệ về lập trình, câu hỏi học tập, nội dung spam và nội dung không phù hợp. Các mẫu được gửi đến AI Server để nhận kết quả kiểm duyệt thông qua trường `is_safe`. Sau đó, nhóm so sánh kết quả AI trả về với nhãn mong đợi để tính tỷ lệ dự đoán đúng.

```
$ node tests/ai/run-moderation-batch-eval.js
[dotenv@17.2.3] injecting env (14) from .env -- tip:
Total: 20
Correct: 20
Wrong: 0
Accuracy: 100.00%
Saved: tests/ai/moderation-results.json
(.venv)
```

Hình 5.3. Kết quả kiểm thử kiểm duyệt nội dung bằng AI

Công thức đánh giá độ chính xác kiểm duyệt được tính như sau:

$$\text{Accuracy} = \text{số mẫu dự đoán đúng} / \text{tổng số mẫu kiểm thử} \times 100\%$$

Kết quả được thống kê trong bảng sau:

Bảng 5.3. Kết quả kiểm thử kiểm duyệt bằng AI

Nhóm mẫu	Số mẫu	Số mẫu đúng	Số mẫu sai	Nhận xét
Nội dung lập trình hợp lệ	8	8	0	AI nhận diện đúng các nội dung trao đổi kỹ thuật.
Câu hỏi học tập hợp lệ	3	3	0	AI phân loại đúng các câu hỏi học tập.
Spam/quảng cáo	4	4	0	AI phát hiện đúng các nội dung spam.

Nội dung không phù hợp/độc hại	5	5	0	AI nhận diện đúng các nội dung công kích cá nhân.
Tổng	20	20	0	Độ chính xác kiểm duyệt trên tập mẫu đạt 100%.

Từ các kết quả trên, nhóm có cơ sở đánh giá bước đầu về khả năng vận hành của hệ thống sau khi hoàn thiện các chức năng chính. Tuy nhiên, phạm vi kiểm thử vẫn còn giới hạn về số lượng mẫu, số lượng bài nộp và kịch bản tải. Vì vậy, các kết quả này được sử dụng như đánh giá ban đầu; trong các giai đoạn tiếp theo, hệ thống cần tiếp tục được kiểm thử với dữ liệu lớn hơn và nhiều kịch bản thực tế hơn.

5.2. Ưu điểm của hệ thống

Sau quá trình xây dựng, hệ thống đã thể hiện được một số ưu điểm nổi bật cả về mặt chức năng lẫn tổ chức kỹ thuật.

Một trong những ưu điểm nổi bật mà ta phải kể đến là từ một diễn đàn lập trình thông thường, hệ thống đã xây dựng thành công diễn đàn lập trình đa chức năng, nơi mà người dùng không chỉ trao đổi kiến thức lập trình thông thường, mà họ còn có thể tham gia các thử thách lập trình đa dạng, sử dụng chatbot tối ưu việc lập trình, nhắn tin trực tiếp với nhau và học tập thông qua công cụ trực quan hóa thuật toán. Sự kết hợp này giúp hệ thống trở thành một nền tảng học tập và thực hành lập trình toàn diện.

Về kiến trúc, hệ thống được tổ chức tách biệt rõ ràng giữa Backend, AI Server và Judge Worker, giúp giảm tải cho Backend chính, đồng thời dễ mở rộng và bảo trì hơn khi cần nâng cấp từng thành phần.

Hệ thống chấm bài được xây dựng theo cơ chế bất đồng bộ thông qua hàng đợi BullMQ, giúp Backend phản hồi nhanh cho người dùng và hạn chế nghẽn request khi có nhiều bài nộp cùng lúc. Bên cạnh đó, Docker Sandbox được sử dụng để cô lập môi

trường thực thi mã nguồn, đảm bảo an toàn cho hệ thống khi xử lý code từ người dùng, đặc biệt với bài SQL còn có cơ chế rollback dữ liệu sau khi chấm.

Điểm mạnh khác là hệ thống hỗ trợ đa dạng dạng bài gồm thuật toán, SQL và backend Node.js, phù hợp hơn với nhu cầu thực tế của người học lập trình. AI cũng được tích hợp vào nhiều nghiệp vụ như hỏi đáp, kiểm duyệt nội dung và phân tích lỗi bài nộp, thay vì chỉ là một tính năng phụ đơn lẻ.

Cuối cùng, phân hệ cá nhân hóa được xây dựng theo hướng rule-based dựa trên dữ liệu hành vi người dùng như topic theo dõi, bài viết đã lưu, lịch sử nộp bài và mức độ thành thạo, giúp hệ thống đưa ra gợi ý phù hợp và dễ kiểm soát trong phạm vi đề án.

5.3. Hạn chế

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế cần tiếp tục cải thiện.

Về cơ chế cá nhân hóa, hệ thống đã xây dựng rule-based recommendation dựa trên nhiều tín hiệu hành vi thực tế như reaction, bình luận, bài viết đã lưu và lịch sử nộp bài, tuy nhiên trọng số vẫn được thiết lập thủ công nên kết quả gợi ý có thể chưa tối ưu khi dữ liệu người dùng tăng lên.

Về chatbot AI, chất lượng phản hồi phụ thuộc vào ngữ cảnh câu hỏi và khả năng xử lý của AI Server, Với các câu hỏi quá phức tạp, thiếu ngữ cảnh hoặc có lỗi lập trình khó phân tích, chatbot có thể trả lời chưa đầy đủ hoặc chưa sát với tình huống của người dùng, đồng thời hệ thống còn phụ thuộc vào giới hạn request và độ ổn định của nhà cung cấp dịch vụ AI bên ngoài.

Về kiểm duyệt nội dung, AI có thể hỗ trợ phát hiện nội dung vi phạm nhưng chưa thể thay thế hoàn toàn quản trị viên do một số trường hợp phụ thuộc nhiều vào ngữ cảnh và đặc thù cộng đồng, vì vậy vẫn cần kết hợp giữa tự động hóa và giám sát thủ công.

Về Judge System, mỗi dạng bài vẫn có những khó khăn riêng như độ bao phủ test case với bài thuật toán, xử lý thứ tự và định dạng kết quả với bài SQL, hay cấu hình

endpoint và expected response với bài backend, cần tiếp tục chuẩn hóa để đạt độ ổn định cao hơn.

Về sandbox, Docker đã giúp cô lập môi trường chạy bài nộp nhưng vẫn cần kiểm soát chặt hơn về giới hạn tài nguyên, quyền truy cập và điều phối số lượng container khi lượng bài nộp tăng cao.

Cuối cùng, giao diện người dùng ở một số màn hình như xem kết quả chấm, theo dõi tiến độ và sử dụng chatbot vẫn cần tối ưu thêm để trực quan và dễ sử dụng hơn, đặc biệt đối với người mới học lập trình.

5.4. Hướng phát triển

Dựa trên kết quả đã đạt được và các hạn chế còn tồn tại, hệ thống có thể tiếp tục được phát triển theo hướng hoàn thiện độ ổn định của các chức năng hiện có, mở rộng khả năng hỗ trợ người dùng và nâng cao khả năng vận hành trong môi trường thực tế.

Trước hết, hệ thống chấm bài tự động cần được tiếp tục tối ưu để xử lý ổn định hơn khi có nhiều bài nộp cùng lúc. Các bộ test case cho bài thuật toán, SQL và backend Node.js/API cần được chuẩn hóa rõ hơn, đặc biệt là phần so sánh kết quả SQL, rollback dữ liệu sau khi chấm và kiểm thử nhiều endpoint đối với bài backend.

Bên cạnh đó, môi trường Docker Sandbox cần tiếp tục được kiểm soát và hoàn thiện hơn ở mức vận hành. Hiện tại, hệ thống đã có một số cơ chế bảo vệ như chạy container bằng user riêng, tắt quyền truy cập mạng, giới hạn bộ nhớ, giới hạn CPU... Tuy nhiên, trong các phiên bản tiếp theo, hệ thống vẫn cần bổ sung thêm cơ chế theo dõi tài nguyên, ghi log các bài nộp bất thường, dọn dẹp container ... giúp hệ thống ổn định hơn khi có nhiều bài nộp và giảm rủi ro khi xử lý mã nguồn do người dùng gửi lên.

Đối với các chức năng AI, hệ thống cần thiết kế để cải thiện chất lượng phản hồi theo ngữ cảnh của hệ thống và ít phụ thuộc vào dịch vụ AI bên ngoài hơn. Đồng thời, hệ thống cần kiểm chứng và điều chỉnh các trọng số của chức năng cá nhân hóa dựa trên dữ liệu sử dụng thực tế. Khi dữ liệu đủ lớn, hệ thống có thể nghiên cứu thêm các

phương pháp gợi ý nâng cao hơn như collaborative filtering hoặc embedding-based recommendation.

Ngoài các thử thách lập trình riêng lẻ, hệ thống có thể phát triển thêm chức năng contest hoàn chỉnh. Chức năng này cần hỗ trợ tạo cuộc thi, thiết lập thời gian bắt đầu và kết thúc, quản lý danh sách bài thi, ghi nhận bài nộp trong phạm vi cuộc thi, tính điểm và hiển thị bảng xếp hạng theo contest. Đây là hướng phát triển phù hợp để hệ thống phục vụ tốt hơn các hoạt động luyện tập và thi lập trình trong cộng đồng.

5.5. Nguồn kỹ thuật và khảo sát

Trong quá trình thực hiện đồ án, nhóm sử dụng các nguồn kỹ thuật và khảo sát sau để phục vụ cho việc phân tích yêu cầu, thiết kế hệ thống, xây dựng chức năng và kiểm thử sản phẩm.

- Đường dẫn đến mã nguồn dự án trên GitHub: [Link GitHub](#).
- Đường dẫn đến kết quả khảo sát: [Link kết quả khảo sát](#)
- Đường dẫn đến báo cáo đồ án 1: [DoAn1](#).
- Đường dẫn đến tài liệu đặc tả Usecase: [TongHopDacTaUseCase.pdf](#)
- Đường dẫn đến tài liệu mô tả chi tiết cơ sở dữ liệu:
[TaiLieuMoTaChiTietCSDL](#)

Các nguồn trên được sử dụng xuyên suốt trong quá trình thực hiện đồ án, từ giai đoạn phân tích yêu cầu, thiết kế hệ thống đến cài đặt, kiểm thử và hoàn thiện sản phẩm.

TÀI LIỆU THAM KHẢO

- [1] J. Zhang, M. S. Ackerman, and L. Adamic, “Expertise networks in online communities: Structure and algorithms,” in **Proc. 16th International Conference on World Wide Web (WWW)**, 2007, pp. 221–230.
- [2] HackerRank, **2024 Developer Skills Report**. HackerRank, 2024.
- [3] MDN Web Docs, “HTTP request methods,” Mozilla Developer Network, 2025. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>. [Accessed: Jun. 6, 2026].
- [4] Docker Inc., “Container security FAQs,” Docker Documentation, 2026. [Online]. Available: <https://docs.docker.com/security/faqs/containers/>. [Accessed: Jun. 6, 2026].
- [5] Z. Wan, D. Lo, X. Xia, L. Cai, and S. Li, “Mining sandboxes for Linux containers,” **arXiv preprint arXiv:1712.05493**, 2017.
- [6] A. S. Tanenbaum and D. J. Wetherall, **Computer Networks**, 5th ed. Boston, MA, USA: Pearson, 2011.
- [7] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.
- [8] MDN Web Docs, “HTTP request methods,” Mozilla Developer Network, 2025. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>. [Accessed: Jun. 6, 2026].
- [9] SvelteKit, “Routing,” SvelteKit Documentation, 2026. [Online]. Available: <https://sveltejs-kit.mintlify.app/core-concepts/routing>. [Accessed: Jun. 6, 2026].
- [10] OpenJS Foundation, “About Node.js,” Node.js Documentation, 2026. [Online]. Available: <https://nodejs.org/en/about>. [Accessed: Jun. 6, 2026].
- [11] Express.js, “Routing,” Express Documentation, 2026. [Online]. Available: <https://expressjs.com/en/guide/routing.html>. [Accessed: Jun. 6, 2026].

- [12] Express.js, “Using middleware,” Express Documentation, 2026. [Online]. Available: <https://expressjs.com/en/guide/using-middleware>. [Accessed: Jun. 6, 2026].
- [13] Python Software Foundation, “The Python Tutorial,” Python Documentation, 2025. [Online]. Available: <https://docs.python.org/3.9/tutorial/>. [Accessed: Jun. 6, 2026].
- [14] Prisma, “Prisma ORM,” Prisma Documentation, 2026. [Online]. Available: <https://www.prisma.io/docs/orm>. [Accessed: Jun. 6, 2026].
- [15] PostgreSQL Global Development Group, “About PostgreSQL,” PostgreSQL Documentation, 2026. [Online]. Available: <https://www.postgresql.org/about/>. [Accessed: Jun. 6, 2026].
- [16] Docker Inc., “Docker documentation,” Docker Documentation, 2026. [Online]. Available: <https://docs.docker.com/>. [Accessed: Jun. 6, 2026].
- [17] Socket.IO, “Socket.IO: Bidirectional and low-latency communication for every platform,” Socket.IO Documentation, 2026. [Online]. Available: <https://socket.io/>. [Accessed: Jun. 6, 2026].
- [18] BullMQ, “Queues,” BullMQ Documentation, 2026. [Online]. Available: <https://docs.bullmq.io/guide/queues>. [Accessed: Jun. 6, 2026].
- [19] Redis, “Keys and values,” Redis Documentation, 2026. [Online]. Available: <https://redis.io/docs/latest/develop/using-commands/keyspace/>. [Accessed: Jun. 6, 2026].
- [20] Redis, “TTL,” Redis Documentation, 2026. [Online]. Available: <https://redis.io/docs/latest/commands/ttl/>. [Accessed: Jun. 6, 2026].
- [21] Grafana Labs, “Grafana k6,” Grafana k6 Documentation, 2026. [Online]. Available: <https://grafana.com/docs/k6/latest/>. [Accessed: Jun. 6, 2026].